

Interaktivní nástroj pro vizualizaci algoritmů a datových struktur

Interactive Tool for Visualizing Algorithms and Data Structures

Barbora Kovalská

Diplomová práce

Vedoucí práce: Ing. Radoslav Fasuga, Ph.D.

Ostrava, 2026

Zadání diplomové práce

Student:

Bc. Barbora Kovalská

Studijní program:

N0613A140034 Informatika

Téma:

Interaktivní nástroj pro vizualizaci algoritmů a datových struktur
Interactive Tool for Visualizing Algorithms and Data Structures

Jazyk vypracování:

čeština

Zásady pro vypracování:

Cílem projektu je vytvořit webový nástroj pro vizualizaci datových struktur (např. zásobník, fronta, seznamy, stromy) a simulaci algoritmů (třídící a vyhledávací algoritmy). Aplikace bude sloužit jako výukový prostředek, umožňující interaktivní operace a teoretické vysvětlení jednotlivých datových struktur a algoritmů.

1. Student se seznámí s problematikou datových struktur a základních algoritmů.
2. Student provede analýzu existujících výukových nástrojů zaměřených na vizualizaci algoritmů a navrhne vlastní specifikace pro webovou aplikaci.
3. Student navrhne a implementuje interaktivní webovou aplikaci s přívětivým uživatelským rozhraním. Aplikace bude obsahovat moduly pro vizualizaci základních datových struktur a algoritmů, interaktivní simulaci operací nad strukturami s možností sledování v reálném čase a také teoretické vysvětlení daných postupů.
4. Výsledkem bude plně funkční prototyp webové aplikace a dokumentace popisující použití a implementaci nástroje.
5. Srovnávejte výslednou aplikaci s existujícími aplikacemi.

Seznam doporučené odborné literatury:

- [1] LEVITIN, Anany. Introduction to the design and analysis of algorithms. Third edition. Boston: Pearson, c2012. ISBN 978-0132316811.
- [2] CORMEN, Thomas H.; LEISERSON, Charles Eric; RIVEST, Ronald L. a STEIN, Clifford. Introduction to algorithms. Fourth edition. Cambridge, Massachusetts: The MIT Press, [2022]. ISBN 9780262046305.
- [3] SEDGEWICK, Robert a WAYNE, Kevin Daniel. ****Algorithms****. Fourth edition. Boston: Addison-Wesley, [2011]. ISBN 978-0-321-57351-3
- [4] Gonzalez, V. (2020). JavaScript Data Structures and Algorithms: An Introduction to Solving Complex Problems Using JavaScript. Packt Publishing.
- [5] Bamford, K., & Ruiz, D. (2022). Frontend Architecture for Design Systems: A Modern Guide to UI Development. O'Reilly Media.

Formální náležitosti a rozsah diplomové práce stanoví pokyny pro vypracování zveřejněné na webových stránkách fakulty.

Vedoucí diplomové práce: **Ing. Radoslav Fasuga, Ph.D.**

Datum zadání: 01.09.2025

Datum odevzdání: 30.04.2026

Garant studijního programu: prof. RNDr. Václav Snášel, CSc.

V IS EDISON zadáno: 09.10.2025 14:34:54

Abstrakt

Hlavním cílem této diplomové práce je návrh a implementace interaktivní webové výukové aplikace zaměřené na vizualizaci datových struktur a simulaci třídících algoritmů. Práce se v teoretické části věnuje popisu teorie obsahu aplikace, analýze existujících řešení a výběru vhodných moderních webových technologií. Hlavní část práce podrobně popisuje softwarovou architekturu systému, která striktně odděluje aplikační logiku od prezentační vrstvy pomocí událostmi řízeného modelu a generování předpočítaných stavů. Výsledkem je plně responzivní a modulární výuková aplikace, která studentům umožňuje krok za krokem zkoumat chování algoritmů, interaktivně manipulovat se strukturami a plynule propojovat teoretické znalosti s vizuální reprezentací pseudokódu.

Klíčová slova

vizualizace algoritmů; datové struktury; třídící algoritmy; výuková webová aplikace; SvelteKit; TypeScript; softwarová architektura; e-learning

Abstract

The main objective of this Master's thesis is the design and implementation of an interactive web-based educational application focused on visualizing data structures and simulating sorting algorithms. The theoretical part examines the educational content, existing solutions, and the selection of suitable modern web technologies. The core of the thesis details the software architecture of the system, which strictly separates application logic from the presentation layer using an event-driven model and the generation of precomputed states. The result is a fully responsive and modular educational application that allows students to explore algorithm behavior step-by-step, interactively manipulate structures, and seamlessly connect theoretical knowledge with the visual representation of pseudocode.

Keywords

algorithm visualization; data structures; sorting algorithms; educational web application; SvelteKit; TypeScript; software architecture; e-learning

Poděkování

Ráda bych na tomto místě poděkovala svému vedoucímu Ing. Radoslavu Fasugovi, Ph.D. za odborné vedení a trpělivost během celého procesu psaní této diplomové práce. Taktéž bych chtěla poděkovat svému příteli, rodině a přátelům za jejich neustálou podporu a motivaci, která mi pomohla překonat náročné chvíle během studia.

Obsah

Seznam použitých symbolů a zkratk	8
Seznam obrázků	9
Seznam tabulek	11
Seznam výpisů zdrojového kódu	12
1 Úvod	13
2 Teorie	15
2.1 Datové struktury	15
2.2 Třídící algoritmy	27
2.3 Význam vizualizace ve vzdělávání	35
3 Analýza existujících řešení	36
3.1 Vizualizace datových struktur	36
3.2 Simulace třídících algoritmů	39
3.3 Shrnutí	43
4 Požadavky na výukovou aplikaci	44
4.1 Funkcionalita	44
4.2 Použitelnost	45
4.3 Spolehlivost	45
4.4 Výkonnost	46
4.5 Podpořitelnost a udržitelnost	46
5 Analýza a volba použitých technologií	47
5.1 Architektura aplikace	47
5.2 Strategie renderování	48
5.3 Webový framework	50

5.4	Grafová knihovna	52
5.5	Automatizace a nasazení (CI/CD)	54
5.6	Shrnutí	55
6	Návrh aplikace	57
6.1	Informační architektura a navigace	57
6.2	Uživatelské rozhraní a UX Design	59
6.3	Návrh vizualizačních komponent	62
6.4	Vizuální styl a sémantika barev	64
6.5	Koncepтуální model řízení simulace	66
6.6	Datový model a abstrakce	67
7	Implementace	70
7.1	Celková architektura a modularita systému	70
7.2	Implementační jádro třídících algoritmů	71
7.3	Implementační jádro datových struktur	74
7.4	Vizualizační engine a řešení technologických omezení	76
7.5	Automatizace nasazení	79
7.6	Uživatelské testování a iterativní vývoj	79
7.7	Uživatelské rozhraní a další funkcionality	80
8	Závěr	85
	Literatura	87

Seznam použitých zkratek a symbolů

VŠB	– Vysoká škola Báňská
LIFO	– Last In, First Out
FIFO	– First In, First Out
BST	– Binární vyhledávací strom
SPA	– Single Page Application
MPA	– Multi-Page Application
TTFB	– Time To First Byte
DDD	– Domain-Driven Design
MVC	– Model-View-Controller
API	– Application Programming Interface
AJAX	– Asynchronous JavaScript and XML
JSON	– JavaScript Object Notation
HTML	– Hypertext Markup Language
YAML	– Yet Another Markup Language
URL	– Uniform Resource Locator
HTTP	– Hypertext Transfer Protocol
DNS	– Domain Name System
CNAME	– Canonical Name
SEO	– Search Engine Optimization
CSR	– Client-Side Rendering
SSR	– Server-Side Rendering
SSG	– Static Site Generation
UI	– User Interface
UX	– User eXperience
DOM	– Document Object Model
SVG	– Scalable Vector Graphics
CSS	– Cascading Style Sheets
CI/CD	– Continuous Integration / Continuous Deployment

Seznam obrázků

2.1	Pole a spojový seznam s n prvky	16
2.2	Zásobník a fronta	16
2.3	Binární vyhledávací strom	17
2.4	AVL strom	18
2.5	Pravá (R) a dvojitá pravá-levá (RL) rotace AVL stromu	19
2.6	Červeno-černý strom	20
2.7	Oprava červeno-černého stromu po vložení prvku, případ 1	21
2.8	Oprava červeno-černého stromu po vložení prvku, případ 2	21
2.9	B-strom řádu 3	22
2.10	Vložení klíče do B-stromu řádu 3	23
2.11	Binární halda reprezentovaná stromem	25
2.12	Binární halda reprezentovaná polem	25
5.1	Srovnání životních cyklů stránek v klasické architektuře a v SPA	48
5.2	Srovnání strategií renderování CSR a SSR	49
5.3	Příklad zobrazení stromu pomocí D3.js	52
5.4	Příklad zobrazení stromu pomocí Cytoscape.js	53
5.5	Příklad zobrazení stromu pomocí vis-network	54
6.1	Sitemap aplikace, znázorňující hierarchii stránek a jejich vzájemné propojení	58
6.2	Rozvržení stránek aplikace	59
6.3	Návrh vizualizace datové struktury	60
6.4	Návrh vizualizace třídícího algoritmu	60
6.5	Návrh vizualizace detailu třídícího algoritmu	61
6.6	Návrh obecné vizualizace pole	63
6.7	Návrh detailní vizualizace pole	63
6.8	Barevná paleta výukové aplikace	64
6.9	Třídy pro reprezentaci datových struktur	68
6.10	Třídy pro reprezentaci třídících algoritmů	69

7.1	Uživatelské rozhraní pro interaktivní vizualizaci datových struktur	80
7.2	Obecný přehled třídícího algoritmu s využitím sloupcové vizualizace	81
7.3	Detailní pohled rekursivního algoritmu Quick Sort	81
7.4	Ukázka responzivního chování na mobilním zařízení	82
7.5	Podpora tmavého režimu s plným zachováním kontrastu a sémantiky barev	83
7.6	Komparační zobrazení více třídících algoritmů současně	84

Seznam tabulek

2.1	Srovnání vlastností a časové složitosti operací datových struktur	26
2.2	Srovnání vlastností a složitosti třídících algoritmů	35
3.1	Porovnání analyzovaných aplikací pro vizualizaci datových struktur	39
3.2	Podporované datové struktury v analyzovaných aplikacích	39
3.3	Porovnání analyzovaných aplikací pro vizualizaci třídících algoritmů	42

Seznam výpisů zdrojového kódu

1	Pseudokód pomocné funkce pro výměnu dvou prvků	27
2	Pseudokód algoritmu Bubble Sort	28
3	Pseudokód algoritmu Selection Sort	29
4	Pseudokód algoritmu Insertion Sort	30
5	Pseudokód algoritmu Merge Sort	31
6	Pseudokód algoritmu Quick Sort	32
7	Pseudokód algoritmu Heap Sort	34
8	Příklad konfigurace GitHub Actions pro CI/CD	55
9	Struktura adresářů a souborů implementace	71
10	Zjednodušená implementace časovače a stavů ve třídě StepManager	72
11	Implementace metod paint a record ve třídě DetailedTraceBuilder	73
12	Řízení dvouúrovňového krokování a emitování událostí ve třídě OperationManager	75
13	Abstraktní rozhraní StepHandlerBase definující životní cyklus vizualizace kroku	76
14	Napojení vlastní vrstvy DataStructureAnnotator na událost vykreslení sítě	78

Kapitola 1

Úvod

Pochopení datových struktur a algoritmů představuje důležitý základ pro každého studenta informatiky. Tyto koncepty formují analytické myšlení budoucích softwarových inženýrů a jsou nezbytné pro návrh efektivních a škálovatelných programů. Přesto se jedná o oblast, která studentům tradičně působí obtíže; algoritmy jsou ze své podstaty dynamické procesy plné abstraktních stavů a manipulací s pamětí, které je obtížné plně pochopit pouze ze statických textů, skript či kreseb na tabuli.

Ačkoliv v dnešní době existuje na internetu řada vizualizačních nástrojů, studenti i vyučující často naráží na problémy: roztržitost a nekompatibilitu. Mnoho existujících aplikací je zastaralých, chybí jim responzivita, nepodporují český jazyk, nebo používají definice a pseudokódy, které se drobně či dokonce zásadně liší od materiálů probíraných na konkrétních univerzitách. Tato nejednotnost má za následek stav, kdy studenti musí neustále přepínat mezi různými zdroji a porovnávat jejich informace se skripty, a to je spíše matoucí než prospěšné pro jejich učení. Chybí zde moderní, sjednocující výuková aplikace, která by byla nejen vizuálně atraktivní, ale především by mohla být oficiálně integrována do výuky a sladěna s konkrétními univerzitními skripty.

Hlavním cílem této diplomové práce je proto návrh a implementace interaktivní výukové webové aplikace, která tento problém řeší. Aplikace si klade za cíl poskytnout studentům vizuální a intuitivní prostředí pro zkoumání datových struktur a třídících algoritmů. Nabízí unikátní kombinaci pohledů – interaktivní manipulaci se stromy, obecné vizualizace řazení prvků nebo detailní krokování rekursivních algoritmů s vazbou na dynamický pseudokód, což je kombinace funkcionalit, která u mnoha existujících řešení chybí.

Aplikace je primárně cílena na studenty univerzitních oborů informatiky jakožto podpůrný nástroj k přednáškám a cvičením. Její moderní uživatelské rozhraní a přístupnost jsou navrženy tak, aby bariéra vstupu byla co nejnižší. Sloužit tak může i středoškolským studentům, samoukům nebo komukoliv, kdo má zájem rozšířit své porozumění fungování základních počítačových algoritmů.

Text práce je logicky členěn do několika kapitol. Po úvodu následuje Kapitola 2, která shrnuje teoretické základy implementovaných datových struktur a algoritmů. Kapitola 3 nabízí rešerši a

kritickou analýzu existujících vizualizačních řešení na trhu. Na základě zjištěných nedostatků jsou v Kapitole 4 specifikovány požadavky na novou aplikaci. Kapitola 5 se věnuje výběru vhodných moderních webových technologií a strategií renderování. Kapitola 6 detailně popisuje softwarový návrh, informační architekturu a uživatelské rozhraní, zatímco Kapitola 7 rozebírá samotnou implementaci, architektonická řešení a překonávání technologických limitů. Práce je zakončena Kapitolou 8, která shrnuje dosažené výsledky a nastiňuje možnosti budoucího rozvoje aplikace.

Kapitola 2

Teorie

V této kapitole jsou popsány teoretické základy týkající se obsahu aplikace. Nejprve je uveden přehled různých datových struktur, které aplikace podporuje, včetně jejich vlastností, operací a požadavků na vizualizaci. Dále jsou diskutovány třídící algoritmy, které budou implementovány, a jejich význam pro pochopení základních konceptů informatiky. Nakonec je popsán přístup k vizualizaci těchto datových struktur a algoritmů, včetně použitých technik a metod pro efektivní zobrazení a interakci s uživatelem.

2.1 Datové struktury

V této kapitole jsou popsány datové struktury, které jsou podstatné pro oblast informatiky. Každá datová struktura je charakterizována svými vlastnostmi, operacemi a požadavky na vizualizaci.

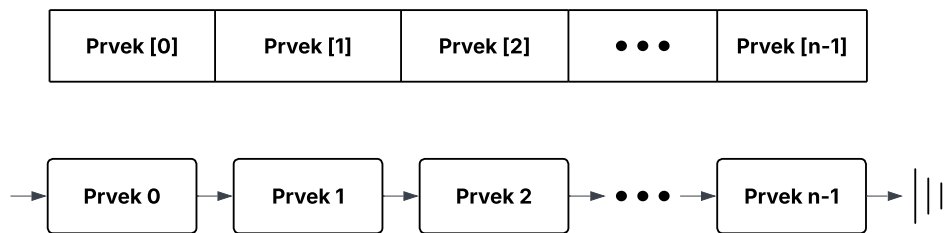
2.1.1 Lineární datové struktury

Pro účely výuky datových struktur je rozumné začít s lineárními datovými strukturami, které jsou jednodušší na pochopení a vizualizaci. Tyto struktury zahrnují mimo jiné pole, spojové seznamy, zásobníky a fronty.

I přesto, že tyto struktury jsou relativně jednoduché a již moc dobře známé, v této práci budou teoreticky uvedeny, aby byla zajištěna úplnost a konzistence s dalšími částmi práce.

Pole a spojový seznam

Základním stavebním kamenem lineárních datových struktur jsou pole a spojové seznamy. Pole (*Array*) poskytují rychlý přístup k prvkům pomocí indexů, zatímco spojové seznamy (*Linked List*) umožňují dynamickou alokaci paměti a efektivní vkládání a mazání prvků pomocí ukazatelů. Obrázek 2.1 znázorňuje rozdíl mezi těmito dvěma strukturami. [1]



Obrázek 2.1: Pole a spojový seznam s n prvky

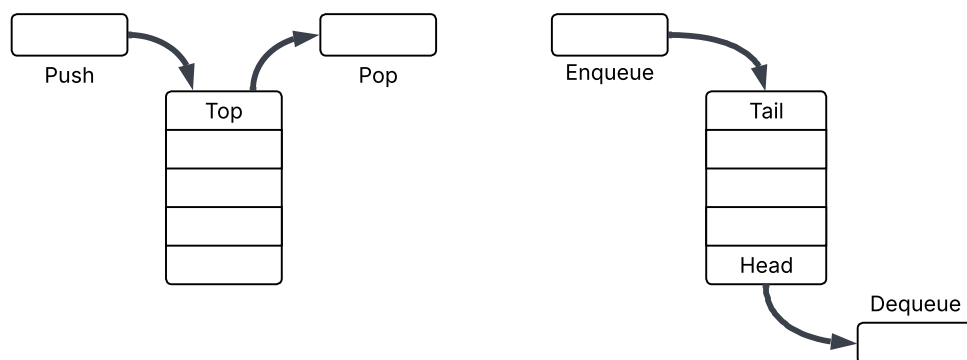
Spojový seznam je tvořen uzly, kde každý uzel obsahuje datovou část a ukazatel na další uzel v seznamu. V případě obousměrného spojového seznamu obsahuje každý uzel také ukazatel na předchozí uzel. Tato vlastnost umožňuje obousměrnou navigaci. Seznam musí mít ukazatel na první uzel (*head*) a často také na poslední uzel (*tail*) pro efektivní přidávání prvků na konec seznamu. [2]

Spojový seznam i pole se využívají pro implementaci abstraktnějších seznamů, které obsahují sekvenci prvků a poskytují operace pro přidávání, odebrání a přístup k prvkům. [1]

Zásobník a fronta

Speciálním případem lineárních seznamů jsou zásobníky (*Stack*) a fronty (*Queue*), jejichž vizualizaci je možné vidět na Obrázku 2.2. Jejich hlavní charakteristikou je předem specifikovaný způsob mazání a přidávání prvků.

Zásobník funguje na principu LIFO (*Last In, First Out*), poslední prvek přidáný do zásobníku je z něj tedy odebrán jako první. Operace přidání prvku se běžně nazývá *push* a operace odebrání prvku se nazývá *pop*. V zásobníku je přístup k prvkům omezen na vrchol zásobníku, jenž se označuje jako *top*. [2]



Obrázek 2.2: Zásobník a fronta

Naopak fronta funguje na principu FIFO (*First In, First Out*), kde první prvek přidaný do fronty je prvním, který je z ní odebrán. Operace přidání prvku se nazývá *enqueue* a operace odebrání prvku se nazývá *dequeue*. Fronta se implementuje pomocí ukazatele, který sleduje začátek (*head*) fronty, a často bývá doplněna o ukazatel na konec (*tail*) fronty. Přítomnost obou ukazatelů umožňuje efektivní přidávání a odebírání prvků. [2]

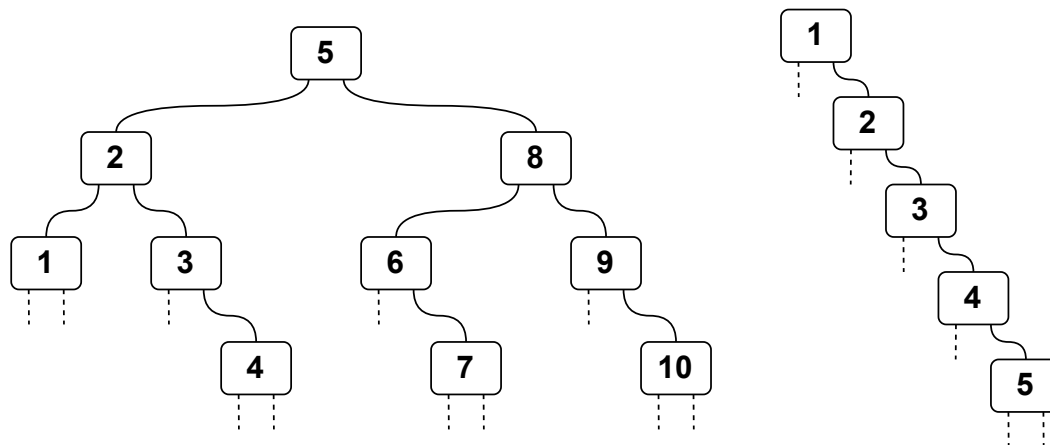
Obě struktury mohou být efektivně implementovány pomocí spojových seznamů nebo polí, v závislosti na požadavcích na výkon a paměťovou efektivitu. [1]

2.1.2 Nelineární (stromové) datové struktury

Mezi nelineární datové struktury patří stromy, hierarchické struktury představující spojitý acyklický graf, kde každý prvek (uzel) může mít více potomků. Stromy obecně podporují operace **vkládání**, **mazání** a **vyhledávání** prvků, přičemž složitost těchto operací závisí na konkrétním typu stromu a jeho struktuře. Tyto struktury se často používají pro reprezentaci hierarchických dat, jako jsou souborové systémy, organizační struktury nebo různé druhy indexů v databázích. V následujících podkapitolách jsou popsány různé typy stromů, které budou implementovány v aplikaci, včetně jejich vlastností, operací a požadavků na vizualizaci.

Binární vyhledávací strom

Binární vyhledávací stromy (*Binary Search Trees, BST*) jsou speciálním typem binárních stromů, kde každý uzel splňuje následující vlastnost: klíče v levém podstromu jsou menší než klíč v uzlu, a klíče v pravém podstromu jsou větší než klíč v uzlu. Tato vlastnost umožňuje efektivní vyhledávání, vkládání a mazání prvků ve stromu. Příklad binárního vyhledávacího stromu je znázorněn na Obrázku 2.3. [2]

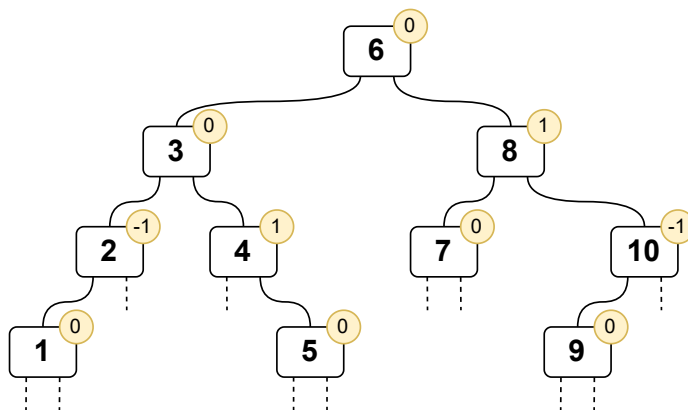


Obrázek 2.3: Binární vyhledávací strom

Efektivita operací nad BST závisí na jeho výšce. Zde nastává problém degradace BST, kdy se strom stává nevyváženým a jeho výška se blíží počtu uzlů, což vede k lineární složitosti operací. V nejhorším případě se BST může stát podobným spojovému seznamu, a tím se výrazně snižuje jeho výkon. Degradovaný strom je možné vidět na Obrázku 2.3. I přesto, že tedy operace nad BST mají průměrnou složitost $O(\log n)$, v nejhorším případě může být složitost $O(n)$. Pro zefektivnění těchto operací se používají samovyvažující se stromy, které z binárního vyhledávacího stromu vychází. [1]

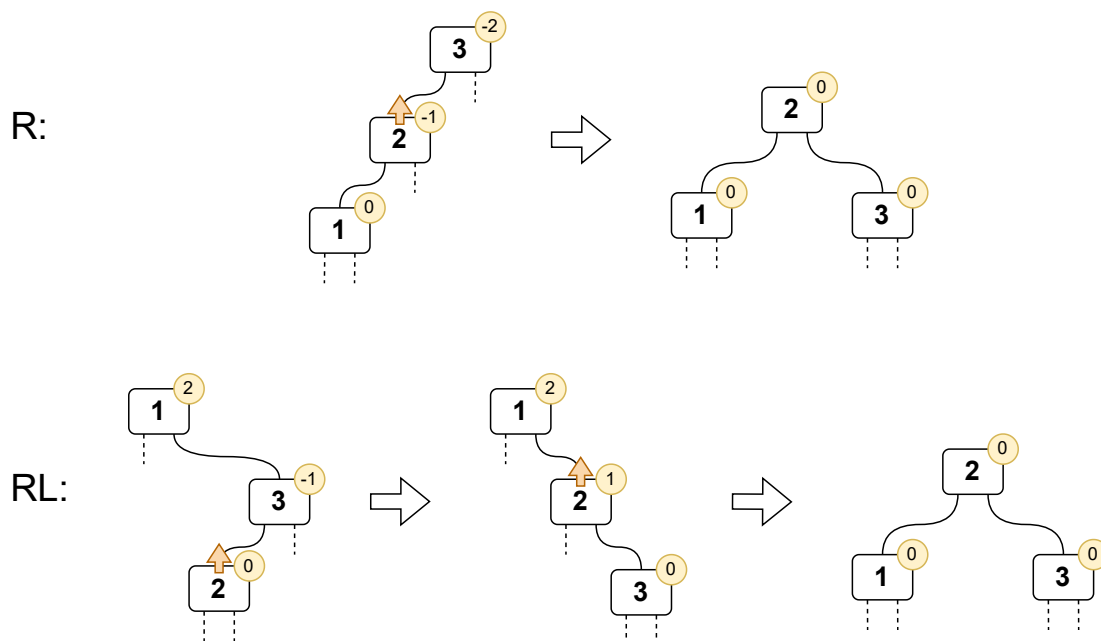
AVL strom

Samovyvažující se AVL stromy jsou pojmenovány po svých tvůrcích, G. M. Adelson-Velsky a E. M. Landis, kteří je představili v roce 1962. AVL strom je binární vyhledávací strom, který udržuje vyváženost tím, že zajišťuje, aby výškový rozdíl mezi levým a pravým podstromem každého uzlu byl nejvýše 1. Tato vlastnost je patrná na Obrázku 2.4, kde má každý uzel přidánu informaci o jeho vyvážení, znázorněnou jako číslo ve žlutém kruhu, které pro tento typ stromu může nabývat pouze hodnot -1, 0 nebo 1. Vyvážení je definováno jako rozdíl mezi výškou levého a pravého podstromu. V samotné implementaci se často v každém uzlu ukládá právě výška podstromu namísto vyvážení. Tento přístup umožňuje efektivní výpočet vyvážení během operací vkládání a mazání. AVL stromy zajišťují, že všechny operace mají složitost $O(\log n)$, a proto jsou často používány v situacích, kde je důležitá rychlost přístupu k datům. [1, 2]



Obrázek 2.4: AVL strom

Vyvážení AVL stromů se udržuje pomocí rotací, které jsou prováděny po každé operaci vkládání nebo mazání, pokud dojde k narušení vyváženosti. Rotace představuje lokální transformaci podstromu právě toho vrcholu, kde je podmínka vyváženosti narušena (kde nabyla hodnoty -2 nebo 2). Existují čtyři typy rotací: jednoduchá pravá rotace, jednoduchá levá rotace, dvojitá levá-pravá a dvojitá pravá-levá rotace, přičemž levé a pravé varianty jsou zrcadlové obrazy. [1]



Obrázek 2.5: Pravá (R) a dvojitá pravá-levá (RL) rotace AVL stromu

Názornou ukázkou pravých variant rotací (R a RL) je možné vidět na Obrázku 2.5. Během rotace se mění struktura podstromu. Díky této změně lze obnovit vyváženost stromu a zajistit, že všechny operace zůstanou v logaritmické složitosti. Například **pravá rotace** se provádí, když levý podstrom vrcholu je příliš vysoký, a zahrnuje přesunutí levého potomka vrcholu nahoru a samotného vrcholu dolů do pravého podstromu. Akce přesunutí potomka nahoru je znázorněna oranžovou šipkou nad daným vrcholem. **Dvojitá pravá-levá rotace** se provádí, když pravý podstrom vrcholu je příliš vysoký a zároveň má levý podstrom, který je vyšší než jeho pravý podstrom. Tato rotace zahrnuje nejprve pravou rotaci na pravém potomku vrcholu a poté levou rotaci na vrcholu samotném. [1]

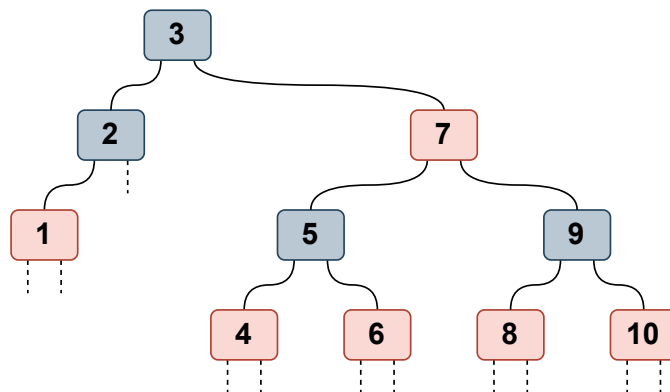
Tyto rotace sice zvyšují složitost implementace AVL stromů, ale zajišťují, že všechny operace nad stromem mají složitost $\Theta(\log n)$. Nutný počet rotací, které mohou být nutné po každé operaci vkládání nebo mazání, sice zabránily AVL stromům stát se standardem pro implementaci seznamů¹, ale jejich myšlenka a principy jsou základem pro další samovyvažující se stromy, které z nich vycházejí. [1]

Červeno-černý strom

Dalším typem samovyvažujícího se stromu je červeno-černý strom, pojmenován podle barvy, kterou jsou označeny jeho uzly. Tento strom udržuje vyváženost pomocí pravidel týkajících se barev uzlů. Dodržování těchto pravidel zajišťuje, že žádná cesta od kořene k listu není více než dvakrát

¹Seznam je v tomto kontextu myšlen jako datová struktura anglicky nazvaná *dictionary*.

delší než jakákoli jiná cesta. Červeno-černé stromy jsou často používány v implementacích map a množin, protože poskytují dobrý kompromis mezi složitostí implementace a výkonem. Příklad červeno-černého stromu ukazuje Obrázek 2.6. [2]



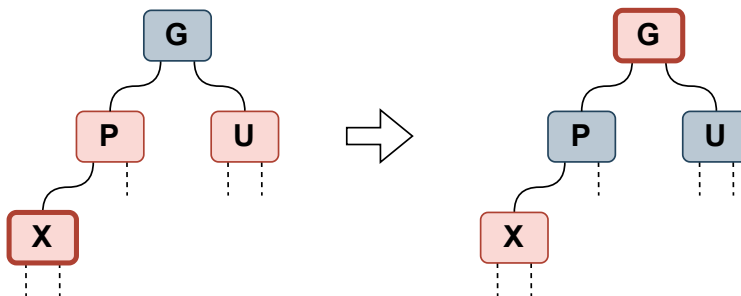
Obrázek 2.6: Červeno-černý strom

Červeno-černé stromy jsou binární vyhledávací stromy, které musí splňovat následující pravidla, aby byla zajištěna jejich vyváženost:

1. Každý uzel je buď červený, nebo černý.
2. Kořen stromu je vždy černý.
3. Všichni nezaplňení potomci listů jsou černí.
4. Pokud je uzel červený, oba jeho potomci jsou černí.
5. Každá cesta od daného uzlu k jeho potomkům obsahuje stejný počet černých uzlů.

Dodržování těchto pravidel zajišťuje, že strom zůstává vyvážený. Tato vlastnost umožňuje efektivní operace vkládání, mazání a vyhledávání s logaritmickou složitostí $\Theta(\log n)$. Implementace červeno-černých stromů zahrnuje rotace a změny barev uzlů, jež po každé operaci vkládání nebo mazání obnovují celkovou vyváženost stromu. Rotace fungují na podobném principu jako u AVL stromů, avšak pravidla pro změnu barev uzlů jsou pro červeno-černé stromy specifická a zaručují zachování vyváženosti i při dočasném narušení pravidel během operací. [2]

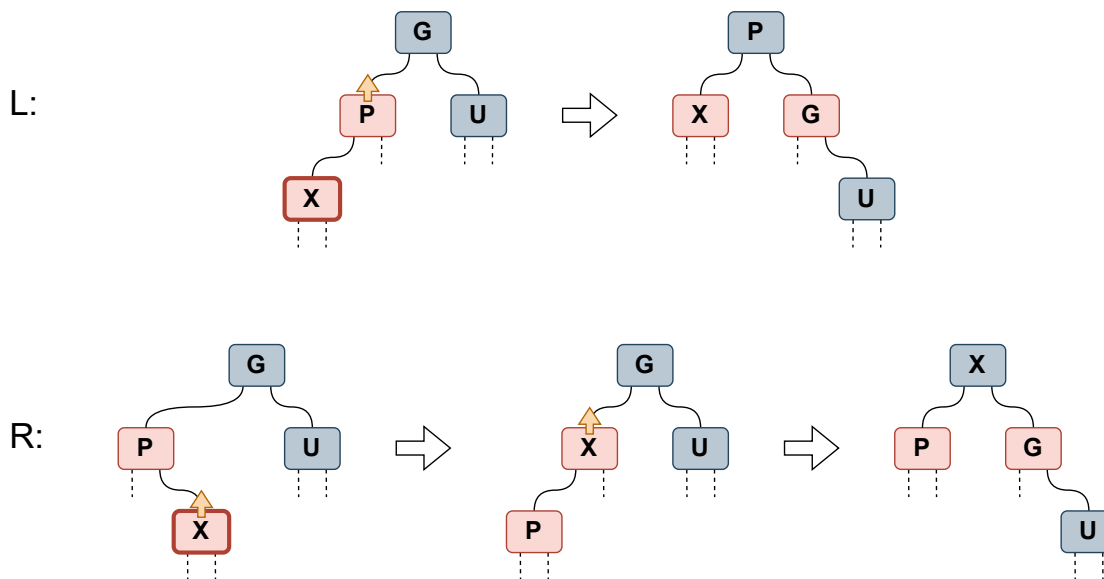
Jako příklad základních úprav červeno-černého stromu je zde popsána implementace **vložení prvku**. Při vkládání nového prvku do struktury se nový uzel nejprve vloží jako červený, aby se minimalizovalo narušení vyváženosti. Následně probíhá kontrola a případné opravy zahrnující rotace a změny barev uzlů, čímž se zajistí opětovné dodržení definovaných pravidel. Pro ilustraci tohoto postupu slouží zjednodušený podstrom zachycující uzly nezbytné pro opravu struktury. Jedná se o



Obrázek 2.7: Oprava červeno-černého stromu po vložení prvku, případ 1

čtyři vrcholy: nově přidaný uzel (**X**), jeho rodič (**P**), strýc (**U**) a prarodič (**G**). Volba konkrétního typu opravy závisí na barvě strýce (**U**) a pozici nově přidaného uzlu (**X**). [3]

První případ nastává, když **strýc (*U*) je červený**, jak je vidět na Obrázku 2.7. V tomto případě se provádí změna barev: *P* a *U* se zbarví na černou, zatímco *G* se zbarví na červenou. Poté se pokračuje v kontrole a případných opravách prarodiče *G*, tato zpětná kontrola je znázorněna tlustým okrajem kolem daného uzlu. Rekurzivně se tedy strom projde až ke kořenu, aby se zajistilo, že všechna pravidla červeno-černého stromu jsou dodržena. Tento proces může vést k dalším změnám barev a rotacím, pokud dojde k narušení pravidel během opravy. [3]



Obrázek 2.8: Oprava červeno-černého stromu po vložení prvku, případ 2

Druhým případem je situace, kdy **strýc (*U*) je černý**, jak je znázorněno na Obrázku 2.8. V tomto případě se provádí nejen změna barev, ale i rotace. Dále se situace větví podle pozice nově přidaného uzlu (**X**) a jeho rodiče (**P**). Pro zjednodušení jsou uvedeny pouze dvě varianty, kdy je *P*

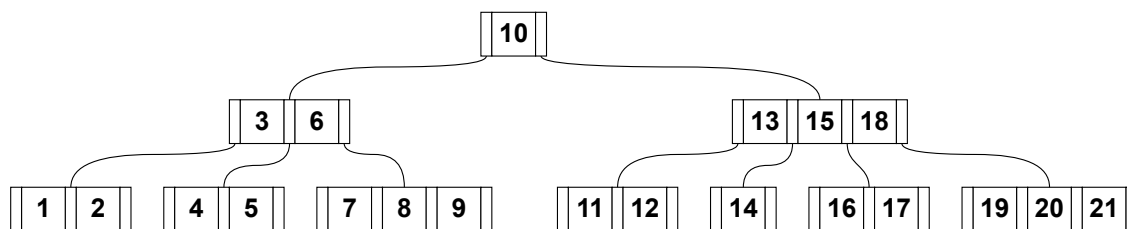
levým potomkem G . Pokud je nový uzel **levým potomkem** (případ L), provede se pravá rotace na G a P se tedy stává novým kořenem podstromu, přičemž během rotace si zmíněné vrcholy vymění barvy. Podobně jako u diagramu AVL stromu, akce přesunutí potomka směrem nahoru je na diagramu znázorněna oranžovou šipkou nad daným vrcholem. Pokud je nový uzel **pravým potomkem** (případ R), provede se nejprve levá rotace na P , čímž se X stává levým potomkem G , a poté pravá rotace na G , přičemž během druhé rotace si X a G vymění barvy. Po těchto úpravách již není potřeba rekursivní kontrola, protože všechna pravidla červeno-černého stromu jsou dodržena a strom zůstává vyvážený. [3]

Odstraňování prvku z červeno-černého stromu je složitější operace, která zahrnuje více případů a může vyžadovat několik rotací a změn barev, aby se udržela vyváženost stromu. Nicméně principy jsou podobné jako při vkládání, kdy se nejprve odstraní uzel a poté se provádí opravy, pokud dojde k narušení pravidel červeno-černého stromu. [2]

B-stromy

Posledním typem datové struktury z oblasti stromů, který bude implementován v aplikaci, jsou B-stromy. B-stromy, které představili R. Bayer a E. McCreight, jsou samovyvažující se stromy navržené pro efektivní ukládání a vyhledávání dat na sekundárních úložištích, jako jsou pevné disky. V praxi se tyto stromy uplatňují primárně při implementaci databázových indexů. B-stromy umožňují ukládání více klíčů v jednom uzlu a mají proměnný počet potomků. Tato architektura zaručuje zachování vyváženosti i při obrovském množství dat.

Strukturu B-stromu, jež se od dříve popsanych typů vizuálně i logicky liší, přibližuje Obrázek 2.9. Každý uzel může obsahovat více datových prvků a mít více potomků. Samotná vyváženost stromu je pak striktně dána tím, že se všechny listy nacházejí na stejné úrovni. [1]



Obrázek 2.9: B-strom řádu 3

B-stromy jsou definovány parametrem m , který se nazývá jeho řádem a může nabývat hodnoty od 2 do nekonečna. Je nutno zmínit, že v jiné literatuře se namísto řádu používá minimální stupeň t , který je roven $\lceil m/2 \rceil$ a udává minimální počet potomků, které musí mít každý uzel kromě kořene a listů. Dále však v tomto textu bude použit pouze řád m . [1, 2]

Každý uzel v B-stromu může obsahovat nejvýše $m - 1$ klíčů a mít nejvýše m potomků. Uzel, který není listem, musí mít alespoň $\lceil m/2 \rceil$ potomků. Tato podmínka zaručuje, že strom zůstává

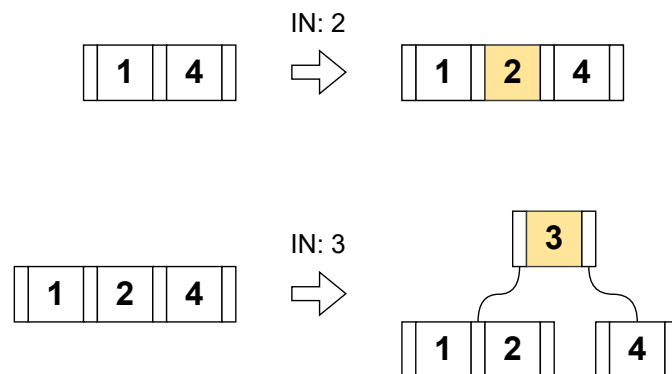
vyvážený. Klíče v uzlu logicky rozdělují rozsah hodnot, které mohou být uloženy v jeho potomcích, a při operacích tedy slouží k navigaci stromem. Všechny listy B-stromu leží na stejné úrovni, a proto mají veškeré cesty od kořene k listům shodnou délku. Operace vkládání a mazání pak zahrnují mechanismy rozdělení a sloučení uzlů pro trvalé udržení celkové struktury a vyváženosti. [1]

Strom řádu m tedy musí splňovat následující vlastnosti:

1. Kořen stromu je buď list, nebo má 2 až m potomků.
2. Každý uzel kromě kořene a listů musí mít mezi $\lceil m/2 \rceil$ a m potomků.
3. Strom je perfektně vyvážený, což znamená, že všechny listy jsou na stejné úrovni.

Vyhledávání v B-stromu začíná u kořene a porovnává hledanou hodnotu s klíči v uzlu. Pokud se hodnota shoduje s některým z klíčů, operace je úspěšná a hledání končí. Pokud se hodnota neshoduje, klíče se použijí jako hraniční body určující, do kterého potomka se má hledání přesunout. Tento proces se opakuje, dokud algoritmus nenalezne požadovanou hodnotu nebo nedosáhne listu. Dosažení listu bez nalezení shody indikuje, že hledaná hodnota není ve stromu přítomna. [1]

Operace **vložení** klíče do B-stromu je přímočará, vyžaduje však případné rozdělení uzlů, pokud dojde k překročení maximální kapacity. Pokud je uzel plný (obsahuje $m - 1$ klíčů) a je třeba do něj vložit nový prvek, uzel se rozdělí na dva a prostřední klíč se přesune do nadřazeného uzlu. Tento proces může rekurzivně pokračovat až ke kořeni a v případě jeho naplnění vyvolá vytvoření kořene zcela nového. Oba případy ilustruje Obrázek 2.10; nejprve zachycuje vložení klíče 2 do neplného listu a následně vložení klíče 3. Při tomto druhém kroku již dochází k rozdělení listu a přesunu klíče 3 nahoru do nadřazeného uzlu, který se stává novým kořenem. Nově přidané klíče jsou na obrázku zvýrazněny oranžovou barvou. [1]



Obrázek 2.10: Vložení klíče do B-stromu řádu 3

Operace **mazání** klíče z B-stromu je složitější, protože může vést k narušení struktury stromu a jeho vyváženosti. Pokud se klíč určený k odstranění nachází v listu, jednoduše se smaže. Pokud je klíč

uložen ve vnitřním uzlu, musí být nahrazen buď největším klíčem z levého podstromu (predecessor), nebo naopak nejmenším klíčem z pravého podstromu (successor). Po odstranění prvku může dojít k situaci, kdy uzel obsahuje méně než $\lceil m/2 \rceil - 1$ klíčů. Tento stav vyžaduje dodatečnou opravu stromu. Samotná oprava zahrnuje buď přesun klíče z přilehlého sourozeneckého uzlu (pokud má dostatek klíčů), nebo sloučení s tímto sourozencem. Následně se rekurzivně kontroluje rodičovský uzel, dokud se neobnoví kompletní vyváženost struktury. [1]

Architektura B-stromů je cíleně navržena tak, aby minimalizovala počet diskových operací potřebných pro vyhledávání, vkládání a mazání dat. Z tohoto důvodu jsou tyto struktury masivně využívány v databázových a souborových systémech. Schopnost udržovat striktní vyváženost a efektivně spravovat rozsáhlé objemy dat na sekundárních úložištích představuje stěžejní faktor jejich popularity, a proto bylo rozhodnuto o jejich začlenění do vytvářené výukové aplikace.

2.1.3 Prioritní fronty a haldy

Prioritní fronty jsou abstraktní datové struktury, které umožňují ukládání prvků s přiřazenou prioritou a poskytují operace pro vkládání prvků a odebírání prvku s nejvyšší prioritou. Oproti běžným frontám, které fungují na principu FIFO, umožňují prioritní fronty flexibilnější způsob správy prvků. Tato vlastnost nachází široké uplatnění v mnoha algoritmech a aplikacích, jako je například plánování úloh či hledání nejkratší cesty. Halda (*Heap*) je konkrétní implementací prioritní fronty, která využívá stromovou strukturu k efektivnímu ukládání a správě prvků podle jejich priorit. Na závěr teorie datových struktur jsou tedy popsány vlastnosti prioritních front a hald, včetně jejich operací a požadavků na vizualizaci.

Binární halda (Heap)

Binární halda je datová struktura, která může být reprezentována jak polem, tak binárním stromem. V této práci je jako příklad uvedena takzvaná **max-heap**, kde každý rodičovský uzel má hodnotu větší nebo rovnou hodnotám svých potomků, druhá varianta, **min-heap**, se z tohoto popisu dá analogicky odvodit.

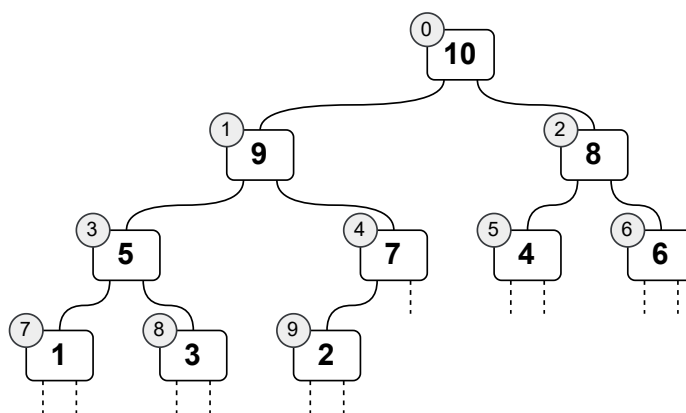
Halda se často používá pro implementaci prioritních front, protože umožňuje efektivní vkládání prvků a nalezení odebírání prvku s nejvyšší prioritou. Díky své struktuře umožňuje binární halda provádět tyto operace s logaritmickou složitostí, je potvrzeno, že v kořenu stromu je vždy ten největší prvek, a dobře se na ni aplikují rekurzivní algoritmy, neboť libovolný podstrom haldy je také haldou. [1]

Jak již bylo zmíněno výše, binární halda reprezentovaná stromem musí splňovat dvě podmínky:

1. Strom musí být kompletní, což znamená, že všechny úrovně stromu jsou zcela zaplněné kromě poslední úrovně, která je zaplněna zleva doprava.

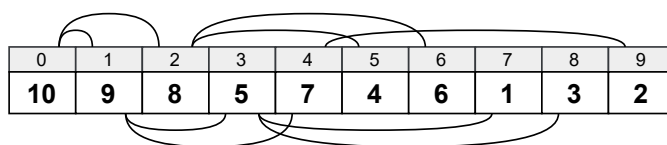
2. Každý rodičovský uzel musí mít hodnotu větší nebo rovnou hodnotám svých potomků.²

Stromovou reprezentaci binární haldy splňující tyto podmínky je možné vidět na Obrázku 2.11. V šedých kruzích u jednotlivých uzlů je znázorněn jejich index, který odpovídá jejich pozici v poli.



Obrázek 2.11: Binární halda reprezentovaná stromem

Binární halda může být také efektivně reprezentována polem (Obrázek 2.12), kde pro každý prvek v poli platí následující vztahy: pro prvek na indexu i jsou jeho levý potomek na indexu $2i$, pravý potomek na indexu $2i + 1$ a rodičovský uzel na indexu $\lfloor i/2 \rfloor$. Tato reprezentace umožňuje efektivní přístup k prvkům haldy, dovoluje algoritmům pracovat s daty bez nutnosti explicitní reprezentace stromové struktury, a je často používána v implementacích algoritmů, jako je Heapsort. [2]



Obrázek 2.12: Binární halda reprezentovaná polem

Základními operacemi nad binární haldou jsou **vložení** a **odebrání** prvku s nejvyšší prioritou. Při vložení nového prvku do haldy se nový prvek umístí na konec stromu (nebo do poslední pozice pole) a poté se provádí operace zvaná *heapify-up*, která zajišťuje, že vlastnost haldy je zachována. Tato operace zahrnuje porovnávání nově vloženého prvku s jeho rodičovským uzlem a případné prohození, pokud je nový prvek větší než jeho rodič. Tento proces se opakuje, dokud není vlastnost haldy obnovena nebo dokud nedojde k dosažení kořene stromu. [1]

Při odebrání prvku s nejvyšší prioritou (kořene stromu) se tento prvek odstraní a nahradí se posledním prvkem v haldě. Poté se provádí operace zvaná *heapify-down*. Probíhá porovnávání nově

²Platí pro variantu max-heap, pro min-heap je podmínka opačná.

umístěného prvku s jeho potomky a případné prohození s největším potomkem, pokud je nový prvek menší než některý z jeho potomků. Proces se opět opakuje až do splnění vlastnosti haldy nebo dosažení listu stromu. [1]

Operace vkládání a odebrání prvku s nejvyšší prioritou v binární haldě mají složitost $O(\log n)$. Tato vlastnost z ní činí efektivní datovou strukturu pro implementaci prioritních front a pro použití v algoritmech, které vyžadují rychlý přístup k prvkům s nejvyšší prioritou, jako jsou například algoritmy pro hledání nejkratší cesty nebo plánování úloh. Její časté využití je také v třídícím algoritmu Heapsort, který bude detailněji popsán v Kapitole 2.2.2. [2]

2.1.4 Srovnání popsaných datových struktur

Pro přehlednost jsou v této části shrnuty podstatné vlastnosti a časové složitosti základních operací (vyhledávání, vkládání a mazání) u všech výše popsaných datových struktur. Tabulka 2.1 poskytuje srovnání průměrných a nejhorších případů složitosti, které přímo ovlivňují volbu konkrétní struktury pro daný typ úlohy. V posledním sloupci je také uveden hlavní princip, na kterém daná datová struktura funguje, a který je důležitý pro pochopení jejího chování v rámci vizualizace a implementace v aplikaci.

Struktura	Vyhledávání	Vložení/Smazání	Hlavní princip
Spojový seznam	$O(n)$	$O(1) / O(n)$ ³	Dynamické uzly
Zásobník	$O(n)$	$O(1)$	LIFO princip
Fronta	$O(n)$	$O(1)$	FIFO princip
BST	$O(\log n)/O(n)$	$O(\log n)/O(n)$	Uspořádaný strom
AVL strom	$O(\log n)$	$O(\log n)$	Přísná vyváženost
Červeno-černý	$O(\log n)$	$O(\log n)$	Pravidla barev
B-strom	$O(\log n)$	$O(\log n)$	Více klíčů v uzlu
Binární halda	$O(1)$ ⁴	$O(\log n)$	Max/Min vlastnost

Tabulka 2.1: Srovnání vlastností a časové složitosti operací datových struktur

Jak vyplývá z tabulky, zatímco lineární struktury jsou vhodné pro specifické typy přístupu k datům (jako LIFO u zásobníku), u vyhledávacích úloh dominují stromové struktury. Samovyvažující se stromy (AVL a červeno-černý) pak řeší problém degradace BST tím, že garantují logaritmickou složitost i v nejhorším případě. Speciální vlastnosti má B-strom, který díky své šířce optimalizuje přístup k datům v sekundárních úložištích, a binární halda, která je optimalizována pro operace s prioritami.

³Konstantní čas platí pro vkládání/mazání na koncích seznamu (u head/tail).

⁴Namísto hledání konkrétního prvku, což by mělo složitost $O(n)$, je možné získat prvek s nejvyšší prioritou (kořen haldy) v konstantním čase $O(1)$.

Pro účel výukové aplikace se zde tedy vymezují základní typy struktur, každá s konkrétními operacemi a stylem zobrazení nutným pro její názorné představení uživatelům.

2.2 Třídící algoritmy

V této kapitole jsou popsány třídící algoritmy, které budou implementovány v aplikaci. Každý algoritmus je charakterizován svým principem fungování, složitostí a významem pro pochopení základních konceptů informatiky.

Většina třídících algoritmů uvedených v této kapitole během práce s polem často využívá vyměňování dvou prvků mezi sebou. Pro zjednodušení implementace a zvýšení přehlednosti kódu je vhodné mít pomocnou funkci pro výměnu dvou prvků v poli, která bude využívána napříč různými algoritmy. Tato funkce má název **Swap**, a její pseudokód je uveden ve Výpisu 1. Kromě této funkce bude také využita funkce **Length**, která vrací délku pole a její implementace je zřejmá a není zde uvedena.

```
1 Function Swap( $A, i, j$ ):  
2    $tmp \leftarrow A[i]$   
3    $A[i] \leftarrow A[j]$   
4    $A[j] \leftarrow tmp$ 
```

Výpis 1: Pseudokód pomocné funkce pro výměnu dvou prvků

Právě funkce **Swap** je základem pro vizualizaci třídících algoritmů, protože umožňuje jasně zobrazit, které prvky jsou během třídění vyměňovány. Druhou důležitou operací je porovnávání prvků, které je v pseudokódu reprezentováno pomocí operátorů porovnávání v podmínce (například $<$, $>$, $==$ atd.) a bude také vizualizováno, aby uživatel mohl sledovat, které prvky jsou porovnávány a jak se rozhoduje o jejich výměně. [4]

2.2.1 Algoritmy s kvadratickou časovou složitostí

Jako úvod do problematiky třídících algoritmů jsou nejprve popsány metody s kvadratickou časovou složitostí, které jsou často považovány za základní a slouží jako výchozí bod pro pochopení složitějších postupů. Tyto algoritmy se vyznačují tím, že v nejhorším případě provádějí přibližně n^2 operací. Z tohoto důvodu nejsou vhodné pro zpracování velkého množství dat, avšak výborně slouží k ilustraci základních principů a k výuce algoritmického myšlení.

V této kapitole budou uvedeni tři základní zástupci ze zmíněné kategorie: *Bubble Sort*, *Selection Sort* a *Insertion Sort*. U každého z nich je následně rozebrán princip fungování, časová složitost a případné optimalizace přispívající ke zlepšení výkonu ve specifických situacích.

Bubble sort

Prvním třídícím algoritmem s kvadratickou časovou složitostí, který bude popsán, je *Bubble Sort*. Tento algoritmus funguje na principu opakovaného procházení seznamu, přičemž v každém průchodu nechá nejvyšší prvek „vystoupat“ na konec seznamu, ostatně z analogie k bublinám ve vodě získal svůj název. Algoritmus pokračuje v procházení seznamu, dokud není celý seznam seřazený. [3]

Jednotlivé kroky algoritmu *Bubble Sort* při třídění pole názorně ilustruje Výpis 2. V každém kroku jsou porovnány sousední prvky a dle podmínky vyměněny, což vede k postupnému „vystupování“ největších prvků na konec pole.

```
1 Function BubbleSort(A):  
2    $n \leftarrow \text{Length}(A)$   
3   for  $i \leftarrow 0$  to  $n - 1$  do  
4     for  $j \leftarrow 0$  to  $n - i - 1$  do  
5       if  $A[j] > A[j + 1]$  then  
6         Swap( $A[j], A[j + 1]$ )
```

Výpis 2: Pseudokód algoritmu Bubble Sort

Tento algoritmus může být optimalizován pomocí příznaku, který sleduje, zda došlo k nějaké výměně během průchodu polem. Pokud během celého průchodu nedojde k žádné výměně, znamená to, že pole je již seřazené, a výpočet může být předčasně ukončen. Tato úprava výrazně zlepšuje celkový výkon v situacích, kdy je pole již téměř seřazené.

I když je tento algoritmus jednoduchý a snadno pochopitelný, jeho kvadratická časová složitost $O(n^2)$ ho činí neefektivním pro třídění velkých množství dat, a proto se v praxi používá jen zřídka, spíše jako pedagogický nástroj pro ilustraci základních principů třídění. Jeho výhodou také může být jeho stabilita a konstantní prostorová složitost $O(1)$, protože třídění probíhá přímo v původním poli bez potřeby dalšího prostoru pro pomocné struktury, dá se tedy využít pro stroje s velmi omezenými zdroji. [3]

Selection sort

Další algoritmus s kvadratickou časovou složitostí je *Selection Sort*, který je často považován za jeden z nejjednodušších třídících algoritmů. V každé iteraci vždy hledá nejmenší prvek v nesetříděné části pole a vymění ho s prvním prvkem této části, čímž postupně rozšiřuje setříděnou část pole. Prvky se tímto způsobem přesouvají, dokud algoritmus nedosáhne konce pole. [3]

Z příkladu na Výpisu 3 je možné vidět, jak algoritmus pracuje. Sekvenčně prochází nesetříděnou část pole, „vybere“ nejmenší prvek a vymění ho s prvním prvkem této části, čímž se rozšiřuje

setříděná část. Ve srovnání s předchozím algoritmem se tedy setříděné pole vytváří zleva, zatímco u algoritmu Bubble Sort se největší prvky přesouvají doprava.

```
1 Function SelectionSort( $A$ ):
2    $n \leftarrow \text{Length}(A)$ 
3   for  $i \leftarrow 0$  to  $n - 1$  do
4      $\text{minIndex} \leftarrow i$ 
5     for  $j \leftarrow i + 1$  to  $n$  do
6       if  $A[j] < A[\text{minIndex}]$  then
7          $\text{minIndex} \leftarrow j$ 
8     Swap( $A[i], A[\text{minIndex}]$ )
```

Výpis 3: Pseudokód algoritmu Selection Sort

Co se týče optimalizace, u algoritmu *Selection Sort* nejsou běžně používané žádné specifické optimalizace, z části i proto, že algoritmus již provádí minimální počet výměn (přesně $n - 1$). To může být výhodné v situacích, kdy jsou výměny nákladné. Oproti předchozímu algoritmu Bubble Sort není tento algoritmus stabilní, protože může měnit pořadí prvků se stejnou hodnotou během výměn. Jeho časová i prostorová složitost je stejná jako u Bubble Sort, tedy $O(n^2)$ pro časovou složitost a $O(1)$ pro prostorovou složitost, protože třídění probíhá přímo v původním poli. [3]

Insertion sort

Poslední algoritmus s kvadratickou časovou složitostí, který bude popsán, je *Insertion Sort*. Stejně jako u předchozího algoritmu se zde setříděné pole vytváří zleva, ale místo toho, aby hledal nejmenší prvek v neseřazené části, *Insertion Sort* postupně posune všechny prvky větší než aktuální klíč doprava a vloží klíč na správnou pozici. Tento proces se opakuje pro každý prvek v poli, dokud není celé pole seřazené. [3]

Logiku přesouvání prvků je možné vidět na Výpisu 4, kde se prvně vybere klíč (aktuální prvek) a poté se porovnává s předchozími prvky v setříděné části pole. Pokud je klíč menší než některý z těchto prvků, tyto prvky se posunou doprava, dokud nenajdou správnou pozici pro klíč, který se poté vloží na tuto pozici.

Tento algoritmus nabízí známou optimalizaci – místo neustálého prohazování klíče s většími prvky se jeho hodnota nejprve uloží do pomocné proměnné. Následně se pouze posouvají větší prvky, dokud není nalezena správná pozice, na kterou se klíč nakonec vloží. Tato optimalizace snižuje počet výměn, a tudíž může zlepšit výkon algoritmu, zejména v případech, kdy je pole již téměř seřazené.

Insertion Sort se řadí mezi stabilní algoritmy, protože během posouvání nemění vzájemné pořadí prvků se shodnou hodnotou. Jeho časová složitost dosahuje v nejhorším případě $O(n^2)$, k této situaci

```

1 Function InsertionSort( $A$ ):
2    $n \leftarrow \text{length}(A)$ 
3   for  $i \leftarrow 1$  to  $n - 1$  do
4      $j \leftarrow i$ 
5     while  $j > 0$  and  $A[j - 1] > A[j]$  do
6       Swap( $A[j - 1], A[j]$ )
7        $j \leftarrow j - 1$ 

```

Výpis 4: Pseudokód algoritmu Insertion Sort

dochází tehdy, když je pole původně seřazeno v opačném pořadí. Naopak v nejlepším případě, kdy jsou data již předem seřazená, klesá časová složitost na $O(n)$, protože každý prvek je porovnán pouze jednou a nevyžaduje žádné posuny. Pro prostorovou složitost platí $O(1)$, protože třídění probíhá přímo v původním poli bez potřeby další paměti pro pomocné struktury. [3]

2.2.2 Algoritmy s logaritmickou časovou složitostí

Po seznámení s algoritmy s kvadratickou časovou složitostí je vhodné přejít k těm, které jsou efektivnější a mají logaritmickou časovou složitost, v nejhorším případě tedy provádějí přibližně $O(n \log n)$ operací. Tyto algoritmy jsou schopné třídit velké množství dat mnohem rychleji než algoritmy s kvadratickou složitostí, a proto jsou často používány v praxi. V této části budou popsány tři základní algoritmy s logaritmickou časovou složitostí: *Merge Sort*, *Quick Sort* a *Heap Sort*.

Merge sort

Merge Sort je algoritmus založený na principu *divide and conquer*. Algoritmus rekurzivně rozděljuje pole na dvě poloviny, dokud nedosáhne pole o velikosti jednoho prvku, které je považováno za seřazené. Výsledky se poté postupně slučují zpět dohromady, přičemž při slučování se porovnávají prvky z obou polovin a vytváří se nové pole, které je seřazené. Tento proces se provádí rekurzivně a na konci se získá celé pole seřazené. [1]

Princip fungování algoritmu *Merge Sort* je znázorněn na Výpisu 5. Samotná funkce **MergeSort** je velmi jednoduchá, její jediná funkcionalita je nalezení prvku zhruba uprostřed zadaného pole, rekurzivní zavolání sama sebe pro levou a pravou polovinu a následné zavolání funkce **Merge**, která provádí samotné slučování.

```

1 Function Merge( $A, l, m, r$ ):
2    $n_1 \leftarrow m - l + 1$ 
3    $n_2 \leftarrow r - m$ 
4   for  $i \leftarrow 0$  to  $n_1 - 1$  do
5      $L[i] \leftarrow A[l + i]$ 
6   for  $j \leftarrow 0$  to  $n_2 - 1$  do
7      $R[j] \leftarrow A[m + 1 + j]$ 
8    $i \leftarrow 0, j \leftarrow 0, k \leftarrow l$ 
9   while  $i < n_1$  and  $j < n_2$  do
10    if  $L[i] \leq R[j]$  then
11       $A[k] \leftarrow L[i]$ 
12       $i \leftarrow i + 1$ 
13    else
14       $A[k] \leftarrow R[j]$ 
15       $j \leftarrow j + 1$ 
16     $k \leftarrow k + 1$ 
17  while  $i < n_1$  do
18     $A[k] \leftarrow L[i]$ 
19     $i \leftarrow i + 1, k \leftarrow k + 1$ 
20  while  $j < n_2$  do
21     $A[k] \leftarrow R[j]$ 
22     $j \leftarrow j + 1, k \leftarrow k + 1$ 

23 Function MergeSort( $A, l, r$ ):
24   if  $l < r$  then
25      $m \leftarrow l + (r - l)/2$ 
26     MergeSort( $A, l, m$ )
27     MergeSort( $A, m + 1, r$ )
28     Merge( $A, l, m, r$ )

```

Výpis 5: Pseudokód algoritmu Merge Sort

Implementace funkce **Merge** je rozsáhlejší. Na začátku se levá a pravá část pole překopírují do dvou pomocných struktur. Následně se obě pole sekvenčně procházejí zleva a vzájemně se porovnávají jejich prvky. Menší prvek z obou polí se vloží do výsledného pole a index se posune o jednu

pozici doprava. Tento proces se opakuje, dokud nejsou zpracována všechna data. Pokud jedno z polí dojde dříve, zbylé prvky z toho druhého se jednoduše překopírují na konec výsledného pole.

Tento algoritmus dosahuje časové složitosti $O(n \log n)$ v nejhorším, průměrném i nejlepším případě, protože vždy rozděluje pole na dvě poloviny a při slučování prochází všechny prvky. Prostorová složitost je však horší než u dříve popsaných kvadratických algoritmů. Během slučování je totiž vyžadován dodatečný prostor pro pomocná pole, a celková prostorová náročnost tedy narůstá na $O(n)$ – tento algoritmus tedy není *in-place*. Nicméně *Merge Sort* je stabilní a velmi dobře se paralelizuje. Tyto vlastnosti z něj činí ideální algoritmus pro třídění velkých datových sad, zejména pak v prostředí s více procesory. [3]

Quick sort

Dalším velmi efektivním a často používaným algoritmem je *Quick Sort*. Stejně jako *Merge Sort* je založen na principu *divide and conquer*. Jeho časová složitost dosahuje v průměrném a nejlepším případě $O(n \log n)$, ale v nejhorším případě může vzrůst až na $O(n^2)$. Tento případ je obvykle způsoben nevhodným výběrem dělicího prvku (pivot), podle kterého se pole rozděluje na dvě části, jak bude vysvětleno níže. Nicméně s vhodnými optimalizacemi lze pravděpodobnost nejhoršího případu minimalizovat a zachovat tak očekávanou složitost $O(n \log n)$ pro libovolná vstupní data. [3]

```
1 Function Partition(A, low, high):
2   pivot ← A[high]
3   i ← low − 1
4   for j ← low to high − 1 do
5     if A[j] ≤ pivot then
6       i ← i + 1
7       Swap(A[i], A[j])
8   Swap(A[i + 1], A[high])
9   return i + 1

10 Function QuickSort(A, low, high):
11   if low < high then
12     p ← Partition(A, low, high)
13     QuickSort(A, low, p − 1)
14     QuickSort(A, p + 1, high)
```

Výpis 6: Pseudokód algoritmu Quick Sort

Hlavní funkce tohoto algoritmu je podobná jako v minulém případě. Pokud je pole větší než jeden prvek, zavolá se funkce **Partition**, která rozdělí pole na dvě části podle zvoleného pivotu, jehož index vrací. Následně se funkce **Quick Sort** rekurzivně volá pro obě části pole.

Funkce **Partition** nejprve zvolí pivot, v příkladu na Výpisu 6 se jedná o poslední prvek pole. Poté prochází data od začátku a porovnává každý prvek s pivotem. Pokud je prvek menší nebo roven pivotu, provede se jeho výměna s prvkem na indexu i (který začíná na začátku pole) a tento index se posune o jednu pozici doprava. Po dokončení průchodu polem se pivot vymění s prvkem na indexu i . Tím se zajistí, že všechny hodnoty menší nebo rovné pivotu se nacházejí nalevo od něj a všechny větší napravo. Funkce následně vrátí finální index pivotu, který slouží jako dělicí bod pro další rekurzivní volání.

Hlavní optimalizace tohoto algoritmu se točí kolem efektivního výběru vhodného pivotu. Pokud je totiž pivot zvolen nevhodně, například jako nejmenší nebo největší prvek v poli, může dojít k situaci, kdy jedna část pole zůstane prázdná a druhá bude obsahovat všechny zbylé prvky. Tento extrém vede k nejhoršímu možnému scénáři s časovou složitostí $O(n^2)$. Pro zlepšení výkonu se proto často používá náhodný výběr pivotu, metoda *median-of-three* (při níž se jako pivot volí medián z prvního, prostředního a posledního prvku) nebo jiné heuristiky. Tyto přístupy pomáhají zaručit rovnoměrnější rozdělení pole a tím i optimální chod algoritmu. [3]

Algoritmus *Quick Sort* se řadí mezi nestabilní metody, protože během výměn může narušit původní pořadí prvků se stejnou hodnotou. Jeho prostorová složitost dosahuje v průměrném případě $O(\log n)$, což je dáno pamětí alokovanou pro rekurzivní volání. V nejhorším případě však může vzrůst až na $O(n)$, pokud jsou všechny prvky shodné nebo pokud je pole již předem seřazené a pivot vždy spadne na pozici lokálního extrému. [3]

Heap sort

Jako poslední uvedený algoritmus, který bude v této kapitole popsán, je algoritmus *Heap Sort*. Oproti dvěma předchozím algoritmům je založen na principu *transform and conquer*, protože pracuje s datovou strukturou zvanou halda, která již byla představena v Kapitole 2.1.3 a je možné ji efektivně reprezentovat polem. Algoritmus využívá toho, že přetvoření pole do haldy je možné provést v lineárním čase $O(n)$, a poté opakovaně odebírá prvek s nejvyšší prioritou (kořen haldy) a umísťuje ho na konec pole, čímž postupně vytváří seřazené pole. Druhá část algoritmu má složitost $O(n \log n)$ a díky tomu, že obě části algoritmu jsou prováděny sekvenčně, celková časová složitost algoritmu *Heap Sort* je $\Theta(n \log n)$ v nejhorším, průměrném i nejlepším případě. [1]

Podobně jako u dvou předchozích příkladů se implementace skládá z hlavní funkce **HeapSort**, která je volána na celé pole a která využívá pomocnou funkci **Heapify** pro přetvoření pole do haldy. Nejprve se pole přetvoří do struktury max-heap, a je tedy zajištěno, že největší prvek je na kořeni haldy. Poté se tento prvek vymění s posledním prvkem pole a z o jeden prvek menšího pole se znovu vytvoří halda.

Funkce **Heapify** je zodpovědná za udržení vlastnosti haldy. Pro každý uzel v poli se porovnává s jeho potomky a pokud je některý z potomků větší než rodičovský uzel, provede se výměna a funkce

```

1 Function Heapify( $A, n, i$ ):
2    $largest \leftarrow i$ 
3    $l \leftarrow 2i + 1$ 
4    $r \leftarrow 2i + 2$ 
5   if  $l < n$  and  $A[l] > A[largest]$  then
6      $largest \leftarrow l$ 
7   if  $r < n$  and  $A[r] > A[largest]$  then
8      $largest \leftarrow r$ 
9   if  $largest \neq i$  then
10    Swap( $A[i], A[largest]$ )
11    Heapify( $A, n, largest$ )

12 Function HeapSort( $A$ ):
13    $n \leftarrow \text{Length}(A)$ 
14   for  $i \leftarrow n/2 - 1$  to 0 do
15     Heapify( $A, n, i$ )
16   for  $i \leftarrow n - 1$  to 1 do
17     Swap( $A[0], A[i]$ )
18     Heapify( $A, i, 0$ )

```

Výpis 7: Pseudokód algoritmu Heap Sort

se rekurzivně volá pro tohoto potomka, aby se zajistilo, že vlastnost haldy je zachována pro celý strom.

Jak již bylo zmíněno, algoritmus *Heap Sort* má časovou složitost $\Theta(n \log n)$ v nejhorším, průměrném i nejlepším případě, protože přetvoření pole do haldy je lineární operace a následné odebrání prvku s nejvyšší prioritou a udržování vlastnosti haldy má logaritmickou složitost. Prostorová složitost je $O(1)$, protože třídění probíhá přímo v původním poli bez potřeby dalšího prostoru pro pomocné struktury. Algoritmus *Heap Sort* je nestabilní, protože může měnit pořadí prvků se stejnou hodnotou během výměn. [1]

2.2.3 Srovnání třídících algoritmů

Pro usnadnění výběru vhodného algoritmu a pro přehlednost teoretických poznatků jsou v této kapitole shrnuty podstatné vlastnosti všech implementovaných algoritmů. Srovnání se zaměřuje na časovou složitost v různých případech, prostorovou náročnost a stabilitu.

Z Tabulky 2.2 je patrné, že algoritmy s kvadratickou časovou složitostí jsou vhodné spíše pro menší datové sady nebo pro specifické případy, jako je téměř seřazené pole u algoritmu *Insertion Sort*, kde jeho nejlepší časová složitost dosahuje lineárních hodnot $O(n)$.

Algoritmus	Časová složitost		Prostorová složitost	Stabilita
	Průměrná	Nejhorší		
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(1)$	Ano
Selection Sort	$O(n^2)$	$O(n^2)$	$O(1)$	Ne
Insertion Sort	$O(n^2)$	$O(n^2)$	$O(1)$	Ano
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n)$	Ano
Quick Sort	$O(n \log n)$	$O(n^2)$	$O(n)$	Ne
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(1)$	Ne

Tabulka 2.2: Srovnání vlastností a složitosti třídících algoritmů

Naopak algoritmy s logaritmickou složitostí vykazují výrazně vyšší efektivitu u velkých datových sad. *Merge Sort* má sice vyšší prostorovou náročnost $O(n)$, ale na rozdíl od algoritmů *Quick Sort* a *Heap Sort* garantuje stabilitu třídění. *Quick Sort* je v praxi často nejrychlejší, avšak jeho výkon je silně závislý na volbě pivotu, jak je patrné v riziku nejhoršího případu $O(n^2)$. *Heap Sort* představuje vyvážený kompromis, který garantuje logaritmický čas i v nejhorším případě při zachování konstantní prostorové složitosti.

Výše uvedené algoritmy byly tedy označeny jako stěžejní pro implementaci v rámci výukové aplikace, protože pokrývají široké spektrum třídících technik a umožňují uživatelům pochopit různé přístupy k třídění dat, jejich výhody a nevýhody.

2.3 Význam vizualizace ve vzdělávání

Pochopení datových struktur a algoritmů vyžaduje od studentů vysokou míru abstraktního myšlení, a představuje tedy jednu z nelehkých překážek v informatickém vzdělávání. Nástroje pro vizualizaci algoritmů pomáhají tuto bariéru překonat, avšak samotná přítomnost vizualizace automaticky nezaručuje zlepšení studijních výsledků. Z rozsáhlé metaanalýzy vyplývá, že úspěšnost učení je ovlivněna především mírou aktivního zapojení studenta. Studenti, kteří mohou s vizualizací aktivně manipulovat a krokovat ji, dosahují prokazatelně hlubšího porozumění než ti, kteří animaci pouze pasivně sledují. [5]

Empirické studie navíc potvrzují, že studenti využívající interaktivní prostředí dosahují signifikantně lepších výsledků v porovnání s kontrolními skupinami odkázanými na tradiční studijní materiály. [6] Nástroje, které se zaměřují na experimentování – kde student může například vkládat uzly a okamžitě sledovat vizuální odezvu struktury – fungují v podstatě jako moderní „hyperučebnice“ (*hypertextbooks*), jež efektivně propojují textovou teorii s okamžitou vizuální praxí. [7]

Tato akademická zjištění plně opodstatňují potřebu vývoje vysoce interaktivních aplikací pro podporu výuky teoretické informatiky a tvoří základní pedagogický předpoklad pro návrh systému popsaného v této práci.

Kapitola 3

Analýza existujících řešení

V této kapitole jsou popsány vybrané existující aplikace, které slouží k vizualizaci algoritmů a datových struktur. Cílem je analyzovat jejich funkce, uživatelské rozhraní a možnosti interakce, aby bylo možné identifikovat silné a slabé stránky těchto nástrojů. Tato analýza poslouží jako základ pro návrh a implementaci nového interaktivního nástroje, který bude lépe vyhovovat potřebám uživatelů.

Analýza se nejprve zaměří na vizualizaci datových struktur a následně na simulaci třídících algoritmů. Vybrané aplikace budou hodnoceny z hlediska jejich uživatelské přívětivosti, rozsahu podporovaných algoritmů a datových struktur, možnosti interakce a přizpůsobení vizualizací.

3.1 Vizualizace datových struktur

Nástroje popsané v této kapitole se zaměřují na vizualizaci různých datových struktur, jako jsou stromy, grafy, zásobníky a fronty.

3.1.1 Gnarley Trees

První aplikací, kterou tato práce analyzuje, je *Gnarley Trees*, interaktivní nástroj pro vizualizaci a manipulaci s různými typy stromových datových struktur. [8]

Obsah

Aplikace se zaměřuje především na stromy a haldy, v kontextu různých způsobů ukládání dat a jejich efektivity. U stromů jde například o binární vyhledávací stromy, červeno-černé stromy, AVL stromy, B-stromy a různé další varianty těchto struktur. U front a hald jsou to například prioritní fronty implementované pomocí běžných hald, binárních hald, Fibonacciho hald a dalších.

Každý typ datové struktury má možnost vizualizace základních operací, u stromů například vkládání, mazání a vyhledávání uzlů, u hald a front pak vkládání, inkrementaci hodnoty prvku a

odstraňování prvků s nejvyšší prioritou. Dále je zde také možnost vložit předem definovaný počet náhodných prvků a tím rychle zaplnit danou strukturu. Tyto interakce jsou prováděny pomocí uživatelského rozhraní, které umožňuje uživatelům zadávat hodnoty a sledovat, jak se datová struktura mění v reálném čase. Vizualizace je doplněna o animace, které umožňují mimo jiné i krokování operací a usnadňují pochopení dynamiky datových struktur. Animace jsou jednoduché, ale efektivní, včetně možnosti krokování vpřed i vzad.

Vizuál

Jedná se o webovou aplikaci napsanou v jazyce Java, vykreslování grafů je realizováno pomocí knihovny CheerpJ. Uživatelské rozhraní je přehledné, umožňuje snadný přístup k různým datovým strukturám a jejich operacím. Kromě toho aplikace ke všemu přikládá vysvětlivky, a zjednodušuje tak práci uživatelům, kteří se s těmito koncepty teprve seznamují.

Co se týče uživatelského přizpůsobení, stránka má pouze základní stylování, bez možnosti výběru barevných schémat nebo přizpůsobení vzhledu. Stránka také zřejmě má možnost změny jazyka z angličtiny do slovenštiny, ale její zapnutí není na první pohled zřejmé.

3.1.2 Data Structure Visualizations

Další aplikací je *Data Structure Visualizations*, výukový nástroj z Univerzity San Francisco, který nabízí vizualizace různých datových struktur a algoritmů. [9]

Obsah

Aplikace pokrývá širokou škálu témat, jak v oblasti datových struktur, tak i algoritmů. Mezi datové struktury patří například zásobníky, fronty, spojové seznamy, stromy (včetně binárních vyhledávacích stromů a hald) a grafy. U každé datové struktury jsou k dispozici vizualizace základních operací, jako je vkládání, mazání a vyhledávání prvků. U algoritmů jsou zahrnuty různé třídící algoritmy (například Bubble Sort, Insertion Sort, Merge Sort) a algoritmy pro prohledávání grafů (například Dijkstrův algoritmus nebo algoritmy na získání kostry grafu).

Interakce s vizualizacemi není sjednocená jako u minulé aplikace, každá datová struktura ji má implementovanou jinak. Animace pro jednotlivé operace jsou velmi jednoduché a oproti předchozí aplikaci postrádají vysvětlivky. Krokování funguje opět vpřed i vzad, včetně možnosti nastavení rychlosti animace. Jako nedostatek lze uvést i absenci možnosti naplnění datové struktury náhodnými hodnotami, což by zrychlilo testování a experimentování s různými scénáři. Třídící algoritmy jsou velmi jednoduché, umožňují uživateli náhodně zamíchat pole čísel, zvolit si třídící algoritmus a sledovat jeho průběh. Jako velmi zajímavé rozšíření aplikace poskytuje možnost napsat si vlastní algoritmus v jazyce JavaScript a vizualizovat jeho průběh.

Vizuál

Aplikace je webová, napsaná v jazyce JavaScript, s jednoduchým uživatelským rozhraním. Celkový vzhled není příliš moderní a uživatelsky přívětivý, ale funkční. Navigace na stránce je mírně komplikovaná, protože uživatel musí procházet různými odkazy, aby se dostal k požadované datové struktuře nebo algoritmu. Ovládání vizualizací je stylováno pomocí tlačítek a vstupních polí, ve stejném duchu jako předchozí aplikace.

Možnosti přizpůsobení jsou omezené, bez možnosti změny barevných schémat nebo vzhledu vizualizací. Responzivita stránky je značně omezená a její použití na mobilních zařízeních je obtížné. Aplikace je dostupná pouze v angličtině.

3.1.3 B Plus Tree

Poslední aplikací zaměřující se na datové struktury, jejíž analýzu tato práce provede, je *B Plus Tree*, interaktivní nástroj pro vizualizaci B+ stromů. [10]

Obsah

Aplikace se specializuje výhradně na B+ stromy, neobsahuje tedy jiné datové struktury. U B+ stromů jsou k dispozici vizualizace základních operací, jako je vkládání, mazání a vyhledávání klíčů. Mimo tyto poskytuje i možnost nastavit danému stromu maximální počet klíčů na vnitřní uzel nebo list a také možnost generování náhodných klíčů pro rychlé naplnění stromu. Chybí zde vysvětlivky k jednotlivým operacím, krátký popis struktury se věnuje spíše využití B+ stromů v databázových systémech.

Interakce s vizualizacemi je intuitivní, uživatel může zadávat hodnoty prostřednictvím vstupních polí a sledovat, jak se strom mění v reálném čase. Animace nemají možnost krokování, po provedení operace se strom aktualizuje rovnou do nového stavu, a také je není možné krokovat zpět.

Vizuál

Tato aplikace je rovněž webová, s velmi jednoduchým ale moderním uživatelským rozhraním. Obsah je tak malý, že není potřeba složitější navigace, vše je dostupné přímo na hlavní stránce. Ovládání vizualizací je realizováno pomocí tlačítek a vstupních polí. Stránka nenabízí žádné možnosti přizpůsobení vzhledu nebo změnu jazyka. Aplikace je velmi responzivní a funguje dobře na různých zařízeních.

3.1.4 Porovnání

V této kapitole jsou shrnuty klady a zápory jednotlivých analyzovaných aplikací zaměřených na vizualizaci datových struktur. Následující tabulka (Tabulka 3.1) poskytuje přehledné srovnání jejich vlastností.

Vlastnost	Gnarley Trees	Data Structure Visualizations	B Plus Tree
Podpora více datových struktur	Ano	Ano	Ne
Uživatelské přizpůsobení	Omezené	Omezené	Žádné
Krokování animací	Ano (vpřed i vzad)	Ano (vpřed i vzad)	Ne
Vysvětlivky k operacím	Ano	Ne	Ne
Generování náhodných dat	Ano	Ne	Ano
Responzivita	Dobrá	Slabá	Výborná
Jazyková podpora	Angličtina, slovenština	Angličtina	Angličtina

Tabulka 3.1: Porovnání analyzovaných aplikací pro vizualizaci datových struktur

Dále byla provedena analýza a porovnání konkrétních datových struktur, které výše zmíněné aplikace podporují. Tabulka 3.2 shrnuje, které datové struktury jsou v jednotlivých aplikacích podporovány.

Datová struktura	Gnarley Trees	Data Structure Visualizations	B Plus Tree
Binární vyhledávací strom	Ano	Ano	Ne
AVL strom	Ano	Ano	Ne
Červeno-černý strom	Ano	Ano	Ne
B-strom	Ano	Ano	Ne
B+ strom	Ne	Ano	Ano
Spojový seznam	Ano ¹	Ne	Ne
Zásobník	Ano ¹	Ano	Ne
Fronta	Ano ¹	Ano	Ne
Halda	Ano	Ano	Ne

Tabulka 3.2: Podporované datové struktury v analyzovaných aplikacích

Z uvedeného shrnutí je patrné, že nejkomplexnější podporu datových struktur nabízí aplikace *Data Structure Visualizations*, která pokrývá téměř všechny uvedené struktury. *Gnarley Trees* nabízí také širokou škálu datových struktur, ale některé z nich jsou dostupné pouze v teoretické části aplikace a nejsou vizualizovány. Nejméně obsáhlá je aplikace *B Plus Tree*, která se sice specializuje pouze na B+ stromy, ale zato je v této oblasti velmi dobře propracovaná.

3.2 Simulace třídících algoritmů

Tato kapitola se věnuje aplikacím, které umožňují vizualizaci a simulaci různých třídících algoritmů, jako jsou Bubble Sort, Quick Sort, Merge Sort a další.

¹Pouze v teoretické části.

3.2.1 Sort Visualizer

Aplikace *Sort Visualizer* je interaktivní nástroj pro vizualizaci různých třídících algoritmů, který umožňuje uživatelům sledovat průběh třídění v reálném čase spolu s popisem algoritmů a jejich časovou a prostorovou složitostí. [11]

Obsah

Sort Visualizer nabízí simulace několika běžných třídících algoritmů, které dělí na logaritmické (Quick Sort, Merge Sort, Heap Sort), kvadratické (Bubble Sort, Insertion Sort, Selection Sort) a další (např. Radix Sort). Přidává také možnost, jak si uživatel může vytvořit vlastní algoritmus pomocí jazyka JavaScript a vizualizovat jeho průběh.

Interakce se simulací je značně omezená, uživatel si může pouze zvolit algoritmus, velikost pole a rychlost animace. Animace spočívá v zobrazení sloupců reprezentujících hodnoty v poli, které se mění podle průběhu třídění. Zajímavé zpestření animace jsou zvukové efekty, které doprovázejí porovnávání a výměny prvků a svým tónem reprezentují hodnotu daného prvku. Krokování animace není možné.

Každý algoritmus je doplněn o krátký popis jeho principu, časové složitosti v nejlepším, průměrném a nejhorším případě a také o prostorovou složitost. Přiloženy jsou také zdrojové kódy daného algoritmu v několika programovacích jazycích. Zdrojové kódy jsou však statické, nelze je během simulace nijak měnit a také nejsou zvýrazněny části kódu, které se právě vykonávají.

Vizuál

Tato webová aplikace je napsaná pomocí webového frameworku Flask, s moderním a přehledným uživatelským rozhraním. Navigace na stránce je jednoduchá, všechny třídící algoritmy jsou vždy dostupné z navigačního panelu. Samotná vizualizace je vizuálně velmi poutavá a v doprovodu se zvukovými efekty působí přívětivě.

Aplikace nenabízí žádné možnosti přizpůsobení vzhledu nebo změnu jazyka. Výchozí barevné schéma je v temném režimu. Responzivita stránky je skvělá, stránka funguje dobře téměř v jakémkoliv rozlišení.

3.2.2 VisuAlgo

Jedná se o další webovou aplikaci, sloužící jako výukový nástroj pro vizualizaci všemožných algoritmů a datových struktur, která byla vyvinuta na National University of Singapore. Aplikace *VisuAlgo* nabízí širokou škálu vizualizací, včetně třídících algoritmů. [12]

Obsah

Rozsah této aplikace je velmi široký, pokrývá mnoho témat v oblasti algoritmů a datových struktur. Stránka nabízí vizualizace pro různé datové struktury, jako jsou zásobníky, fronty, spojové seznamy, stromy a grafy. Kromě toho obsahuje i vizualizace různých algoritmů, včetně třídících algoritmů, algoritmů pro prohledávání grafů a dalších. Obsahuje i interaktivní cvičení a kvízy, které pomáhají uživatelům lépe pochopit koncepty.

V kontextu třídících algoritmů nabízí vizualizace pro algoritmy jako Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, Quick Sort a Heap Sort. Animace jsou jednoduše znázorněny opět pomocí sloupců reprezentujících hodnoty v poli. Uživatel má možnosti nastavení rychlosti přehrávání, plné krokování animace vpřed i vzad a také možnost pozastavení animace. Kromě toho aplikace poskytuje i podrobné vysvětlení principu každého algoritmu, včetně jeho časové a prostorové složitosti. Během animace je přístupný i pseudokód algoritmu, který je zvýrazňován podle aktuálně vykonávané části.

Vizuál

Webové stránky mají značně zaplněné uživatelské rozhraní, pro nové uživatele se tedy mohou zdát trochu matoucí. Navigace na stránce je relativně dobře organizovaná, s jasnými odkazy na různé sekce a témata. Stránka s vizualizacemi se může zdát přeplněná ještě více, protože obsahuje mnoho ovládacích prvků a informací. Toto okno také obsahuje vysvětlivky, které jsou provedeny ve vyskakovacích oknech, které uživatele zahltní při prvním použití.

Výchozí barevné schéma je světlé, uživatel si nemůže změnit režim na tmavý. Responzivita stránky je dobrá, když se rozlišení zmenší na nedostatečnou šířku, aplikace uživatele upozorní, že vizualizace nemusí fungovat správně. Aplikace podporuje změnu jazyka, na výběr uživatelé mají z angličtiny, čínštiny a indonéštiny.

3.2.3 Toptal Sorting Algorithms Animations

Poslední analyzovanou aplikací je *Sorting Algorithms Animations* od společnosti Toptal, téměř jednostránkový webový nástroj pro vizualizaci a porovnání různých třídících algoritmů. [13]

Obsah

Aplikace nabízí vizualizace pro několik běžných třídících algoritmů, které již byly zmíněny výše. Jednotlivé algoritmy jsou umístěny v matici, kde každý sloupec reprezentuje jeden algoritmus a každý řádek představuje typ vstupních dat (např. náhodná data, téměř seřazená data nebo obráceně seřazená data). Pomocí tohoto rozložení může uživatel snadno porovnat výkon různých algoritmů na různých typech vstupů.

Interakce s vizualizacemi je velmi omezená, uživatel si může pouze zvolit algoritmus a typ vstupních dat. Animace jsou jednoduché, zobrazují sloupce reprezentující hodnoty v poli, které se mění podle průběhu třídění. Krokování animace není možné.

Uživatel má také možnost zobrazit si detail konkrétního algoritmu, kde je k němu přiložena vysvětlivka, pseudokód a časová a prostorová složitost. Rozkliknutí konkrétního datového typu zase zobrazí konkrétní porovnání časové složitosti všech algoritmů na daném typu vstupu a výčet poznatků o jejich výkonu.

Vizuál

Jedná se o velmi jednoduchou webovou aplikaci. Hlavní stránka obsahuje pouze výše uvedenou matici algoritmů a typů vstupních dat spolu s obecnou diskuzí na téma třídících algoritmů. Detail algoritmu je veden ve stejném stylu, animace jsou pouze větší a jsou doplněny o pseudokód a diskuzi ke konkrétnímu algoritmu.

Stránka nemá žádné možnosti přizpůsobení vzhledu nebo změnu jazyka. Výchozí barevné schéma je světlé. Jako jediná z dosud analyzovaných aplikací také obsahuje reklamy. Responzivita stránky je dobrá, funguje dobře na různých zařízeních.

3.2.4 Porovnání

V této kapitole jsou shrnuty klady a zápory jednotlivých analyzovaných aplikací zaměřených na vizualizaci třídících algoritmů. Následující tabulka (Tabulka 3.3) poskytuje přehledné srovnání jejich vlastností.

Vlastnost	Sort Visualizer	VisuAlgo	Sorting Algorithms Animations
Uživatelské přizpůsobení	Žádné	Žádné	Žádné
Krokování animací	Ne	Ano (vpřed i vzad)	Ne
Vysvětlivky k operacím	Ano	Ano	Ano
Zvukové efekty	Ano	Ne	Ne
Responzivita	Výborná	Dobrá	Dobrá
Jazyková podpora	Angličtina	Angličtina, čínština, indonéština	Angličtina

Tabulka 3.3: Porovnání analyzovaných aplikací pro vizualizaci třídících algoritmů

Z konkrétní analýzy vyšlo najevo, že všechny tři aplikace podporují základní množinu nejznámějších třídících algoritmů, ale liší se v možnostech interakce a uživatelského přizpůsobení. *VisuAlgo* nabízí nejvíce možností interakce, včetně krokování animací a podrobného pseudokódu, zatímco *Sort Visualizer* a *Topical Sorting Algorithms Animations* mají omezené možnosti interakce. Zvukové efekty jsou unikátní funkcí *Sort Visualizer*, která přidává další vrstvu zážitku z vizualizace.

3.3 Shrnutí

Analýza existujících řešení, provedená v této kapitole, poskytla cenné poznatky o současných nástrojích pro vizualizaci datových struktur a třídících algoritmů. Z analýzy vyplynulo, že zatímco některé aplikace nabízejí širokou škálu funkcí a interakcí, jiné se zaměřují na specifické datové struktury nebo algoritmy s omezenými možnostmi přizpůsobení.

Na základě porovnání podporovaných datových struktur bylo rozhodnuto, že modelová aplikace bude zahrnovat vizualizaci následujících datových struktur: binární vyhledávací strom, AVL strom, červeno-černý strom, B-strom, B+ strom, spojový seznam, zásobník, fronta a halda. Tato volba byla učiněna s ohledem na jejich význam v oblasti informatiky a jejich časté využití v praxi.

Co se týče třídících algoritmů, modelová aplikace bude podporovat vizualizaci následujících algoritmů: Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, Quick Sort a Heap Sort. Tyto algoritmy byly vybrány na základě jejich rozšířenosti a různorodosti přístupů k třídění dat.

Zdůrazněna byla také potřeba uživatelsky přívětivého rozhraní, které umožní snadnou navigaci a interakci s vizualizacemi. Dále byly identifikovány funkce, které by měly být zahrnuty do modelové aplikace, včetně možnosti krokování animací, generování náhodných dat a poskytování vysvětlivek k jednotlivým operacím.

Kapitola 4

Požadavky na výukovou aplikaci

Pro úspěšný vývoj a implementaci webové aplikace pro vizualizaci datových struktur a algoritmů je nezbytné stanovit jasné požadavky, které budou sloužit jako vodítko během celého vývojového procesu. Jako inspirace pro tyto požadavky byly použity existující nástroje popsané v Kapitole 3. Požadavky jsou rozděleny do kategorií dle FURPS analýzy, zahrnujících funkcionalitu, použitelnost, spolehlivost, výkonnost a podporovatelnost a jsou popsány v následujících kapitolách.

4.1 Funkcionalita

Tato kapitola definuje hlavní funkční požadavky na systém, které zahrnují vizualizační schopnosti, interaktivní prvky a vzdělávací obsah.

1. Moduly pro vizualizaci datových struktur

- Implementace a vizualizace struktur: zásobník, fronta, lineární seznamy, pole a stromy.
- Grafická simulace standardních operací nad těmito strukturami.

2. Moduly pro simulaci algoritmů

- Algoritmy řazení: Bubble sort, Merge sort, Quick sort a další.
- Algoritmy vyhledávání: lineární a binární vyhledávání.

3. Interaktivní ovládání simulací

- Funkce krokování algoritmu (vpřed i vzad) pro detailní analýzu průběhu.
- Možnost modifikace vstupních dat v reálném čase s okamžitým promítnutím do vizualizace.

4. Teoretický a metodický podklad

- Formální vysvětlení principů a typických scénářů užití u každé struktury a algoritmu.
- Specifikace časové a prostorové algoritmické složitosti.
- Zobrazení odpovídajícího pseudokódu pro lepší pochopení implementace.

5. Lokalizace rozhraní

- Plná podpora vícejazyčného rozhraní, minimálně v rozsahu českého a anglického jazyka.

6. Architektura na straně klienta

- Provoz aplikace v rámci webového prohlížeče bez nutnosti instalace dodatečných doplňků.
- Provádění veškerých simulací a interakcí lokálně na zařízení uživatele.
- Persistence uživatelského nastavení a historie prostřednictvím webového úložiště (local storage).

4.2 Použitelnost

Cílem požadavků na použitelnost je zajistit nízkou bariéru vstupu a intuitivní ovládání pro cílovou skupinu studentů.

1. Ergonomie a ovládání

- Intuitivní uživatelské rozhraní doplněné o kontextové nápovědy pro složitější ovládací prvky.
- Konzistentní navigační systém pro přehledný přechod mezi moduly struktur a algoritmů.

2. Didaktická podpora

- Synergie vizualizace, teorie a vysvětlujících textů pro podporu uživatelů s různou úrovní vstupních znalostí.

3. Vizuální prezentace

- Implementace plynulých animací pro jasnou identifikaci změn stavu datových struktur.
- Moderní, přehledný a čistý design uživatelského rozhraní odpovídající současným standardům (UI/UX).

4.3 Spolehlivost

Požadavky na spolehlivost se zaměřují na stabilitu systému a korektní zpracování uživatelských dat.

1. Stabilita systému

- Zajištění plné funkčnosti a stability při zpracování datových sad o rozsahu až 100 prvků.
- Validace a ošetření všech neplatných uživatelských vstupů před jejich zpracováním.

2. Odolnost vůči chybám

- Implementace informativních chybových hlášení, která uživatele srozumitelně navedou k nápravě při nekorektní operaci.

4.4 Výkonnost

Důležitým aspektem výkonnosti je plynulost grafického výstupu a rychlá odezva na interakci.

1. Optimalizace vykreslování

- Zajištění konstantní plynulosti animací nezávisle na zvolené velikosti vstupních dat (v rámci stanovených limitů).

2. Efektivita výpočtů

- Optimalizované zpracování algoritmů na straně klienta pro minimalizaci latence mezi uživatelským vstupem a vizuální odezvou.

4.5 Podpořitelnost a udržitelnost

Požadavky definují technický rámec pro budoucí rozšiřování a údržbu aplikace.

1. Modularita a separace zájmů

- Striktní oddělení vizualizační vrstvy od aplikační logiky.
- Definice jasného a jednoduchého rozhraní (API) mezi jednotlivými komponentami systému.
- Architektura UI umožňující snadné úpravy vzhledu bez zásahu do logiky (např. pomocí CSS proměnných).

2. Technologický zásobník

- Vývoj klientské části s využitím komponentního frameworku Svelte.
- Využití specializovaných knihoven pro vizualizaci dat (např. D3.js nebo vis.js).

3. Dokumentace a rozšiřitelnost

- Detailní dokumentace systémové architektury pro usnadnění budoucího vývoje.
- Návrh systému umožňující snadné přidávání nových datových struktur, algoritmů nebo nových podporovaných jazyků.

Kapitola 5

Analýza a volba použitých technologií

Pro úspěšnou realizaci projektu je prvně nutné vhodně zvolit technologie, které budou použity v jeho implementaci. Na základě analýzy v této kapitole budou vybrány technologie týkající se architektury aplikace a strategie renderování, pomocí čehož bude dále určen vhodný webový framework a knihovna pro vizualizaci grafů a stromů. Na závěr bude zmíněn také nástroj pro automatizaci procesu sestavení a nasazení aplikace, stejně jako platforma pro hosting výsledné statické webové aplikace. Tato analýza bude začátkem implementace výukové aplikace a bude sloužit jako základ pro její vývoj.

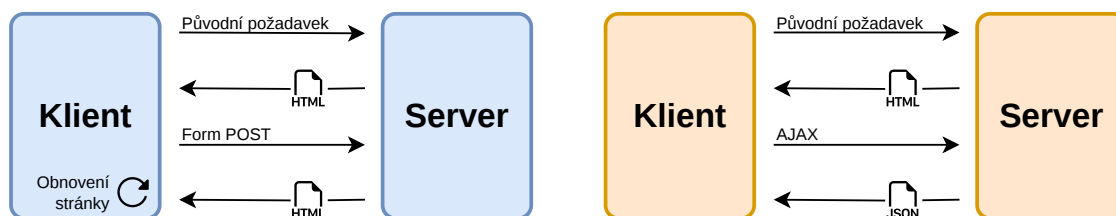
5.1 Architektura aplikace

Ještě před samotnou volbou vhodného frameworku je důležité provést analýzu možné softwarové architektury pro výukovou aplikaci. Architektura určuje nejen to, kde bude implementována aplikační logika, ale také definuje způsob komunikace mezi klientem a serverem a strategii vykreslování uživatelského rozhraní.

5.1.1 Klasická webová architektura

Tradičním přístupem je více stránková architektura (*Multi-Page Architecture*, dále *MPA*), založená na modelu rozhraní, kde každá interakce uživatele nebo odeslání formuláře vyvolá kompletní obnovení rozhraní ze serveru. Logika aplikace je v tomto případě primárně soustředěna na straně serveru. [14]

Příklad životního cyklu v této architektuře je zobrazen na Obrázku 5.1 (levá část). Při každé akci uživatele je odeslán HTTP požadavek na server, který data zpracuje a jako odpověď vrátí novou HTML stránku. V praxi to znamená nutnost znovunačtení celého dokumentu s každou interakcí, a to může vést k prodlevám a neefektivnímu využití přenosového pásma, jelikož se opakovaně přenáší i statické části rozhraní (např. navigace).



Obrázek 5.1: Srovnání životních cyklů stránek v klasické architektuře a v SPA

5.1.2 Jednostránková webová aplikace

Jako reakce na omezení tradičního modelu se v posledních letech stal standardem přístup zvaný jednostránková webová aplikace (*Single Page Application*, dále *SPA*). Hlavním rysem SPA je, že po úvodním načtení základního značení (markup) aplikace již nevyžaduje opětovné načítání celé stránky. Namísto toho aplikace nezávisle žádá pouze o nezbytná data, typicky prostřednictvím technologie AJAX. [14]

Jak je znázorněno na pravé části Obrázku 5.1, odpovědi ze serveru jsou přenášeny ve formátu JSON, což prohlížeči umožňuje reaktivně aktualizovat pouze dotčené části DOM bez přerušení uživatelského zážitku. Tento model simuluje chování nativních desktopových aplikací s minimální odezvou. [15]

Výhodou tohoto přístupu je plynulejší interaktivita, která je pro výukový nástroj s animacemi zásadní. Díky tomu, že logika běží na straně klienta, mohou být aplikace částečně funkční i při nestabilním připojení, přičemž synchronizace probíhá po obnovení spojení. Nevýhodou je však vyšší náročnost na počáteční stažení JavaScriptových balíčků a komplikovanější optimalizace pro vyhledávače (SEO), jelikož obsah není pro roboty (*web-crawlers*) okamžitě dostupný v surovém HTML. [15] Pro účely interního výukového nástroje však výhody plynulého vykreslování animací převažují nad požadavky na SEO.

5.2 Strategie renderování

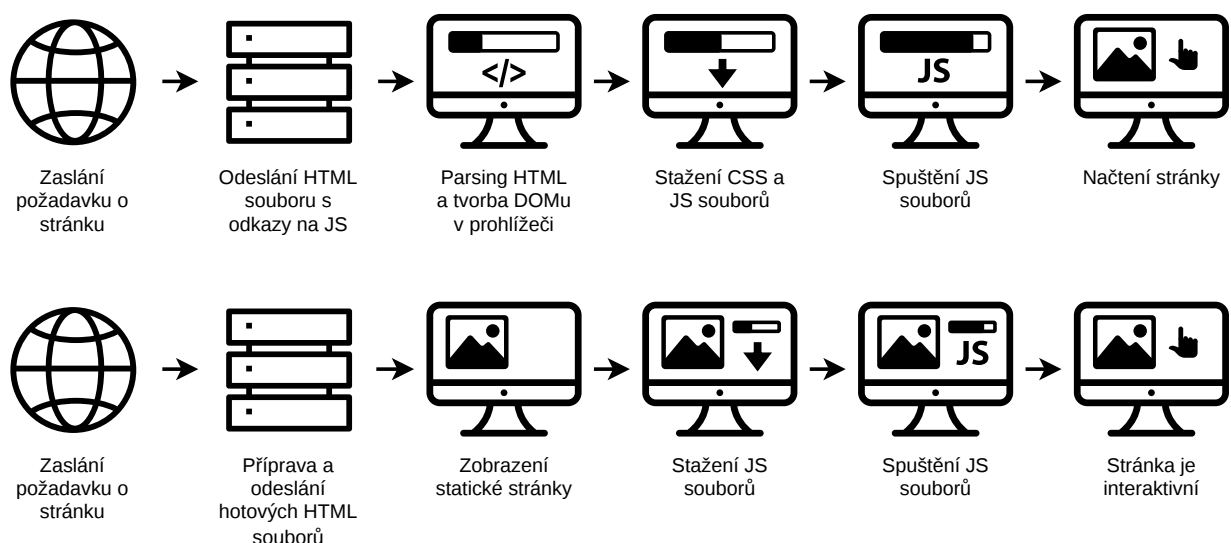
Dalším důležitým aspektem architektury je strategie renderování, tedy způsob, jakým se generuje a zobrazuje obsah na stránce. Různé strategie se liší zejména tím, kde se provádí zpracování a generování HTML kódu, a to zásadně ovlivňuje nejen uživatelský zážitek a výkon, ale také optimalizaci pro vyhledávače (SEO).

5.2.1 Client-Side Rendering

Varianta Client-Side Rendering (CSR) je běžnou strategií pro jednostránkové aplikace (SPA), u které se veškeré generování HTML kódu provádí až na straně klienta. Server vrací pouze minimální

prázdný HTML dokument s odkazem na JavaScriptový soubor, který klient musí stáhnout a spustit pro zobrazení obsahu. Princip fungování CSR je zobrazen v horní části Obrázku 5.2.

Hlavní výhodou CSR je plynulá interaktivita a rychlé přechody po prvotním načtení aplikace. Zároveň se minimalizuje výpočetní zátěž serveru, který figuruje pouze jako poskytovatel dat. Na druhou stranu, nutnost stažení a spuštění objemných skriptů vede k delší prodlevě při prvním načtení, což může zhoršit uživatelský zážitek na zařízeních s nižším výkonem. Další nevýhodou je horší indexovatelnost pro vyhledávače (SEO), jelikož webové prohlížeče (*web-crawlers*) při prvním průchodu aplikací často nevidí vygenerovaný obsah. [16, 17]



Obrázek 5.2: Srovnání strategií renderování CSR a SSR

5.2.2 Server-Side Rendering

Jak už název napovídá, druhou strategií je Server-Side Rendering (SSR), kdy se zpracování a generování HTML kódu provádí na straně serveru. Klient tak obdrží hotovou stránku, kterou může prohlížeč okamžitě zobrazit (viz dolní část Obrázku 5.2). Tato prvotní stránka však zatím není interaktivní. Plnou funkčnost získává až po stažení JavaScriptu prostřednictvím procesu zvaného *hydrate*. Během ní klientský skript převezme kontrolu nad již vyrenderovaným statickým HTML a naváže na něj potřebnou aplikační logiku a interaktivitu. [18]

Zásadní výhodou SSR je vynikající podpora SEO, jelikož je obsah okamžitě přístupný webovým prohlížečům bez nutnosti spouštět JavaScript. Kvantitativní měření potvrzují, že SSR dosahuje výrazně lepších výsledků v metrikách prvotního načtení (TTFB), čímž poskytuje rychlou vizuální odezvu nezávisle na výkonu koncového zařízení. Nevýhodou je naopak vyšší zátěž serveru, který musí dynamicky generovat HTML pro každý požadavek a jehož vytížení může při větším množství uživatelů vést k degradaci výkonu. [17, 16]

5.2.3 Static Site Generation

Poslední strategií v tomto výčtu je Static Site Generation (SSG). Přístup je podobný SSR v tom, že klient obdrží předpřipravený HTML dokument, avšak k jeho generování nedochází při každém požadavku, ale již ve fázi sestavování aplikace (*build time*). Server následně klientovi pouze distribuuje hotové statické soubory.

Mezi výhody SSG patří bleskově rychlé načítání, výborné SEO a minimální nároky na provoz serveru, jelikož odpadá nutnost generování dat v reálném čase. Nevýhodou je složitější správa dynamického obsahu – jakákoliv změna dat vyžaduje nové sestavení celé aplikace, které je u rozsáhlých projektů s často se měnícím obsahem značně neefektivní. [17]

Důležitým faktem je, že SSG nevylučuje následnou interaktivitu. V kontextu moderních frameworků se často využívá k vygenerování statického základu, který je po načtení na straně klienta *hydratován* a dále funguje jako plnohodnotná SPA. Tento kombinovaný model je ideální pro výukové aplikace s fixními teoretickými materiály, kde je prioritou rychlost doručení a vysoká míra interaktivity v samotných vizualizacích. [17]

5.3 Webový framework

V dnešní době existuje mnoho různých webových frameworků, které umožňují vývoj jednostránkových webových aplikací. Mezi nejpopulárnější patří React, Angular, Vue a Svelte. Každý z nich má své výhody a nevýhody, které je třeba zvážit při výběru toho nejvhodnějšího pro konkrétní projekt.

5.3.1 React

React je jedním z nejpopulárnějších JavaScriptových frameworků pro vývoj webových aplikací. Je vyvíjen společností Meta a má velkou komunitu vývojářů, a proto pro něj existuje mnoho knihoven a nástrojů, které usnadňují vývoj. React používá komponentový přístup a zakládá se na deklarativním stylu programování UI. [19]

Speciálním konceptem, který React zavádí, je tzv. *Virtual DOM*, který slouží jako virtuální reprezentace skutečného DOM. Když dojde ke změně stavu aplikace, React nejprve aktualizuje Virtual DOM a poté porovná tuto aktualizaci s předchozím stavem Virtual DOM. Na základě tohoto porovnání React efektivně aktualizuje pouze ty části skutečného DOM, které se změnily. Tomuto procesu se říká *reconciliation* a je jedním z hlavních faktorů, které přispívají k vysokému výkonu knihovny React. [19]

5.3.2 Angular

Jako další z nejznámějších webových frameworků je zde Angular, který je vyvíjen společností Google. Jedná se o kompletní framework, který nabízí širokou škálu funkcí a nástrojů pro vývoj webových

aplikací. Je založen na TypeScriptu, používá komponentový přístup k vývoji UI a také poskytuje robustní systém pro správu stavu nebo směřování. [20]

Angular je známý svou komplexností, je zaměřen na velké a složité aplikace, které vyžadují robustní řešení. Jeho učení může být náročnější než u jiných frameworků, ale nabízí mnoho funkcí, které mohou být užitečné pro vývoj rozsáhlých aplikací. [20]

5.3.3 Vue

Vue je často považován za lehčí a jednodušší alternativu k Reactu a Angularu. Je vyvíjen komunitou a nabízí jednoduchý a intuitivní způsob, jak vytvářet interaktivní uživatelská rozhraní. Podobně jako React využívá komponenty a virtuální DOM, ale jeho syntaxe je často považována za přehlednější a snadněji pochopitelnou. Naučit se ho je poměrně snadné, proto je často doporučován pro začínající vývojáře. [21]

Používá se pro vývoj menších a středně velkých aplikací, je možné ho využít jako SPA i fullstack SSR či SSG řešení. Jeho flexibilita a jednoduchost ho činí oblíbeným mezi vývojáři, kteří hledají rychlé a efektivní řešení pro své projekty. [21]

5.3.4 Svelte

Svelte je posledním frameworkem, který bude v této kapitole zmíněn. Je relativně nový a přináší inovativní přístup k vývoji webových aplikací, protože se zaměřuje na kompilaci. Na rozdíl od ostatních frameworků, které provádějí většinu své práce v prohlížeči, Svelte kompiluje komponenty do vysoce optimalizovaného JavaScriptu, který běží přímo v prohlížeči. To znamená, že nevyužívá virtuální DOM a místo toho generuje efektivní kód, který aktualizuje DOM přímo, což může vést k lepšímu výkonu a menší velikosti balíčku. [22]

Podobně jako výše zmíněné frameworky, i Svelte používá komponenty a vývojáři umožňuje psát aplikaci deklarativně. Jeho syntaxe je velmi jednoduchá a přehledná, a spojuje jak HTML, CSS, tak JavaScript do jednoho souboru. Komponenta je následně zkompilována do efektivního JavaScript souboru, který se načítá a spouští v prohlížeči. Absence virtuálního DOM a minimalizace kódu, který musí prohlížeč interpretovat za běhu, činí Svelte ideálním pro interaktivní vizualizace, kde je vyžadována vysoká snímková frekvence animací. [22]

SvelteKit

Pro zjednodušení vývoje webových aplikací byl jako nadstavba na Svelte vyvinut meta-framework SvelteKit, který poskytuje nástroje jako směřování, správu stavu, podporu pro různé strategie renderování (CSR, SSR, SSG) a další. Pro potřeby této práce je důležitá podpora adaptérů, konkrétně *adapter-static*, který umožňuje transformaci aplikace do sady statických souborů připravených pro hosting bez nutnosti serveru. [22]

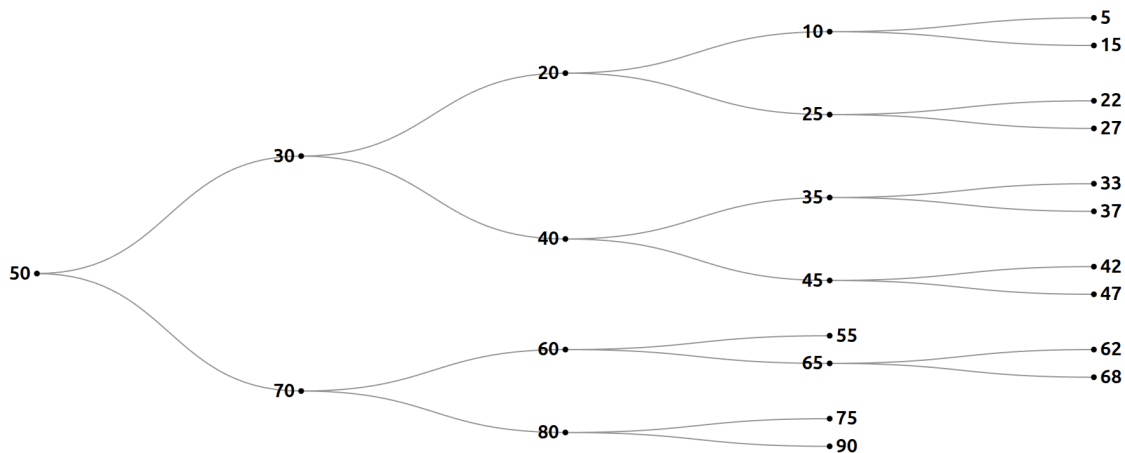
5.4 Grafová knihovna

Jak již bylo zmíněno v Kapitole 4, výuková aplikace bude obsahovat vizualizace datových struktur, nejčastěji stromů a grafů. Je tedy nutné zvolit vhodnou knihovnu pro jejich zobrazení, přičemž je důležité zvážit nejen možnosti zobrazení, ale také možnosti animace a interakce s grafem.

Pro účely porovnání byla u každé zvažované knihovny implementována stejná vizualizace binárního vyhledávacího stromu, aby bylo zřejmé, jaké možnosti zobrazení a interakce nabízí každá z nich. Následující podkapitoly se věnují jednotlivým knihovnám a jejich porovnání.

5.4.1 Knihovna D3

Knihovna D3.js je open-source nástrojem umožňujícím vizualizaci dat pomocí webových standardů, jako jsou HTML, SVG a CSS. Grafy jsou vytvářeny pomocí DOM manipulace a díky tomu nabízí velkou interaktivitu a možnosti přizpůsobení. D3 je velmi komplexní a nabízí širokou škálu funkcí pro práci s daty, ale může být náročný na učení a implementaci, zejména pro složitější vizualizace. [23]



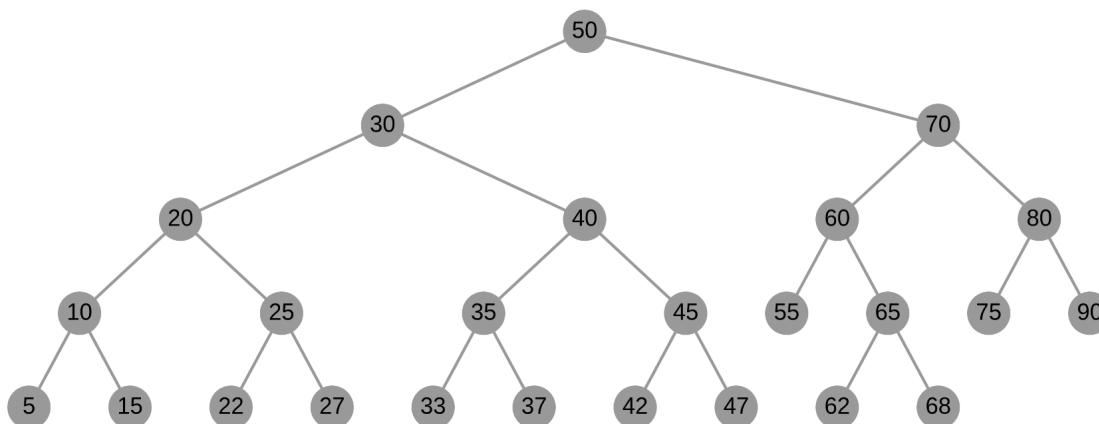
Obrázek 5.3: Příklad zobrazení stromu pomocí D3.js

Příklad použití D3 pro zobrazení stromu je zobrazen na Obrázku 5.3, přičemž knihovna zde využívá algoritmus *tidy tree* pro úspornější uspořádání hierarchických dat. D3 se zaměřuje hlavně na vizualizaci velkého množství dat a dokumentů, pro tento projekt se to však jeví zbytečně složité a náročné na implementaci, protože se nejedná o rozsáhlou vizualizaci, ale spíše o jednoduché zobrazení stromů a grafů s interakcí a animací.

5.4.2 Knihovna Cytoscape

Další možností je knihovna Cytoscape.js, která je navržena speciálně pro vizualizaci grafů a sítí. Opět se jedná o open-source nástroj, oproti tomu předchozímu však jeho vykreslování neprobíhá pomocí DOM, ale využívá HTML5 canvas, který umožňuje efektivnější zobrazení a lepší výkon,

zejména u větších grafů. Cytoscape.js nabízí širokou škálu funkcí pro práci s grafy, je poměrně snadný na použití a má dobrou dokumentaci. [24]



Obrázek 5.4: Příklad zobrazení stromu pomocí Cytoscape.js

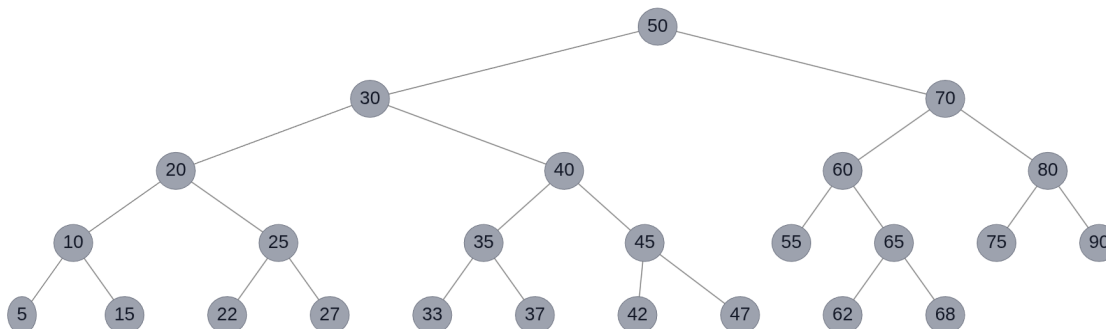
Opět je přiložen příklad vykreslení stejného binárního vyhledávacího stromu jako v předchozím případě, tentokrát však pomocí Cytoscape.js (Obrázek 5.4). Tento typ zobrazení je již uživatelsky přívětivější a přehlednější a také poskytuje více možností pro interakci s grafem, například přetahování uzlů, přiblížení či oddálení nebo různé styly zobrazení.

5.4.3 Knihovna vis-network

Zvažována byla také knihovna vis-network, která je součástí většího balíčku vis.js, jež byl vytvořen pro jednoduchou vizualizaci 2D i 3D grafů, strukturovaných dat, časových os a také právě sítí a grafů. Stejně jako Cytoscape.js využívá pro vykreslování HTML5 canvas a nabízí tudíž dobrý výkon i u větších grafů. Samotná knihovna vis-network se snaží být co nejjednodušší pro vývojáře, pro zobrazení grafu stačí pouze definovat uzly a hrany a zbytek se postará o vykreslení a interakci. Nabízí také různé možnosti pro přizpůsobení zobrazení, například různá rozvržení, styly a možnosti interakce. [25]

Na Obrázku 5.5 je zobrazen strom vykreslený pomocí vis-network, přičemž se opět jedná o stejná data jako v předchozích případech. Tento způsob zobrazení je velmi přehledný a uživatelsky přívětivý, co se týče interakce s grafem poskytuje podobné možnosti jako Cytoscape.

Animace grafu je zajištěna pomocí fyzikálního modelu, který simuluje síly mezi uzly a hranami, přímou možnost nastavení animace však knihovna nenabízí, ale existuje zde alespoň možnost nastavení pozic uzlů manuálně. Dalším přínosem této knihovny je implementace pro hierarchický layout, která je zabudovaná přímo do nastavení grafu, takže není nutné počítat pro každý vrchol souřadnice. Poskytuje taktéž velmi užitečnou možnost odposlouchávání událostí, jako například kliknutí na uzel, čehož bude možné využít pro implementaci interakce s grafem.



Obrázek 5.5: Příklad zobrazení stromu pomocí vis-network

5.5 Automatizace a nasazení (CI/CD)

Pro zajištění efektivního vývoje a nasazení aplikace je důležité implementovat procesy pro kontinuální integraci a kontinuální nasazení (CI/CD). Tyto procesy umožňují automatizovat testování, kompilaci a nasazení aplikace, čímž zrychlují vývoj a udržují aplikaci vždy v aktuálním stavu. Pro tento projekt bude využit GitHub Actions, nástroj pro automatizaci pracovních postupů přímo integrovaný do platformy GitHub.

5.5.1 GitHub Actions

Tato platforma pro CI/CD umožňuje kompilovat, testovat a nasazovat kód přímo z repozitáře na GitHubu, přičemž spuštění těchto procesů lze nastavit na různé události, například při pushnutí kódu do repozitáře nebo při vytvoření požadavku typu pull request. Mimo to je možné pomocí něj spravovat štítky nebo provádět další akce, které mohou být užitečné pro vývojový proces. Spouštění pracovních postupů se provádí na virtuálních strojích, které jsou k dispozici pro různé operační systémy a prostředí a jsou poskytnuty zdarma platformou GitHub. [26]

Proces je definován pomocí YAML souborů, jež specifikují jednotlivé kroky a akce, které se mají provést. Příklad takového souboru je přiložen na Výpisu 8. Tento proces má následující fáze:

1. **Příprava prostředí (Checkout & Setup):** Virtuální stroj stáhne zdrojový kód z repozitáře a připraví potřebné prostředí (Node.js).
2. **Instalace závislostí (Install):** Stažení všech externích knihoven definovaných v projektu, které jsou nezbytné pro sestavení aplikace.
3. **Sestavení aplikace (Build):** Kompilace zdrojového kódu, převedení Svelte komponent do JavaScriptu a generování optimalizovaných statických souborů připravených pro nasazení.
4. **Nasazení (Deployment):** Automatické nahrání výsledných souborů na hosting (GitHub Pages), čímž se nová verze okamžitě zpřístupní uživatelům.

```
1  name: Deploy to GitHub Pages
2
3  on:
4    push:
5      branches: ["main"]
6
7  jobs:
8    build_and_deploy:
9      runs-on: ubuntu-latest
10     steps:
11       - uses: actions/checkout@v4
12
13       - name: Setup Node.js
14         uses: actions/setup-node@v4
15         with:
16           node-version: "20"
17
18       - name: Install dependencies
19         run: npm install
20
21       - name: Build project
22         run: npm run build
23
24       - name: Deploy to GitHub Pages
25         uses: JamesIves/github-pages-deploy-action@v4
26         with:
27           folder: build
28           branch: gh-pages
```

Výpis 8: Příklad konfigurace GitHub Actions pro CI/CD

Tento přístup minimalizuje riziko chyb při ručním nasazování a zaručuje, že verze aplikace na produkčním serveru vždy odpovídá aktuálnímu stavu zdrojového kódu v hlavní větvi repozitáře.

5.5.2 GitHub Pages

Pro hosting statické webové aplikace bude využita služba GitHub Pages, která umožňuje snadné a bezplatné nasazení statických stránek přímo z repozitáře na GitHubu. GitHub Pages poskytuje jednoduchý způsob, jak hostovat statické webové stránky bez nutnosti spravovat vlastní server nebo platit za hostingové služby. Stačí pouze nahrát statické soubory do repozitáře a nastavit správnou větev pro publikování, a stránka je okamžitě dostupná na veřejné URL adrese. [27]

5.6 Shrnutí

Na základě provedené analýzy byl pro realizaci projektu zvolen moderní technologický stack postavený na architektuře **SPA**. Co se týče strategie renderování, bude využita kombinace **SSG** pro

generování statických HTML souborů a **CSR** pro zajištění interaktivity, čímž se umožní rychlé načítání stránek a zároveň poskytne plynulý uživatelský zážitek.

Jako framework byl zvolen **Svelte** spolu s jeho meta-frameworkem **SvelteKit**, především pro jeho efektivitu, absenci virtuálního DOM a nativní podporu animací, což je stěžejní pro plynulé zobrazení kroků algoritmů a operací nad datovými strukturami. Využití **TypeScriptu** v rámci celého vývoje zajišťuje typovou bezpečnost při práci se složitými datovými strukturami.

Pro samotnou vizualizaci grafů a stromů byla vybrána knihovna **vis-network**, která díky svému hierarchickému rozvržení a možnosti odposlouchávání událostí umožňuje přehledné a interaktivní zobrazení jak pro jednoduché, tak i složitější datové struktury. Celý proces sestavení a nasazení aplikace je automatizován pomocí **GitHub Actions** a výsledná statická aplikace je hostována na **GitHub Pages**, aby byl vývoj co nejvíce ulehčen a aby byla omezena lidská chyba. Tento technologický stack poskytuje solidní základ pro vývoj výukové aplikace, která bude funkční rychlá a uživatelsky přívětivá.

Kapitola 6

Návrh aplikace

Posledním krokem před samotnou implementací výukové aplikace je vytvoření jejího návrhu, který je zdokumentován v této kapitole. Soustředí se prvně na hrubý náčrt uživatelského rozhraní, jednotlivých stránek a vizualizací, které bude aplikace poskytovat. Dále se zaměří na vizuální styl detailněji a stanoví jasné standardy pro použití barev, ikon a dalších grafických prvků. Pozornost bude věnována také architektuře aplikace, detailům návrhu datových modelů a rozmístění funkcionality v rámci projektu. Na závěr budou specifikovány všechny akce, které bude mít uživatel k dispozici, spolu s ovládacími prvky s nimi spjatými a také mechanismy, které zajišťují jejich fungování.

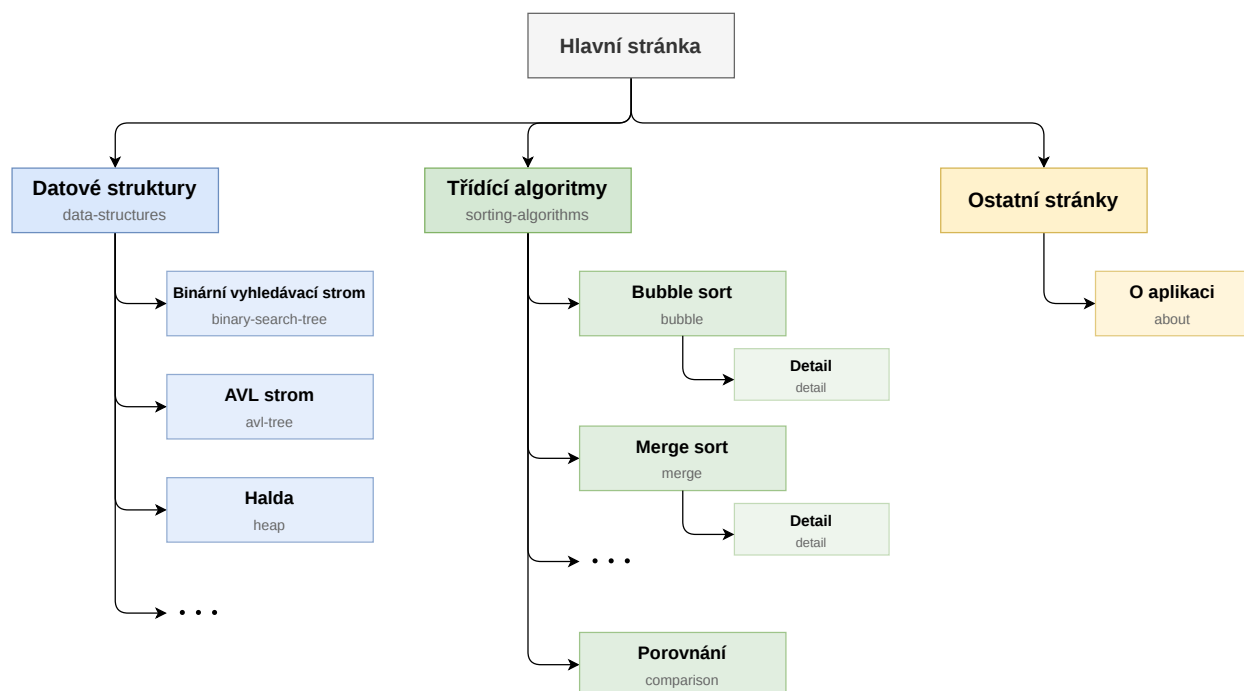
6.1 Informační architektura a navigace

Jako první část, kterou je třeba dobře a logicky navrhnout, je informační architektura aplikace. Vzhledem k tomu, že hlavním účelem vyvíjeného nástroje je podpora výuky algoritmů a datových struktur, byl při návrhu kladen maximální důraz na intuitivnost. Při práci s aplikací musí být student schopen rychle najít požadovanou látku. Proto je v této sekci popsáno nejprve celkové uspořádání obsahu aplikace a dále jsou definovány navigační koncepty pro pohyb mezi stránkami.

6.1.1 Struktura aplikace

Hlavním principem, který byl důležitý pro návrh informační struktury, byla její hloubka. Pro výukové aplikace je zásadní, aby tato informační hloubka byla mělká, protože se uživatel musí být schopen dostat na požadovanou interaktivní vizualizaci v minimálním počtu kroků. Sitemap diagram aplikace (Obrázek 6.1) znázorňuje hierarchii stránek a jejich propojení, přičemž jednotlivé obdélníky představují stránky a jsou rozšířeny i o přesnou cestu, která na danou stránku vede v rámci URL adresy.

Hlavní stránka aplikace slouží jako pouhý rozcestník a obsahuje pouze odkazy na další stránky, směřující uživatele do jedné z dvou hlavních větví: *Datové struktury* a *Třídící algoritmy*. Ve větví



Obrázek 6.1: Sitemap aplikace, znázorňující hierarchii stránek a jejich vzájemné propojení

Datové struktury se stránka dále větví na jednotlivé struktury, přičemž každá z nich má svou dedikovanou stránku, která v sobě integruje veškerou teorii i samotnou simulaci. Podobně uspořádané jsou také stránky pro *Třídící algoritmy*, každý algoritmus má svou stránku s přehledem a obecnou simulací, aby se uživatel mohl seznámit s daným algoritmem z širšího pohledu. Dále má uživatel možnost přejít na stránku *Detail*, kde je algoritmus simulován detailněji, s krokovatelým pseudokódem algoritmu a interaktivní vizualizací paměti. Větev *Třídící algoritmy* na závěr poskytuje také stránku s porovnáním všech algoritmů, aby si jejich chod uživatelé mohli srovnat, který z nich je neefektivnější. Kromě stránek zaměřených čistě na výukový obsah má nástroj také stránku *O aplikaci* s obecnými informacemi o jeho účelu a využití.

6.1.2 Navigační koncept

Návrh se dále zaměřuje na navigační prvky uživatelského rozhraní aplikace. Výchozí layout stránky obsahuje fixní horní lištu, která je dostupná napříč celou aplikací a je tedy zajištěno, že uživatel je vždy schopen jednoduše navigovat, kam potřebuje. Navigační prvky jsou na všech stránkách umístěny stejně, aby byla zajištěna konzistence.

Umístění i styl navigačních prvků se drží známých standardů webového designu, čímž je využita uživatelská znalost těchto principů, takže se může soustředit na samotný výukový obsah stránky. Vzhledem k tomu, že jde o Single Page Application (Kapitola 5.1.2), změna stránky je okamžitá a nedochází k problikávání obrazovky, což přispívá k plynulosti výuky.

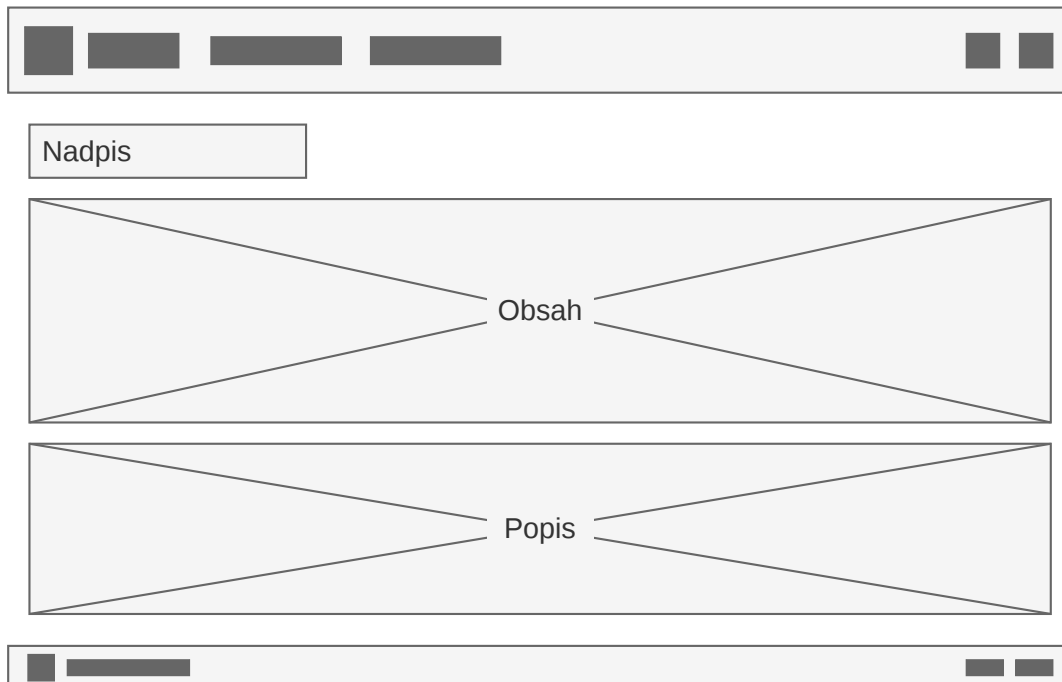
6.2 Uživatelské rozhraní a UX Design

Po definici struktury informací v aplikaci se tato kapitola zaměřuje na samotnou prezentaci obsahu uživateli. Návrh uživatelského rozhraní (UI) a uživatelského zážitku (UX) je pro výukovou aplikaci kritický, neboť je nutné udržet pozornost uživatele u samotné probírané látky, nikoliv u zdlouhavého pochopení ovládání aplikace. Cílem návrhu bylo tedy vytvořit čisté a přehledné prostředí, které zajišťuje interaktivní vizualizace a poskytuje intuitivní ovládací prvky pro ovládání simulací.

6.2.1 Rozvržení uživatelského rozhraní

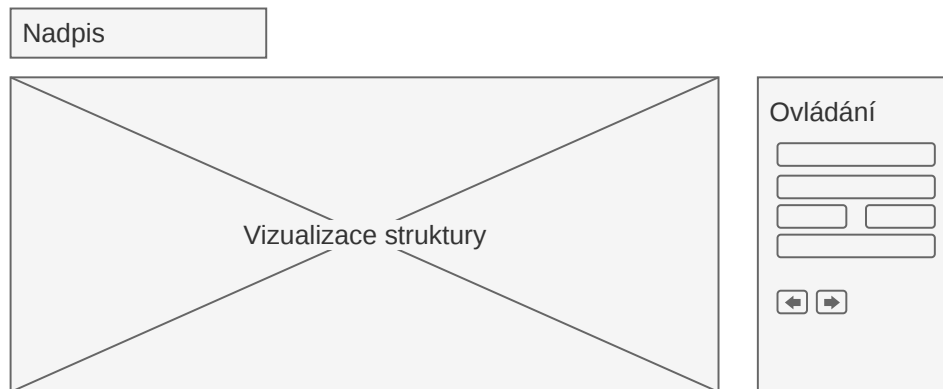
Struktura rozvržení stránek byla navržena s důrazem na jednoduchost a přehlednost. Každá stránka je rozdělena do několika sekcí, které jsou jasně oddělené a logicky uspořádané. Koncepty rozvržení jsou zpracovány ve formě drátěných modelů (*wireframe*), které jsou popsány v následujících odstavcích.

Na prvním diagramu (Obrázek 6.2) je vyobrazen obecný layout webové aplikace. Fixní horní lišta zajišťuje konzistentní možnost navigace mezi stránkami a zároveň dává aplikaci vizuální rámec. Lišta obsahuje ikonu a název aplikace, rozbalovací menu pro zobrazení odkazů jak na datové struktury, tak třídící algoritmy a také tlačítka pro uživatelské přizpůsobení na pravé straně. Podobným způsobem je navržena i spodní lišta, která sice není fixní, ale obsahuje tutéž informaci o názvu aplikace a také doplňující tlačítka na pravé straně.



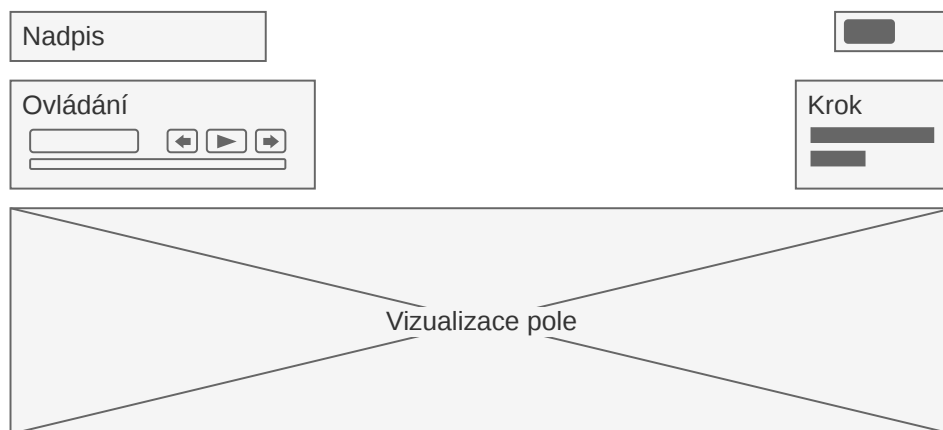
Obrázek 6.2: Rozvržení stránek aplikace

Co se týče konkrétního obsahu, každá vizualizace obsahuje *Nadpis*, tedy název dané struktury nebo algoritmu, a *Popis*, text vysvětlující základní principy a obecnou teorii ohledně tématu. Tento popis je zamýšlen jako doplňující výukový materiál, který studentům pomůže pochopit detaily. *Nadpis* i *popis* jsou stejné pro všechny stránky, podporují tak konzistenci a také možnost znovupoužití za pomoci implementace jako modulární komponenta uživatelského rozhraní. Poslední částí obecného rozvržení je *Obsah*, který, jak název napovídá, obsahuje konkrétní vizualizaci a simulaci, a spolu s nadpisem je konkrétně navržen v následujících odstavcích.



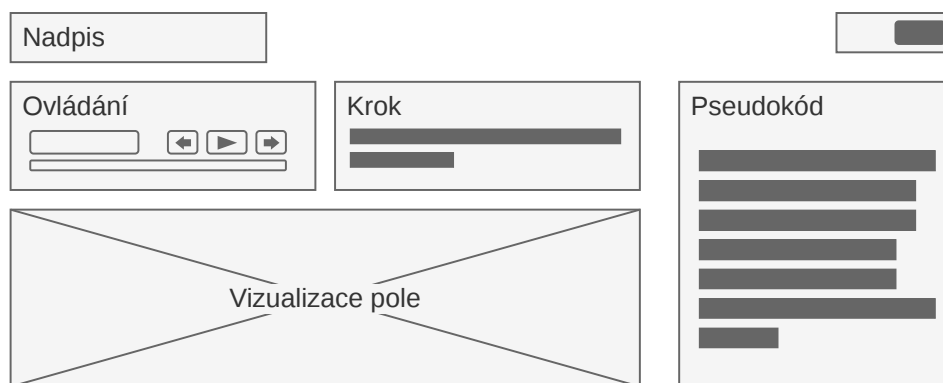
Obrázek 6.3: Návrh vizualizace datové struktury

Neboť se u datových struktur podporovaných výukovou aplikací jedná především o stromy, pro vizualizaci datové struktury je důležité mít dostatečný horizontální prostor, aby měl strom prostor růst do šířky. Z tohoto důvodu zabírá komponenta *Vizualizace struktury* většinu horizontálního prostoru stránky. Na pravé straně obrazovky je umístěno *Ovládání*, které uživatelům zajišťuje pohodlné volání operací nad datovou strukturou a jejich krokování tak, aby bylo vždy při ruce. Celkový návrh pro tuto vizualizaci je znázorněn na Obrázku 6.3.



Obrázek 6.4: Návrh vizualizace třídícího algoritmu

Jak již bylo definováno výše, stránky pro simulaci třídících algoritmů mají dvě úrovně vizualizace, přičemž první z nich je obecná, která zobrazuje pouze pole a jeho prvky, které se mění v průběhu algoritmu. Vpravo od nadpisu je tedy u třídících algoritmů přidán přepínač, kterým si uživatelé mohou zvolit úroveň vizualizace. Návrh (Obrázek 6.4) je velmi jednoduchý, neboť zde není nutné uživateli předávat mnoho informací. *Ovládání* simulace je umístěno pod nadpisem, opět s možností krokování a také posuvníku pro úpravu rychlosti animací. Na pravé straně je pak malý informační panel nadepsaný *Krok*, předávající detailnější informaci o momentálním kroku. Hlavní část, tedy samotná *Vizualizace pole* je roztažena přes celou spodní část stránky, aby byl maximalizován prostor, a tudíž množství prvků v poli.



Obrázek 6.5: Návrh vizualizace detailu třídícího algoritmu

Jako poslední z Wireframe návrhů je přiloženo rozvržení detailního vizuálu třídícího algoritmu (Obrázek 6.5). Pro *Ovládání* i *Krok* jsou využity identické komponenty jako u předchozího vizuálu. Přepínač mezi úrovněmi je nyní přesunut na druhou stranu, uživatel se tak může kdykoli vrátit zpět. *Vizualizace pole* je značně zmenšena, pro detailní popis algoritmu totiž není potřeba velké pole s mnoha prvky, jako tomu bylo u první vizualizace. Důležitým rozšířením je blok *Pseudokód*, kde má uživatel příležitost sledovat řídicí tok algoritmu během krokování, přičemž aktuální řádek kódu bude zvýrazněn barevným pozadím.

6.2.2 Interaktivita a ovládání

Pro příjemné UX a efektivní předání informací uživateli byly kontrolní prvky simulace rozvrženy pomocí Gestalt zákonů, především tedy zákona blízkosti. Tlačítka pro ovládání simulace a krokování jsou tedy u sebe v běžném, uživatelům známém uspořádání. Při interakci tlačítka dávají uživateli zpětnou vazbu změnou odstínu a drobným pohybem. Uživateli je také dána možnost měnit vstup zamícháním pole nebo vygenerováním datové struktury s náhodnými prvky.

Speciálně u datových struktur je také nutné definovat specifické operace, které se nad nimi dají provádět. Téměř všechny stromy tyto operace sdílí, vyvstává zde tedy příležitost generického ovládání. U specifických struktur jako *fronta* nebo *halda* jsou pak jejich operace definovány zvlášť.

Pro vložení, odebrání nebo nalezení prvku má uživatel možnost zadat vstupní hodnotu pomocí číselného vstupního pole.

6.2.3 Responzivita

Výuková aplikace musí podporovat responzivní chování, neboť uživatelé ji nebudou používat pouze na desktopových zařízeních, ale i na přenosných počítačích, tabletech či mobilních zařízeních. Z tohoto důvodu je důležité zmínit responzivitou a definovat, jak se bude aplikace v takových případech chovat.

Pro flexibilní rozložení bude využito přeskupování, jednotlivé stavební bloky uživatelského prostředí se tedy přesunou tak, aby byl prostor maximálně využit. U detailní vizualizace třídícího algoritmu to může znamenat přesun pseudokódu pod vizualizaci pole, aby uživatel stále mohl interagovat s celým obsahem na vertikálním prostoru displeje. Další úprava pro responzivní zobrazení se týká horní navigační lišty, jejíž položky se na menších obrazovkách sbalí do tzv. *hamburger menu* namísto svého původního vzhledu, čímž bude umožněna navigace stránkami i na mobilních zařízeních.

6.3 Návrh vizualizačních komponent

V předchozí kapitole bylo navrženo obecné rozvržení pro všechny stránky, byly však vynechány jedny z nejdůležitějších vizuálních prvků výukové aplikace – samotná vizualizace datových struktur a pole, které se mění v průběhu třídících algoritmů. Tato kapitola tedy poskytuje bližší pohled na tyto komponenty a způsob, jakým budou zobrazeny.

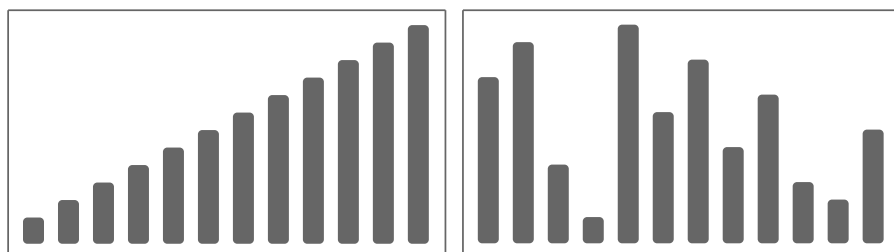
6.3.1 Vizualizace datových struktur

Co se týče datových struktur, základem jejich vizualizace je grafová knihovna, která byla zvolena v předchozí analýze (Kapitola 5.4), tedy *vis-network*. Grafy zobrazené pomocí této knihovny se vykreslují do elementu *canvas* a je u nich zařízeno efektivní vykreslování i pro větší datové sady. Pro vizualizaci stromů je využíván hierarchický layout, který zajišťuje, že uzly jsou uspořádány do vrstev podle své hloubky ve stromu, pro lineární datové struktury se dá stejný princip použít také.

Jelikož u některých datových struktur je třeba zobrazovat doplňující informace (např. informace o vyvážení uzlu AVL stromu nebo identifikace začátku a konce fronty), je u vizualizace nutné implementovat určitý způsob anotace grafu. Anotace se využívá nejen pro zmíněné informace, ale také pro komentář k aktuální operaci a kroku, uživatel tak bude mít další pomocnou vysvětlivku směřující jeho pozornost k důležitým uzlům grafu. Tyto popisky jsou vykresleny taktéž do elementu *canvas*, využívá API knihovny pro zjištění konkrétních souřadnic jednotlivých uzlů grafu. Samotný návrh vizuálu tedy není přiložen, jedná se totiž o základní vykreslení grafu knihovnou *vis-network* zobrazeným výše na Obrázku 5.5, rozšířené o informační popisky.

6.3.2 Obecná vizualizace pole

První ze dvou úrovní vizualizace třídících algoritmů je ta obecná, která zobrazuje pouze pole ve velmi zjednodušené formě, bez zbytečných detailů. Slouží primárně k předvedení, jak algoritmus s polem pracuje ze širšího pohledu, uživatele zde nezajímají konkrétní hodnoty, ale spíše tvar dat a jejich změna v každé iteraci. Proto je tato vizualizace, zobrazená na Obrázku 6.6, složená pouze z jednoduchých obdélníků, které reprezentují jednotlivé hodnoty na daném indexu. Na levé straně je znázorněno setříděné pole, na pravé pak zamíchané, kde je díky různým velikostem prvků jasné vidět, v jakém pořadí by měly ve finální iteraci být.

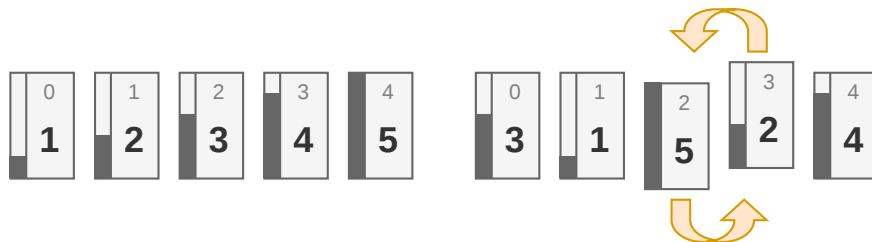


Obrázek 6.6: Návrh obecné vizualizace pole

Jednotlivé prvky pole jsou realizovány jako HTML elementy, jejichž výška odpovídá hodnotě daného prvku, což lze jednoduše realizovat pomocí CSS. Jsou jednobarevné, jejich obarvení je tedy uživatelem lehce zaznamennatelné a umožní názorné zvýraznění momentálně aktivních prvků. Protože se jedná o HTML elementy, je také možné využít CSS animace pro plynulé přechody mezi iteracemi a při přehození prvků.

6.3.3 Detailní vizualizace pole

Pro detailní vizualizaci pole, která je zaměřena na předání hlubšího porozumění algoritmu, je potřeba přidat další informace a vizuální prvky, které pomohou uživateli pochopit, co se v daném kroku děje. Proto je vizualizace na Obrázku 6.7 složená z menšího počtu bloků stejné velikosti namísto sloupců, aby bylo možné lépe využít prostor.



Obrázek 6.7: Návrh detailní vizualizace pole

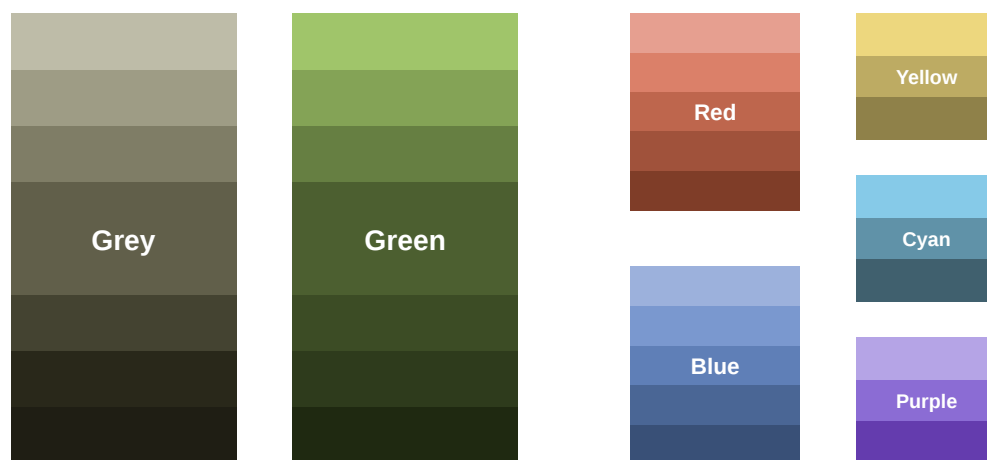
Myšlenka obdélníků s velikostí odpovídající hodnotě prvků zůstává, ale stává se spíše doplňujícím elementem, přidaným na levou stranu každého prvku pole. Tyto prvky jsou však reprezentovány jako bloky stejné výšky, které obsahují informaci jak o indexu v poli, tak hodnotě daného prvku. Na pravé straně se opět nachází nesetříděné pole. Je zde také naznačena animace probíhající při prohazování prvků – vizuální přesun po křivce (tzv. oblouková trajektorie). Tento plynulý přesun dodává celému procesu na názornosti, protože uživatel neztrácí kontext o tom, kam se prvek přemístil. I tato data jsou reprezentována jako HTML elementy, je tedy využita možnost stylů a animací poskytovaných CSS.

6.4 Vizuální styl a sémantika barev

Vizuální styl výukové aplikace neslouží jen k estetickému dojmu, ale je sám o sobě jedním z důležitých nosičů informací. Správné použití barev a typografie snižuje kognitivní zátěž studenta a pomáhá mu lépe sledovat průběh algoritmu.

6.4.1 Barevná paleta

Výběr barevné palety pro uživatelské rozhraní se opírá o základní principy teorie barev a zaměřuje se na podporu pochopení principů učiva. Vzorník barev je přiložen na Obrázku 6.8. Zvolené barvy jsou převážně přirozené a méně satureované, aby u uživatelů nedocházelo k únavě očí při delším používání aplikace.



Obrázek 6.8: Barevná paleta výukové aplikace

Primární barvou uživatelského rozhraní je neutrální šedá, která tvoří čisté a nerušivé pozadí. Sekundární a akcentní barvou je zelená, jež je využívána pro interaktivní prvky, nadpisy a obecně pro dodání webové stránky alespoň nějaké zajímavé barvy. Zbýlé barvy z palety jsou vyhrazeny striktně

pro sémantické obarvování interaktivních vizualizací, aby uživatel snadno rozpoznal jednotlivé stavy algoritmů.

6.4.2 Sémantické barvy vizualizací

Pro správné pochopení datových struktur a algoritmů hraje barva důležitou roli, neboť slouží jako hlavní nositel informace o aktuálním stavu vizualizovaného prvku. Barevná sémantika je rozdělena do dvou hlavních kategorií podle typu vizualizace.

U **datových struktur** je výchozí barvou uzlů zelená. Během průchodu strukturou mění uzly svou barvu podle prováděné operace: azurová barva indikuje hledaný prvek, zatímco modrá označuje nalezený uzel nebo potomka. Pokud je uzel označen ke smazání, je zvýrazněn varovnou červenou barvou. Specifický přístup vyžadují červeno-černé stromy, kde samotná podstata struktury závisí na barvě uzlu. V tomto případě jsou uzly obarveny tmavě šedou (reprezentující černé uzly) a tlumenou červenou barvou. Aby nedocházelo ke konfliktu s indikací mazání, je odstranění uzlu v těchto stromech řešeno pomocí výplně odlišného odstínu. Mezi další specifické sémantické barvy patří žlutá, která indikuje informace o vyvážení uzlu AVL stromu, a modrá, sloužící ke zvýraznění porovnávaných prvků u haldy.

U **třídících algoritmů** je výchozím stavem neutrální šedá barva. Pozornost uživatele je při krokování směřována pomocí červené barvy, jež značí aktuálně porovnávanou dvojici prvků. Pokud dojde k jejich přesunu či prohození, barva se mění na modrou. Jakmile prvek dosáhne své finální pozice a je prokazatelně setříděn, získá zelenou barvu. To uživateli vizuálně usnadňuje oddělit již vyřešenou část pole od stále probíhajícího algoritmu. V případě složitějších algoritmů, jako je Quick Sort, je využita žlutá barva pro jasnou identifikaci pivotu. Při rozdělování pole (Quick Sort a Merge Sort) se pak kombinuje fialová a tyrkysová barva pro vizuální oddělení levé a pravé části.

6.4.3 Typografie

Čitelnost textu je pro výukové materiály naprosto stěžejní, proto byl při návrhu kladen velký důraz na volbu vhodných řezů písma. Pro běžný text uživatelského rozhraní byla zvolena rodina bezpatkových (sans-serif) systémových písem (např. systémový font daného operačního systému, jako je San Francisco na macOS či Segoe UI na Windows). Toto řešení zaručuje, že aplikace působí přirozeně a moderně na jakémkoliv zařízení, přičemž text zůstává perfektně čitelný i při menších velikostech.

Druhá typografická volba se týká výpisů pseudokódu v detailních vizualizacích. Zde je použita rodina neproporcionálních písem (monospace), u kterých má každý znak stejnou šířku. Tento typ písma je nezbytný pro správné vertikální zarovnání kódu, zachování odsazení a především pro jednoznačnou odlišitelnost vizuálně podobných znaků, což je pro pochopení algoritmů nezbytné.

6.5 Konceptuální model řízení simulace

Základním stavebním kamenem každé interaktivní výukové aplikace je způsob, jakým systém pracuje se stavem simulace v reálném čase a jakým způsobem umožňuje uživateli tento stav ovládat. Uživatel musí mít možnost algoritmus pozastavit, krokovat nejen vpřed ale i vzad, aby mohl lépe pochopit, co se v daném kroku děje. Z tohoto důvodu bylo nutné již ve fázi návrhu opustit myšlenku naivního přístupu, kdy by se algoritmus vykonával v reálném čase a uživatel by pouze pozoroval jeho průběh, a místo toho navrhnout komplexní konceptuální model, který umožní plnou kontrolu nad simulací.

6.5.1 Předpočítaná historie stavů

Pro simulaci obou typů vizualizací, tedy datových struktur i třídících algoritmů, byl zvolen architektonický vzor založený na předpočítání historie stavů (*Pre-computed State Queue*). Tento koncept spočívá v tom, že při inicializaci simulace s danými vstupními daty se celý proces okamžitě provede na pozadí. Během tohoto běhu nedochází k přímým úpravám uživatelského rozhraní, ale při každé významné atomické operaci se vygeneruje snímek aktuálního stavu (*snapshot*).

Všechny tyto snímky jsou následně uloženy do sekvenčního pole. Samotná vizualizace pak funguje na principu přehrávače, přičemž řídicí logika si pamatuje pouze aktuální index v tomto poli. Při požadavku na další krok se index inkrementuje a uživatelské rozhraní se překreslí podle dat v novém snímku. Tento přístup přináší zásadní výhody:

- **Okamžitý krok zpět:** Návrat v čase nevyžaduje implementaci složité reverzní logiky, stačí pouze snížit index v poli historie.
- **Oddělení logiky a zobrazení:** Samotný algoritmus nemusí řešit časování ani synchronizaci s CSS animacemi.
- **Předvídatelnost:** Protože je celý průběh znám dopředu, je možné uživateli zobrazit celkový počet kroků potřebných k seřazení pole nebo umožnit okamžité přeskočení na konec algoritmu.

6.5.2 Mechanismy krokování

Zatímco krokování u třídících algoritmů je striktně lineární, datové struktury vyžadují komplexnější přístup. Operace nad nimi totiž mají přirozeně hierarchický charakter – požadavek na vložení prvku do stromu je z pohledu uživatele jedinou akcí, ale z pohledu algoritmu se jedná o sekvenci kroků, které musí být provedeny v určitém pořadí.

Z tohoto důvodu byl koncept krokování u datových struktur navržen dvouúrovňově:

1. **Úroveň operací (Makroúroveň):** Reprezentuje celistvé požadavky uživatele (např. Vlož, Smaž, Najdi). Uživatel má možnost nechat celou operaci plynule proběhnout bez nutnosti manuálního zasahování.

2. **Úroveň kroků (Mikroúroveň):** Reprezentuje jednotlivé atomické úkony uvnitř dané operace. Pokud chce uživatel pochopit detailní fungování struktury, může si operaci pozastavit a procházet ji krok za krokem, přičemž systém mu vizualizuje každý posun ukazatele či změnu v grafu.

Tento model zajišťuje, že aplikace je vhodná jak pro rychlé demonstrace vlastností datových struktur, tak pro detailní studium jejich vnitřních mechanismů.

6.5.3 Mapování logiky na pseudokód

Pro efektivní výuku je nezbytné, aby uživatel viděl přímou souvislost mezi vizuální reprezentací algoritmu a jeho teoretickým zápisem. Za tímto účelem byl navržen systém mapování kroků pomocí abstraktních identifikátorů, interně nazvaných jako části kódu (*codeParts*).

Při generování transakční historie si každý krok třídícího algoritmu nenese informaci o konkrétním textu, ale obsahuje pouze identifikátory těch logických bloků, které se v danou chvíli vykonávají. Uživatelské rozhraní následně tyto identifikátory dynamicky spáruje s jedním či více řádky aktuálně zobrazeného pseudokódu a vizuálně je zvýrazní.

Zásadní výhodou tohoto oddělení aplikační logiky od prezentační vrstvy je značná flexibilita. Systém tak umožňuje snadné přepínání pseudokódu do různých programovacích jazyků za běhu aplikace, přičemž synchronizace mezi běžící animací a zvýrazněnými řádky kódu zůstává plně a bezchybně zachována. Tento způsob řešení je díky možnosti mapovat jeden krok na více řádků pseudokódu také ideální pro algoritmy, které mají složitější vnitřní strukturu. Simulace tak zůstává přehledná a uživatel má vždy jasnou představu o tom, co se v daném kroku děje, a jaké části algoritmu jsou právě aktivní.

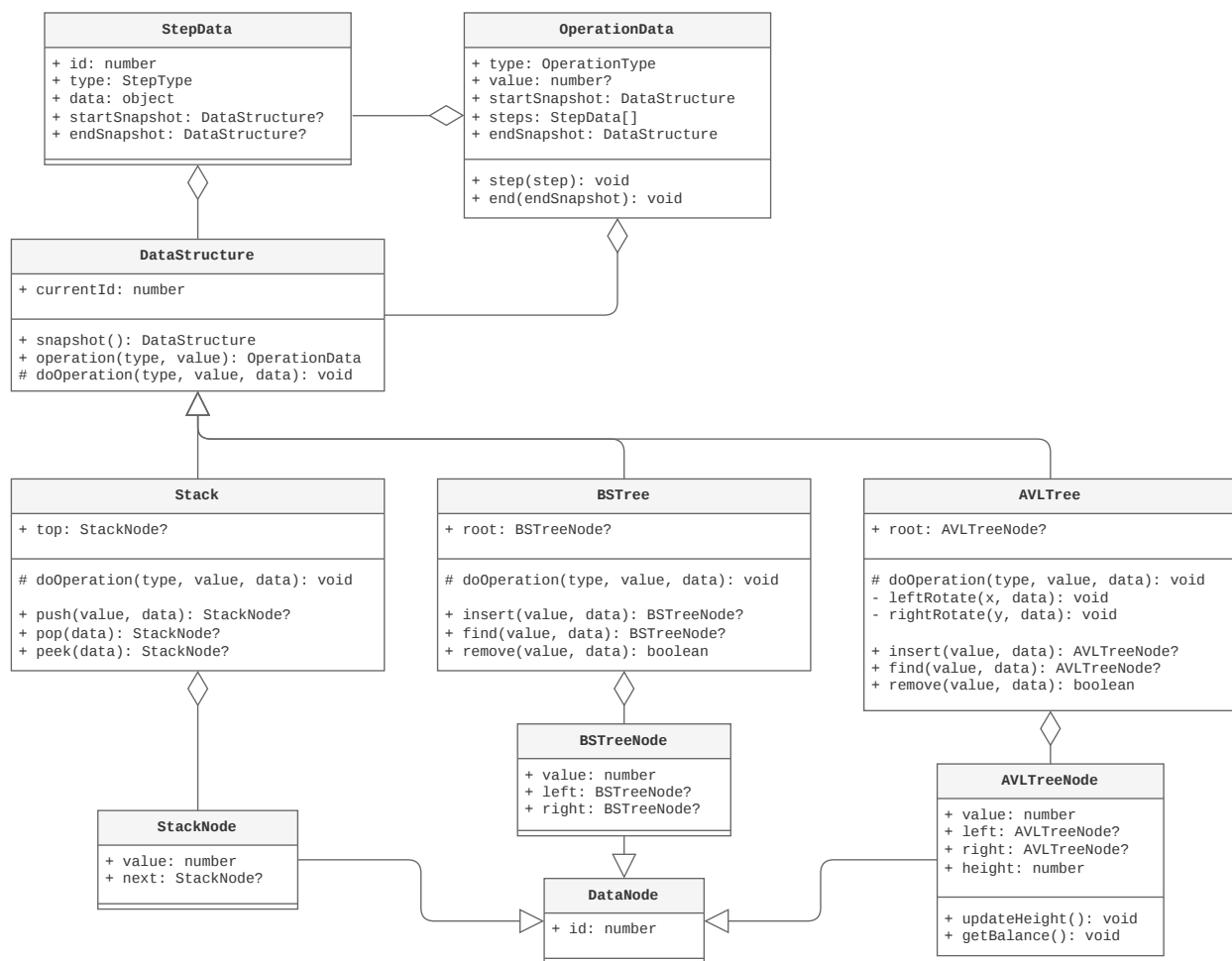
6.6 Datový model a abstrakce

Pro úspěšnou implementaci a plynulou vizualizaci je nezbytné striktně oddělit aplikační logiku algoritmů od vykreslovací vrstvy. Pouhá reprezentace dat pomocí surových čísel je pro interaktivní aplikaci nedostačující. Datové modely proto musí obsahovat i dodatečná metadata potřebná pro sledování animací, identifikaci prvků a jejich sémantické obarvování.

6.6.1 Reprezentace datových struktur

Abstraktní datový model struktur je navržen s ohledem na maximální univerzálnost a snadnou rozšiřitelnost. Jeho zjednodušený konceptuální návrh je znázorněn na Obrázku 6.9.

Na nejnižší úrovni je třída `DataNode` reprezentující každý prvek grafu identifikován unikátním `id`, pomocí něhož je možné datovou strukturu spárovat s vykreslovací knihovnou *vis-network*. Z tohoto základu dědí specifické třídy uzlů (např. pro binární stromy či zásobník), které jej obohacují



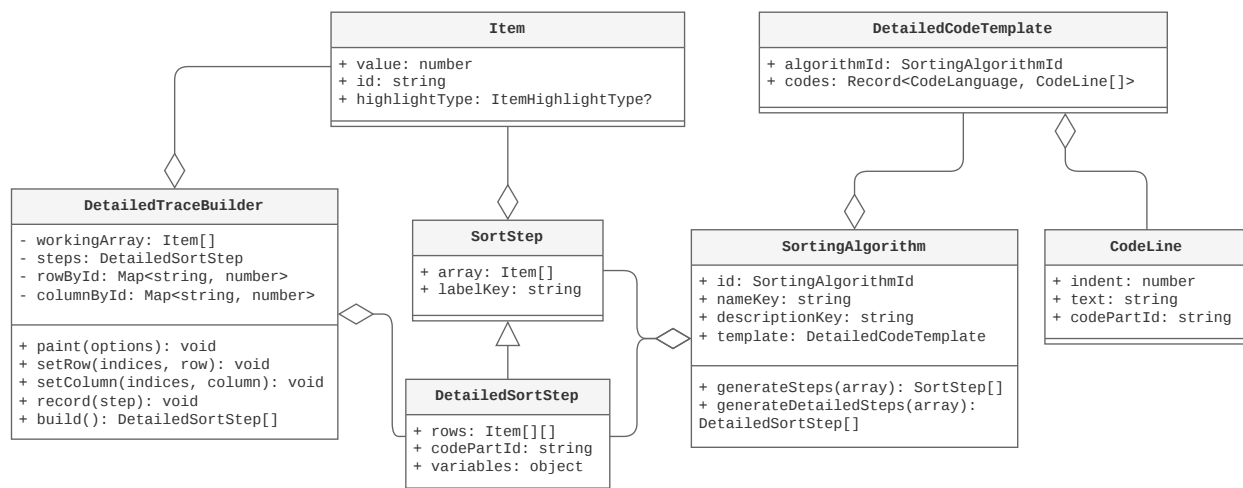
Obrázek 6.9: Třídy pro reprezentaci datových struktur

o samotnou hodnotu, potřebné ukazatele na další uzly a případně specifické metody, například pro aktualizaci informací o vyváženosti u AVL stromů.

Společné rozhraní všech struktur zastřešuje abstraktní třída **DataStructure**. Ta automatizuje přidělování identifikátorů a definuje metodu pro tvorbu neměnných snímků stavu grafu (**snapshot**). Provádění samotných operací je abstrakcí rozděleno do dvou úrovní, které odrážejí hierarchický návrh kroků: **OperationData** obaluje celou operaci (včetně stavu před a po), zatímco **StepData** slouží pro uložení dílčích atomických kroků uvnitř operace. Každý krok v sobě navíc může nést specifická data určující, které uzly se mají v danou chvíli zvýraznit. Díky této jednotné architektuře dokáže vizuální vrstva vykreslit jakoukoliv strukturu bez nutnosti znát její vnitřní implementační detaily.

6.6.2 Rozhraní pro třídící algoritmy

Rovněž návrh třídících algoritmů striktně dodržuje modulární přístup, aby bylo možné algoritmy snadno testovat a rozšiřovat. Třídní diagram této části je přiložen na Obrázku 6.10.



Obrázek 6.10: Třídy pro reprezentaci třídících algoritmů

Nejnižší logickou jednotkou je třída **Item**, která obaluje číselnou hodnotu prvku. Obsahuje unikátní **id** pro plynulé sledování DOM elementů během CSS animací a atribut **highlightType**, jenž řídí sémantické obarvení prvku.

Záznam výpočetní historie algoritmu deleguje systém na třídu **DetailedTraceBuilder**. Ta si udržuje aktuální stav pole (**workingArray**) a postupně do paměti ukládá jednotlivé snímky pomocí metody **record**. Výsledkem je vygenerované pole kroků. Z důvodu podpory dvou úrovní vizualizace existují i dva typy kroků: **SortStep** nese pouze základní informace o stavu pole, zatímco **DetailedSortStep** je rozšířen o dvourozměrnou mřížku pro vizualizaci úrovní u rekurzivních algoritmů (Quick Sort, Merge Sort) a o mapovací identifikátor **codePartId** napojující aktuální část pseudokódu.

Definice samotného pseudokódu je uložena v šabloně **DetailedCodeTemplate**, která pro daný algoritmus specifikuje řádky (**CodeLine**) nezávisle na zvoleném programovacím jazyce. Hlavní třída **SortingAlgorithm** tak funguje pouze jako zastřešující entita – přijme vstupní pole a pomocí definovaných metod (**generateSteps**) vrátí standardizovanou sekvenci těchto kroků, jež si uživatelské rozhraní plynule přehraje.

Kapitola 7

Implementace

V této kapitole je podrobně popsán proces realizace výukové aplikace a technické řešení jejích hlavních komponent. Text navazuje na teoretický návrh představený v předchozí části a zaměřuje se na praktickou aplikaci zvolených technologií a návrhových vzorů.

7.1 Celková architektura a modularita systému

Aplikace je postavena na moderním webovém frameworku SvelteKit s využitím silně typovaného jazyka TypeScript. Návrh softwarové architektury nevyužívá tradiční historické vzory (jako je například striktní MVC), které jsou pro reaktivní frontendové aplikace často svazující. Namísto toho architektura kombinuje principy *Domain-Driven Design* (DDD) s komponentovým přístupem a reaktivním řízením stavu. Tento přístup umožňuje striktní oddělení prezentační vrstvy od aplikační logiky, a to při zachování vysoké plynulosti a responzivity.

Základním pilířem architektury je rozdělení kódu na sdílenou infrastrukturu a dvě na sobě nezávislé logické domény: *Datové struktury* a *Třídící algoritmy*. Toto oddělení bylo zvoleno, jelikož obě domény využívají odlišné renderovací mechanismy (HTML/CSS animace pro algoritmy a Canvas API přes *vis-network* pro struktury) a vyžadují odlišný přístup k řízení simulace.

7.1.1 Struktura adresářů a sdílený kód

Aby bylo zabráněno duplicitě kódu a byl dodržen princip DRY (*Don't Repeat Yourself*), jsou obecné nástroje izolovány od doménově specifické logiky. Celková organizace zdrojových kódů v adresáři `src/lib` a statických souborů je znázorněna na Výpisu 9.

Sdílená infrastruktura zahrnuje adresáře `components` (obsahující globální layout či standardizovaná tlačítka), `utils` a `i18n`. Systém internacionalizace aktuálně poskytuje plnou podporu pro český a anglický jazyk, přičemž architektura umožňuje triviální přidání dalších jazykových mutací.

Důležitým architektonickým prvkem je adresář `server/`, který obstarává načítání a zpracování statických Markdown souborů z adresáře `static/descriptions/`. Tyto soubory obsahují detailní

```

1  src/lib/
2  |-- components/           # Sdílené UI prvky (tlačítka, navigace, globální layout)
3  |-- i18n/                 # Internacionalizace
4  |-- utils/                # Sdílené pomocné nástroje a abstrakce (např. časovač)
5  |-- server/               # Logika pro načítání souborů s popisy během renderingu
6  |
7  |-- data-structures/      # Doména datových struktur
8  |   |-- structures/       # Datové reprezentace (AVL, BST, B-strom...)
9  |   |-- visual/           # Vykreslovací enginy (Animator, Annotator) a vizualizace
10 |   |-- operations/        # Událostní model simulace (OperationManager)
11 |   |-- controls/          # UI komponenty pro ovládání simulace (tlačítka, posuvníky)
12 |   |-- registry.ts        # Registrace struktur pro dynamické načítání
13 |
14 |-- sorting-algorithms/    # Doména třídících algoritmů
15 |   |-- algorithms/        # Konkrétní implementace algoritmů (Merge, Quick...)
16 |   |-- visual/            # Komponenty pro vykreslování stránek vizualizací
17 |   |-- components/        # UI komponenty specifické pro vizualizaci polí
18 |   |-- steps/             # Logika řízení času (StepManager, TraceBuilder)
19 |   |-- registry.ts        # Registrace algoritmů pro dynamické načítání
20 |
21 static/descriptions/      # Zdrojové soubory s výukovými texty
22 |-- about.cs/en.md
23 |-- data-structures/
24 |-- sorting-algorithms/

```

Výpis 9: Struktura adresářů a souborů implementace

výukové texty pro jednotlivé algoritmy. Aplikace využívá výhod Server-Side Renderingu (SSR), kdy se tyto texty načtou, spárují s odpovídající jazykovou lokalizací a vyrenderují do HTML ještě předtím, než se stránka odešle klientovi. Tento mechanismus nejen zrychluje úvodní načtení aplikace, ale také výrazně zlepšuje SEO (optimalizaci pro vyhledávače). Detailní technické řešení tohoto načítání je popsáno v Kapitole 7.7.3.

Samotné logické domény (`data-structures` a `sorting-algorithms`) jsou následně zcela izolované. Každá obsahuje svůj vlastní `registry.ts`, který slouží jako centrální bod pro dynamické načítání podporovaných algoritmů a struktur do uživatelského rozhraní.

7.2 Implementační jádro třídících algoritmů

Jádro domény třídících algoritmů je postaveno na maximální znovupoužitelnosti kódu a oddělení správy stavu od vizualizační vrstvy. Uživatelské rozhraní pro oba typy vizualizací (obecný přehled i detailní simulace) sdílí jednotnou komponentu ovládacího panelu (*Controls*). Tato komponenta zpřístupňuje instanci řídicí třídy, skrze kterou uživatel ovládá přehrávání, krokování, rychlost animace a výběr vstupní datové sady (např. náhodné, téměř setříděné či reverzní pole).

Díky implementaci návrhového vzoru Registry (`registry.ts`) jsou všechny algoritmy a datové sady dynamicky evidovány. Tento přístup mimo jiné umožňuje automatické generování komparač-

ních matic – při přidání nového algoritmu nebo datové sady není nutné modifikovat kód srovnávacích pohledů, systém novou kombinací automaticky detekuje a vyrenderuje.

7.2.1 Stavový automat a správa životního cyklu

Samotný běh animace a správu času zapouzdřuje třída `StepManager`. Funguje jako centrální stavový automat, který využívá reaktivní primitiva frameworku Svelte (`$state`) k udržování informací o aktuálním indexu přehrávaného kroku (`currentStepIndex`), stavu přehrávání (`isPlaying`) a rychlosti (`delayMs`).

Změna indexu uvnitř třídy `StepManager` automaticky vyvolá překreslení závislých komponent, aniž by bylo nutné explicitně volat aktualizací metody. Plynulý běh simulace zajišťuje delegovaný časovač (z `utils`), který v nastaveném intervalu inkrementuje index kroku. Zjednodušená ukázka tohoto mechanismu, zahrnující metody pro spuštění, pozastavení a cyklické posouvání (`tickStep`), je uvedena ve Výpisu 10.

```
1 export class StepManager<T> {
2   steps: T[];
3   currentStepIndex: number;
4   isPlaying: boolean;
5   delayMs: number;
6   private timer: Timer;
7
8   play() {
9     if (this.currentStepIndex >= this.steps.length - 1) {
10      this.currentStepIndex = 0; // Restart, pokud je algoritmus již dokončen
11    }
12    this.isPlaying = true;
13    this.timer.start(this.delayMs, this.tickStep.bind(this));
14  }
15
16  pause() {
17    this.isPlaying = false;
18    this.timer.stop();
19  }
20
21  private tickStep() {
22    if (this.currentStepIndex < this.steps.length - 1) {
23      this.currentStepIndex += 1;
24    } else {
25      this.pause(); // Bezpečně ukončení simulace
26    }
27  }
28 }
```

Výpis 10: Zjednodušená implementace časovače a stavů ve třídě `StepManager`

7.2.2 Generování kroků a optimalizace pohledů

Jak bylo teoreticky navrženo v Kapitole 6.6.2, algoritmy svůj běh neanimují přímo, ale prostřednictvím třídy `TraceBuilder` generují sekvenci neměnných stavů. Záznam každého kroku probíhá ve dvou fázích. Nejprve algoritmus zavolá metodu `paint`, která dotčeným prvkům pole vnitřně přiřadí sémantické stavy (např. porovnávání, přesun) pomocí rozhraní, které je přirozenější pro samotnou implementaci algoritmů. Následně je zavolána metoda `record`, která provede hlubokou kopii (deep copy) celého pole. Tento postup, znázorněný ve Výpisu 11, zachovává neměnnost (immutability) dat a zajišťuje správné fungování UI reaktivity. V metodě `record` se pod povrchem rovněž dynamicky vypočítávají a ukládají pozice prvků v matici rekurzivních úrovní (`rows`), jež jsou důležité pro vizualizaci algoritmů typu Merge Sort. Plné znění této metody však bylo z důvodu přehlednosti zkráceno.

```
1 export class DetailedTraceBuilder {
2   private readonly array: Item[];
3   private readonly steps: DetailedSortStep[] = [];
4
5   paint(options: TracePaintOptions): void {
6     if (options.compared) {
7       options.compared.forEach(
8         (i) => (this.array[i].highlightType = ItemHighlightType.Compare),
9       );
10    }
11    // ...
12  }
13
14  record(step: Omit<DetailedSortStep, "array">): void {
15    const snapshotArray = deepCopy(this.array);
16
17    this.steps.push({
18      array: snapshotArray,
19      codePartId: step.codePartId,
20      stepLabel: step.stepLabel,
21      // ...
22    });
23  }
24
25  build(): DetailedSortStep[] {
26    return this.steps;
27  }
28 }
```

Výpis 11: Implementace metod `paint` a `record` ve třídě `DetailedTraceBuilder`

Zásadním implementačním rozhodnutím pro dodržení principu DRY (*Don't Repeat Yourself*) je způsob generování dvou úrovní vizualizace. Standardní třídicí algoritmy explicitně implementují pouze metodu pro tvorbu detailních kroků (`generateDetailedSteps`). Metoda pro obecný pohled

(`generateSteps`) si tyto detailní kroky vyžádá a programově z nich odfiltruje veškerou vizuální logiku kromě operací typu *Porovnání* a *Prohození*. Výjimku tvoří složité rekurzivní algoritmy, které implementují obě metody odděleně, neboť jejich komplexní práce s rekurzivními podúrovněmi nelze algoritmicky zjednodušit.

7.2.3 Dynamické mapování pseudokódu

Jednou z dalších funkcí simulátoru je podpora dynamického přepínání programovacích jazyků pseudokódu v reálném čase. Je důležité zdůraznit, že tento systém je zcela izolován od globální lokalizace aplikace, jež se stará pouze o překlad samotného textu uživatelského rozhraní.

Mechanismus je implementován pomocí abstrakce `DetailedCodeTemplate`. Při generování historie neukládá `TraceBuilder` statický text kódu, ale pouze logický identifikátor `codePartId`. Uživatelské rozhraní následně reaguje na reaktivní změnu zvoleného jazyka tím, že pro něj vyhledá řádky se shodným identifikátorem a vizuálně je zvýrazní. Protože je volba jazyka nezávislá na stavové proměnné `currentStepIndex`, může uživatel libovolně měnit jazyk i uprostřed běžící simulace. Kód se okamžitě překreslí do syntaxe vybraného jazyka, přičemž algoritmus vizuálně zůstává přesně v tom kroku, ve kterém byl pozastaven.

7.3 Implementační jádro datových struktur

Na rozdíl od třídících algoritmů, které operují nad předem vygenerovaným fixním polem kroků, je doména datových struktur výrazně dynamičtější. Uživatel s těmito strukturami interaktivně manipuluje (vkládá, hledá a maže uzly), což vyžaduje komplexnější architekturu schopnou reagovat na asynchronní události. Z tohoto důvodu byla logika datových struktur striktně oddělena od vizuální vrstvy pomocí událostmi řízeného návrhu (Event-Driven Architecture) a systému specializovaných delegátů.

7.3.1 Událostmi řízený model simulace

Centrálním mozkiem pro řízení stavu datových struktur je třída `OperationManager`. Ta funguje na principu návrhového vzoru Publisher-Subscriber. Místo toho, aby logické modely přímo volaly funkce pro překreslení uživatelského rozhraní, emitují při jakékoliv změně specifické události definované výčtovým typem `EventType` (např. začátek nové operace, změna aktuálního kroku). Komponenty uživatelského rozhraní (stejně jako vizualizační engine) tyto události pouze poslouchají a reaktivně se jim přizpůsobují.

Tento manažer zároveň obstarává logiku hierarchického dvouúrovňového krokování, které bylo navrženo v Kapitole 6.6.1. Jak demonstruje zjednodušený Výpis 12, manažer na základě uživatelského nastavení (`showSteps`) dynamicky rozhoduje, zda posun vpřed (`next`) znamená pouhý

přechod na další atomický krok v rámci právě probíhající operace, nebo zda má přeskočit rovnou na začátek operace následující.

```
1 export class OperationManager {
2     // ... stavové proměnné (currentOperation, currentStep, operations) ...
3
4     public next(): void {
5         if (!this.canDoNext()) return;
6
7         if (this.showSteps) {
8             // Mikro-úroveň: Posun po jednotlivých atomech operace
9             if (
10                 this.currentStep <
11                 this.operations[this.currentOperation].steps.length - 1
12             ) {
13                 this.incrementCurrentStep(); // Interně emituje CurrentStepChanged
14             } else if (this.currentOperation < this.operations.length - 1) {
15                 this.incrementCurrentOperation(); // Interně emituje CurrentOperationChanged
16             }
17         } else {
18             // Makro-úroveň: Plynulý posun po celých operacích
19             if (this.currentOperation < this.operations.length - 1) {
20                 this.incrementCurrentOperation();
21             }
22         }
23     }
24 }
```

Výpis 12: Řízení dvouúrovňového krokování a emitování událostí ve třídě OperationManager

7.3.2 Zpracování kroků a systém delegace

Při emisi události o změně kroku je nutné abstraktní datový záznam (např. „provedena rotace vlevo“) přeložit na konkrétní vizuální povely pro grafickou knihovnu. K tomuto překladu slouží architektonický vzor *Strategy*, implementovaný prostřednictvím systému handlerů.

Základem je abstraktní třída `StepHandlerBase`, jejíž rozhraní je uvedeno ve Výpisu 13. Každá datová struktura (např. AVL strom) implementuje vlastního potomka této třídy. Životní cyklus zpracování jednoho kroku je zde rozdělen do tří fází: `stepSetup` (příprava plátna a aplikace datového snapshotu), `stepRoute` (samotné spuštění animace) a `stepCleanup` (zajištění konzistentního stavu po dojetí animace).

Zásadní roli hraje metoda `stepRoute`, která slouží jako směrovač (dispatcher). Na základě typu kroku (`StepType`) asynchronně deleguje vykonání vizuální logiky na úzce specializované privátní metody (např. pro animaci rotace, přesun uzlu či zvýraznění potomků). Tímto přístupem je zajištěno, že samotný vizualizační engine zůstává univerzální a veškerá specifika chování konkrétní datové struktury jsou bezpečně zapouzdřena v odpovídající `Handler` třídě.

```

1  export abstract class StepHandlerBase {
2      abstract stepSetup(
3          currentStep: StepData,
4          animator: DataStructureAnimator,
5          annotator: DataStructureAnnotator,
6          isForward: boolean,
7      ): Promise<void> | void;
8
9      abstract stepCleanup(
10         currentStep: StepData,
11         animator: DataStructureAnimator,
12         annotator: DataStructureAnnotator,
13         isForward: boolean,
14     ): Promise<void> | void;
15
16     abstract stepRoute(
17         currentStep: StepData,
18         animator: DataStructureAnimator,
19         annotator: DataStructureAnnotator,
20         operationManager: OperationManager,
21         isForward?: boolean,
22     ): Promise<void> | void;
23 }

```

Výpis 13: Abstraktní rozhraní StepHandlerBase definující životní cyklus vizualizace kroku

7.4 Vizualizační engine a řešení technologických omezení

Posledním stěžejním prvkem implementační architektury je vizualizační engine, který má za úkol plynule převádět diskrétní datové modely algoritmů a struktur na srozumitelné animace. Při návrhu této vrstvy bylo nutné vyřešit několik netriviálních technologických výzev, od reprezentace rekurze na frontendové mřížce až po obcházení limitů externích grafických knihoven.

7.4.1 Vizuální reprezentace rekurzivních algoritmů

Standardní třídící algoritmy operují lineárně nad jedním polem, které v uživatelském rozhraní představuje jeden statický řádek prvků. Komplexnější rekurzivní algoritmy, jako jsou Merge Sort a Quick Sort, však logicky pole štěpí a pracují v oddělených podúrovních. Pro jejich pochopení je zásadní, aby student viděl vizuální ekvivalent rekurze.

Z tohoto důvodu je pole v těchto případech rozprostřeno do dvourozměrné mřížky využívající technologii CSS Grid. Při zanoření do rekurze se zpracovávané prvky fyzicky přesouvají do nových vertikálních úrovní (řádků), přičemž přesun je plynulý pomocí CSS animací. U algoritmu Merge Sort má každá úroveň svůj vyhrazený plnohodnotný řádek. Naproti tomu u algoritmu Quick Sort je využito prostorové optimalizace – protože nad sebou nikdy neoperují dva prvky současně, jednotlivé řádky jsou vysoké pouze jako polovina výšky prvku.

Důležitým navigačním prvkem je obarvení pozadí prvků (tzv. *bounding boxes*), které vizuálně ohraničuje tu část pole, ve které se aktuálně rekurze provádí. Toto ohraničení je implementováno pomocí absolutního pozicování, jehož rozměry a posun jsou dynamicky přepočítávány pomocí CSS funkce `calc()`.

7.4.2 Orchestrace a animace grafových struktur

Jak již bylo uvedeno v návrhu, pro vykreslování stromů a dalších datových struktur byla zvolena open-source knihovna `vis-network`. Abstrakci nad jejím API poskytuje `DataStructureAnimator`.

Největší výzvou při plynulé animaci struktur bylo zamezení takzvanému „blikání“ grafu. Pokud by se při každém logickém kroku knihovně podstrčila zcela nová datová sada uzlů, knihovna by zahodila předchozí fyzikální model a celý strom by z ničeho skokově překreslila. `Animator` proto implementuje vlastní mechanismus pro výpočet rozdílů. Při obdržení nového snímku stavu ze třídy `StepHandler` jej porovná s aktuálním stavem na plátně. Následně volá nízkoúrovňové aktualizací metody (jako `update`, `add` či `remove`) pouze nad těmi uzly a hranami, u kterých došlo k logické změně. Tím je zaručeno, že knihovna interpoluje pohyb prvků do plynulé animace.

7.4.3 Implementace vlastní anotační vrstvy

Knihovna `vis-network` disponuje pokročilým fyzikálním a hierarchickým layout enginem. Ten je výhodný pro automatické rozmísťování uzlů, avšak přináší zásadní nevýhodu při potřebě přidat do grafu dodatečné informace. Vložení doplňkových uzlů za účelem zobrazení textových ukazatelů by nevratně narušilo výpočet rozvržení a zničilo by tvar stromu.

Tento problém byl vyřešen implementací nezávislé překryvné vrstvy (*overlay canvas*), kterou obsluhuje třída `DataStructureAnnotator`. Jak ukazuje Výpis 14, tato třída se napojuje na událost `afterDrawing` vyvolávanou přímo vykreslovacím jádrem knihovny. Pokaždé, když knihovna dokončí překreslení, Anotátor nad grafem na vlastním průhledném plátně vykreslí statické popisky.

Tato nezávislá vrstva umožňuje vykreslovat hned několik typů anotací:

- **InfoAnnotation:** Dynamické textové štítky umístěné pod existujícími uzly, sloužící typicky pro zobrazení textu popisujícího aktuální krok.
- **ValueAnnotation:** Speciální štítky pro dočasné hodnoty, které se nevejdou přímo do těla uzlu, nebo znázorňují hodnoty v procesu přesunu.
- **IdAnnotation:** Nízkoúrovňové zobrazení unikátních identifikátorů uzlů, vykreslované pouze v režimu ladění pro vývojáře.
- **LegsAnnotation:** Vykreslování „nožiček“ (prázdných hran) indikujících prázdné větve listů.

```

1  // 1. Napojení vrstvy na vykreslovací cyklus knihovny vis-network
2  this.network.on("afterDrawing", () => {
3      this.annotator.redrawCanvas();
4  });
5
6  // 2. Zjednodušená definice obsluhy vlastního plátna
7  export class DataStructureAnnotator {
8      private ctx: CanvasRenderingContext2D;
9      private shown: boolean = true;
10
11     // ... pole registrovaných anotací
12
13     public redrawCanvas(): void {
14         if (!this.shown) return;
15
16         this.ctx.clearRect(0, 0, this.ctx.canvas.width, this.ctx.canvas.height);
17
18         this.idAnnotations.forEach((a) => a.draw(this.ctx));
19         this.legsAnnotations.forEach((a) => a.draw(this.ctx));
20         this.infoAnnotations.forEach((a) => a.draw(this.ctx));
21         this.valueAnnotations.forEach((a) => a.draw(this.ctx));
22     }
23 }

```

Výpis 14: Napojení vlastní vrstvy DataStructureAnnotator na událost vykreslení sítě

7.4.4 Technologické limity vizualizace B-stromů

Navzdory robustnímu systému tříd `Annotator` a `Animator` existují scénáře, ve kterých zvolená knihovna naráží na své nepřekročitelné architektonické limity. Tímto scénářem jsou především B-stromy.

Hierarchický auto-layout knihovny `vis-network` interně počítá se zhruba konstantní šířkou všech uzlů a udržuje pevnou horizontální vzdálenost mezi sourozenci v dané úrovni stromu. Podstata B-stromů však spočívá v tom, že uzel může dynamicky měnit svou velikost podle toho, kolik klíčů aktuálně obsahuje. Tato vlastnost způsobuje, že při zvýšení počtu klíčů uzlu nedokáže hierarchický layout správně přepočítat relativní vzdálenosti v dané větvi. Tento stav ve specifických případech vede k vizuálnímu překryvu sousedních prvků. Zmíněný nedostatek nelze plně korigovat bez opuštění použité grafické knihovny a naprogramování vlastního vykreslovacího modelu, a představuje tak akceptované omezení implementace. Ve výsledné aplikaci je proto mezi uzly nastavená pevná minimální vzdálenost, která eliminuje překryvy na úkor občasných větších mezer mezi listy stromu.

Zvolená knihovna rovněž neumožňuje explicitně definovat pořadí potomků uzlu. Tento problém vyvstal až během implementace B-stromu, neboť u binárních stromů jsou uzly implicitně řazeny v pořadí jejich přidání do grafu. U struktur s více než dvěma potomky však toto chování neplatí a algoritmus rozložení namísto toho sourozence seřazuje primárně podle jejich identifikátoru (`id`). Pro korektní zobrazení B-stromu tak bylo nutné při každém přidání či odebrání uzlu kompletně

přepočítat identifikátory a přiřadit je na základě průchodu *post-order*. Aby zároveň zůstala plynulá animace přechodů, byla změna indexů doplněna o překladové pole mapující relace mezi původními a novými identifikátory jednotlivých uzlů.

7.5 Automatizace nasazení

Pro zajištění kontinuální dostupnosti aplikace pro koncové uživatele a eliminaci chyb při manuálním nasazování byl implementován plně automatizovaný proces sestavení a publikace. Tento proces využívá platformu GitHub Actions a je definován v konfiguračním souboru `deploy.yml`.

Ačkoliv je aplikace technicky nasazena v infrastruktuře GitHub Pages, pro zvýšení důvěryhodnosti a snadnější zapamatovatelnost byla v rámci implementace nastavena vlastní doména druhého řádu odpovídající zvolenému jménu nástroje. V nastavení DNS záznamů a v konfiguračním rozhraní platformy GitHub byla vytvořena subdoména, která pomocí záznamu CNAME směřuje na servery poskytovatele.

Aplikace je tak vždy dostupná na nastavené adrese¹, přičemž veškeré aktualizace jsou automaticky publikovány bez nutnosti manuálního zásahu.

7.6 Uživatelské testování a iterativní vývoj

Během vývoje aplikace proběhlo uživatelské testování cílovou skupinou, kterou tvořili vysokoškolští studenti. Získané podněty a nalezené nedostatky byly systematicky evidovány pomocí nástroje GitHub Issues. Samotné testování potvrdilo vysokou úroveň navrženého uživatelského prostředí. Uživatelé od počátku intuitivně chápali význam jednotlivých ovládacích prvků a nevykazovali žádné problémy s navigací či pochopením logiky systému.

I přes velmi pozitivní hodnocení celkové ergonomie vedla zpětná vazba k implementaci řady funkčních vylepšení. V oblasti simulace datových struktur se vývoj zaměřil na pokrytí okrajových případů (*edge-cases*), a to primárně u složitějších operací v B-stromech. Do aplikace přibyla možnost volby mezi vygenerováním zcela prázdné, nebo naopak předvyplněné datové struktury. Rozšíření se dočkala také srovnávací matice třídících algoritmů, kam byla doplněna počítadla provedených porovnání a výměn včetně celkového pořadí. Souběžně probíhaly průběžné opravy drobných chyb a průběžné zvyšování spolehlivosti chodu aplikace.

Další iterace se věnovaly úpravám zobrazení a výukového obsahu. Vizuální stránka prošla revizí responzivity a ovládací prvky datových struktur byly pro lepší přehlednost rozděleny do samostatných logických sekcí. Z produkční verze byly skryty vývojářské nástroje, jako jsou detailní informační výpisy či specifické krokovací prvky určené výhradně pro ladění. Úpravou prošly rovněž samotné studijní materiály, u nichž došlo k detailnějšímu rozvedení náročnějších pasáží pro maximální srozumitelnost probírané teorie.

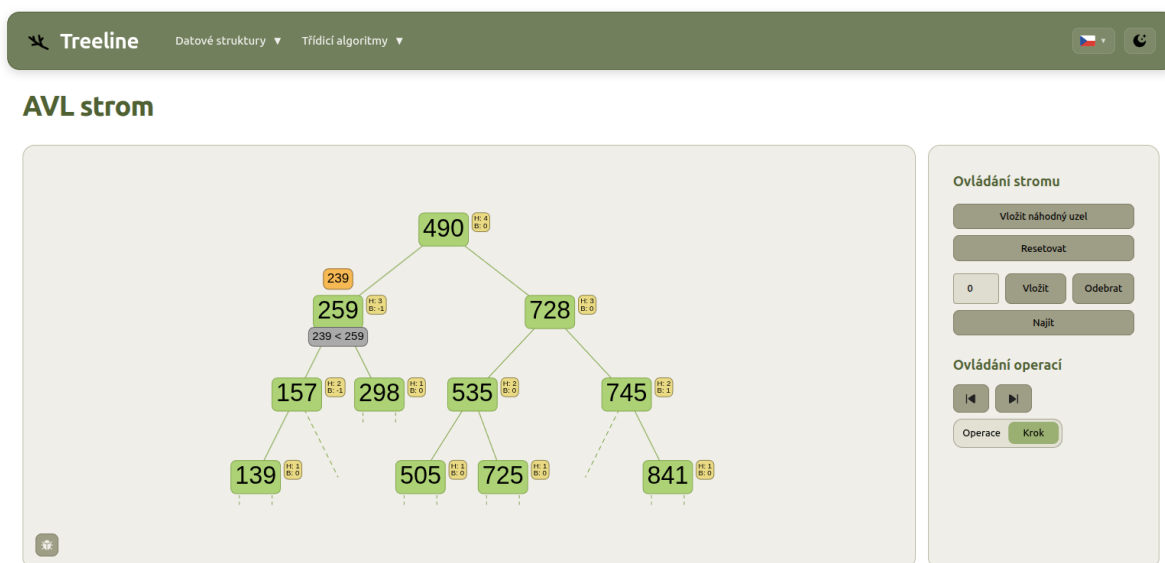
¹<https://treeline.kkobari.eu>

7.7 Uživatelské rozhraní a další funkcionality

Na závěr implementace tato kapitola demonstruje, jak se navržené abstrakce, událostní modely a vizualizační techniky spojují do finálního produktu. Dále také představuje pokročilé funkce, které zvyšují ergonomii a edukativní hodnotu aplikace.

7.7.1 Prezentace vizualizačních modulů

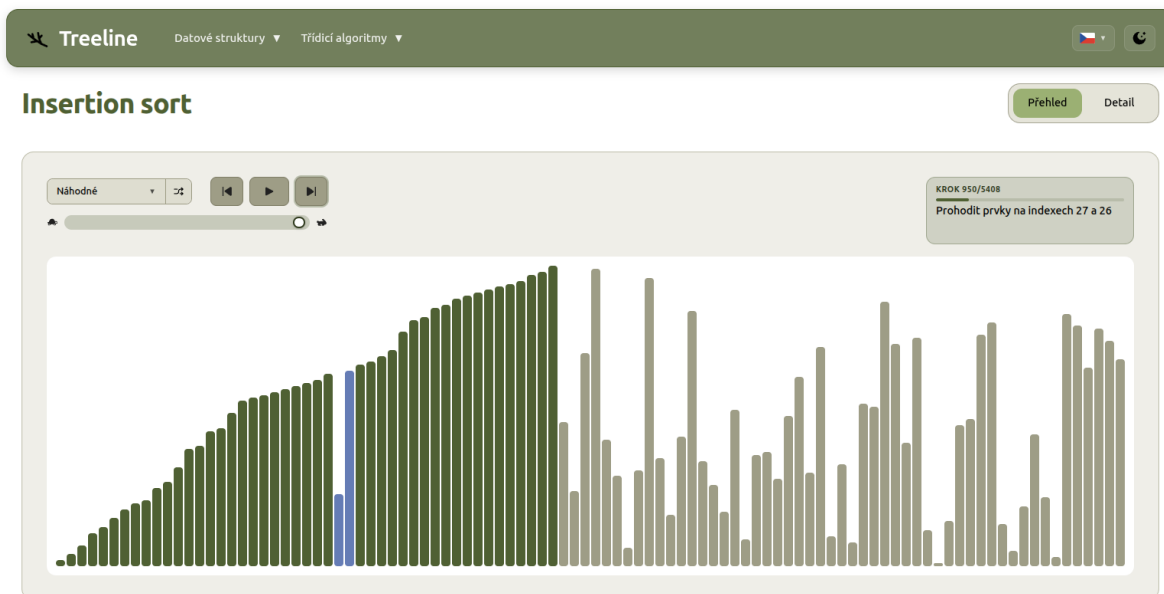
Jednotlivé vizualizační moduly jsou přístupné z hlavní navigace, která je implementována jako horizontální menu v horní části obrazovky. Barva a styl stránek jsou konzistentní napříč celou aplikací a jsou v souladu s barevnou paletou, která byla definována v Kapitole 6.4. V rámci implementace byl pro výukovou aplikaci zvolen název *Treeline*, který se objevuje v logu a dalších vizuálních prvcích.



Obrázek 7.1: Uživatelské rozhraní pro interaktivní vizualizaci datových struktur

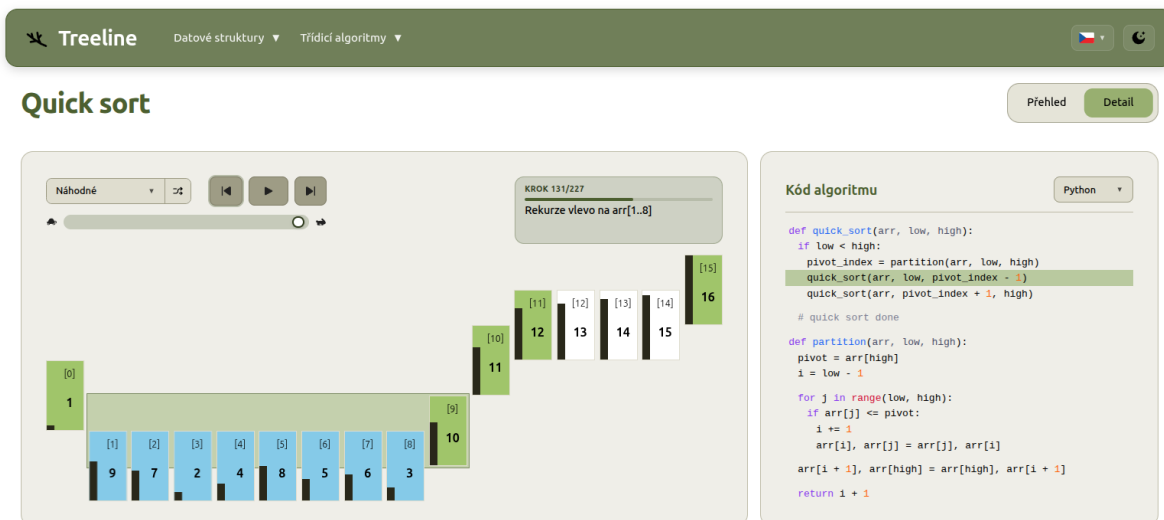
Na Obrázku 7.1 je zachycena stránka pro simulaci datových struktur (na příkladu AVL stromu). Pro graf je vyhrazena většina horizontálního prostoru, zatímco ovládací prvky pro manipulaci se strukturou (vkládání, mazání, hledání) jsou umístěny v pravé postranní liště.

Doména třídících algoritmů je rozdělena na dva vzájemně se doplňující pohledy. Obecný přehled (Obrázek 7.2) nabízí sloupcovou vizualizaci, která je ideální pro demonstraci celkového chování algoritmu nad větším množstvím dat. V rámci běhu algoritmu se dynamicky zvýrazňují porovnávané a přesouvané prvky, kterých je v tomto přehledu 100. Uživatel může postup krokovat, nebo spustit automatické přehrávání s nastavitelnou rychlostí. Má také možnost zvolit si různé vstupní datové sady, jako jsou náhodné, téměř setříděné či reverzní pole.



Obrázek 7.2: Obecný přehled třídícího algoritmu s využitím sloupkové vizualizace

Detailní pohled (Obrázek 7.3), zde demonstrováný na algoritmu Quick Sort, naopak poskytuje hluboký vhled do mikro-operací, vizualizuje úrovně rekurze pomocí dvourozměrné mřížky a dynamicky zvýrazňuje odpovídající řádky pseudokódu. Pole je zde zmenšeno na pouhých 16 prvků, uživatelé tak mají možnost sledovat každou operaci ve větším detailu.

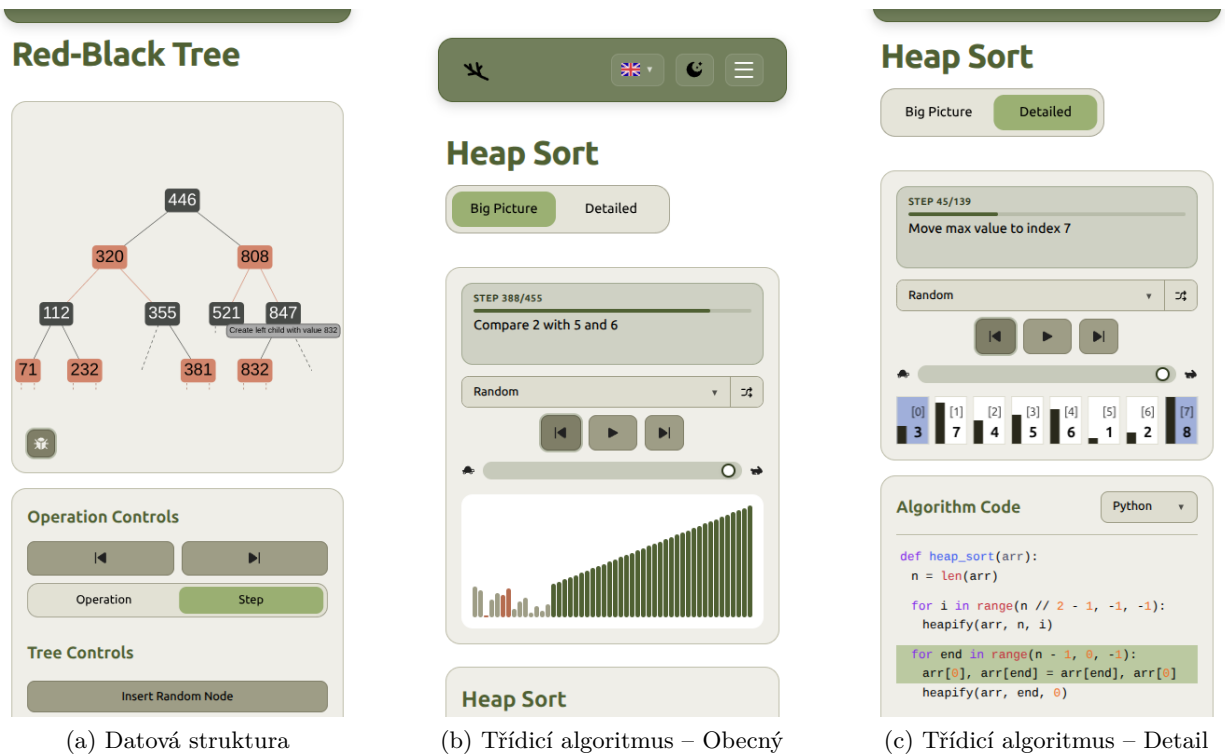


Obrázek 7.3: Detailní pohled rekurzivního algoritmu Quick Sort

7.7.2 Přístupnost a adaptabilita prostředí

Moderní výuková aplikace musí být přístupná širokému spektru uživatelů a zařízení. Adaptabilita prostředí stojí na třech hlavních pilířích: responzivitě, vizuálních motivech a mezinárodní lokalizaci.

Pro zajištění responzivity a podpory tmavého režimu (Dark Mode) byl využit CSS framework Tailwind. Pomocí jeho utilitních tříd se rozložení prvků dynamicky přeskládává v závislosti na velikosti obrazovky a rozhraní tak umožňuje plnohodnotné používání aplikace i na mobilních zařízeních, jak je ukázáno na Obrázku 7.4. Mezi úpravy mimo jiné patří přechod z horizontálního menu na hamburger menu, vertikální rozložení ovládacích prvků a zmenšení datové sady pro třídící algoritmy na 50 a 8 prvků v obecném a detailním pohledu, aby se zachovala přehlednost na menších obrazovkách.



Obrázek 7.4: Ukázka responzivního chování na mobilním zařízení

Aplikace plně respektuje preferenci uživatele ohledně barevného motivu. Inverze barev je navržena tak, aby sémantika barev (např. varovná červená, potvrzovací zelená) zůstala zachována a plně kontrastní v obou režimech (Obrázek 7.5).

Třetím pilířem je plná internacionalizace (i18n). Aplikace je bilingvní a podporuje plynulé přepínání mezi českým a anglickým jazykem. Z architektonického hlediska bylo největší výzvou zajištění překladů uvnitř elementu Canvas, který na rozdíl od HTML DOM nereaguje na změny jazyka automaticky. Tento problém řeší přesun klíčů pro překlad z HTML do datových modelů, které jsou následně zpracovávány vizualizačními komponentami.



Obrázek 7.5: Podpora tmavého režimu s plným zachováním kontrastu a sémantiky barev

7.7.3 Server-Side Rendering výukových materiálů

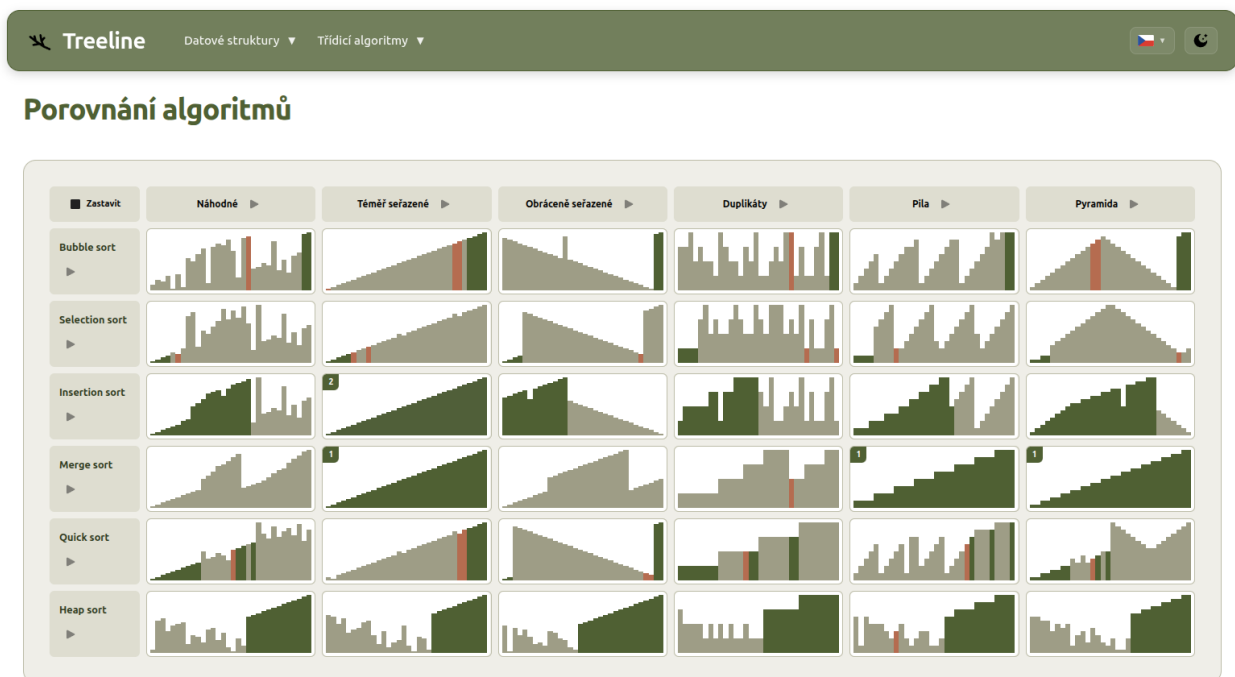
Kromě interaktivních vizualizací obsahuje aplikace také rozsáhlé textové materiály. Tyto texty jsou z vývojářského hlediska udržovány ve formátu Markdown (.md), což usnadňuje jejich editaci a formátování.

Aby se předešlo zatěžování klientského prohlížeče parsováním velkého množství textu a zároveň se maximalizovala optimalizace pro vyhledávače (SEO), využívá aplikace výhod Server-Side Renderingu (SSR). Logika načítání dynamicky identifikuje požadovaný soubor na základě zvolené struktury a aktuálního jazyka (např. `rb_tree.cs.md`). Tento soubor je následně asynchronně načten ze souborového systému serveru a pomocí knihovny `marked` převeden do HTML podoby. Zásadní je zde implementace paměťové cache na straně serveru, která zajišťuje, že se každý soubor parsuje pouze při prvním požadavku. Díky tomuto mechanismu se dramaticky snižuje latence sítě a urychluje vykreslení stránky, protože uživateli je ze serveru odesláno již hotové, zformátované HTML.

7.7.4 Komparace algoritmů v reálném čase

Poslední ukázkou robustnosti a nezávislosti navržených komponent je modul pro komparaci algoritmů (Obrázek 7.6). Tato obrazovka umožňuje uživateli spustit souběžný běh několika různých třídících algoritmů nad stejnými (nebo i odlišnými) datovými sadami a v reálném čase sledovat, jak se liší jejich časová složitost a efektivita.

Tato funkcionality je umožněna díky skutečnosti, že každý vizualizační blok na obrazovce si instancuje svůj vlastní izolovaný `StepManager`. Jeden centrální časovač následně distribuuje impulsy do všech aktivních manažerů, které inkrementují svůj stav zcela nezávisle na ostatních. Výsledkem je vizuálně atraktivní porovnání algoritmů, které empiricky dokazuje teoretické předpoklady o asymptotické složitosti, a zároveň slouží jako zajímavý příklad pro uživatele.



Obrázek 7.6: Komparační zobrazení více třídících algoritmů současně

Kapitola 8

Závěr

Hlavním cílem této diplomové práce bylo navrhnout, zdokumentovat a naimplementovat interaktivní výukovou aplikaci, která studentům usnadní pochopení datových struktur a třídících algoritmů. Tento cíl byl úspěšně splněn vytvořením komplexního webového nástroje *Treeline*, který propojuje teoretické studijní materiály s moderními přístupy v softwarovém inženýrství.

Z inženýrského hlediska je největším přínosem práce návrh modulární a škálovatelné architektury. Aplikace striktně odděluje aplikační logiku algoritmů od prezentační vrstvy. Toho bylo dosaženo implementací událostmi řízeného modelu pro asynchronní operace nad datovými strukturami a mechanismem generování předpočítaných stavů pro plynulou animaci třídících algoritmů. Úspěšně se podařilo překonat i technologické limity použitých knihoven třetích stran, například vývojem vlastní nezávislé anotační vrstvy nad plátnem **vis-network** a adaptivním výpočtem překryvů v rekurzivních algoritmech. Z uživatelského hlediska aplikace nabízí plně responzivní prostředí, dynamické přepínání programovacích jazyků pseudokódu, podporu tmavého režimu a plynulou mezinárodní lokalizaci jak v interaktivních prvcích stránky a jednotlivých vizualizacích, tak v teoretických materiálech, jež vizualizace doplňují.

Zásadní výhodou výsledného řešení je jeho infrastruktura. Díky využití generování statických stránek (SSG), automatizované CI/CD linky přes GitHub Actions a hostingu prostřednictvím GitHub Pages je aplikace zcela nezávislá na komplexních backendových serverech. Tento bezúdržbový model provozu v kombinaci s nasazením na vlastní doménu zajišťuje dlouhodobou udržitelnost a dostupnost projektu do budoucnosti s minimálními provozními náklady – přístup vhodný pro aplikaci zaměřenou na vzdělávání a jednoduchou integraci do výuky. Aplikace má tak velký potenciál stát se nejen cenným podpůrným nástrojem pro oficiální univerzitní výuku předmětů zaměřených na algoritmy, ale může stabilně sloužit i široké veřejnosti a studentům mimo akademickou půdu.

Navržená architektura představuje pevný základ, který je připraven pro další rozšiřování. V rámci budoucího vývoje se nabízí implementace dalších teoretických konceptů, pro které je stávající systém již připraven. Patří mezi ně zejména doména grafových algoritmů (např. procházení grafu do šířky a do hloubky, hledání nejkratší cesty, konstrukce minimální kostry či toky v sítích), kom-

plexnější datové struktury jako hašovací tabulky, nebo různé metody průchodu stromem (inorder, preorder, postorder).

Dalším významným směrem pro budoucí rozvoj je rozšíření aplikace o evaluační modul. Začlenění interaktivních testů, kvízů a praktických cvičení by studentům umožnilo ověřit si, zda danou problematiku po zhlédnutí vizualizace skutečně plně pochopili, a poskytlo by jim cennou zpětnou vazbu.

Výsledná aplikace dokazuje, že i abstraktní a obtížně uchopitelné koncepty teoretické informatiky lze prezentovat vizuálně atraktivní, moderní a přístupnou formou.

Literatura

1. LEVITIN, Anany. *The Design & Analysis of Algorithms*. 2nd ed. Boston: Pearson Addison-Wesley, 2007. ISBN 978-0-321-36413-5.
2. CORMEN, Thomas H.; LEISERSON, Charles Eric; RIVEST, Ronald L.; STEIN, Clifford. *Introduction to algorithms*. 2nd ed. Boston: McGraw-Hill Book Company, 2001. ISBN 978-0262032933.
3. MORIN, Pat. *Open data structures: an introduction*. Edmonton, Alberta: AU Press, 2013. ISBN 978-1-927356-38-8.
4. SEDGEWICK, Robert; WAYNE, Kevin Daniel. *Algorithms*. Fourth edition. Boston: Addison-Wesley, 2011. ISBN 978-0-321-57351-3.
5. URQUIZA-FUENTES, Jaime; VELÁZQUEZ-ITURBIDE, J. Ángel. A Survey of Successful Evaluations of Program Visualization and Algorithm Animation Systems. *ACM Transactions on Computing Education*. 2009, roč. 9, č. 2, s. 1–22.
6. KOGAN, G. A.; CHASSIDIM, Hadas; RABAEV, Irina. The efficacy of animation and visualization in teaching data structures: a case study. *Educational Technology Research and Development*. 2024.
7. FOUH, Eric; AKBAR, Monika; SHAFFER, Clifford A. The Role of Visualization in Computer Science Education. *Computers in the Schools*. 2012, roč. 29, č. 1-2, s. 95–117.
8. KOVÁČ, Jakub. *Gnarley Trees* [online]. 2011. [cit. 2026-01-31]. Dostupné z: <https://kubokovac.eu/gnarley-trees/>.
9. GALLES, David. *Data Structure Visualizations* [online]. University of San Francisco, 2011. [cit. 2026-01-31]. Dostupné z: <https://www.cs.usfca.edu/~galles/visualization/>.
10. DICKEN, Ben. *B plus tree* [online]. PlanetScale, 2024. [cit. 2026-01-31]. Dostupné z: <https://bplustree.app/>.
11. SCANU, Daniel. *Sort Visualizer* [online]. 2021. [cit. 2026-01-31]. Dostupné z: <https://www.sortvisualizer.com/>.
12. HALIM, Steven. *VisuAlgo* [online]. National University of Singapore, 2011. [cit. 2026-01-31]. Dostupné z: <https://visualgo.net/en>.

13. *Sorting Algorithms Animations* [online]. Toptal, 2010. [cit. 2026-01-31]. Dostupné z: <https://www.toptal.com/developers/sorting-algorithms>.
14. JOSEPH, Renien John. Single Page Application and Canvas Drawing. *International journal of Web & Semantic Technology*. 2015, roč. 6, č. 1, s. 29–37.
15. KALUŽA, Marin; VUKELIĆ, Bernard. Comparison of front-end frameworks for web applications development. *Zbornik Veleučilišta u Rijeci*. 2018, roč. 6, č. 1, s. 261–282.
16. ALAA, Alkhaled; OSTAFIE, Cosmin. Rendering Matters: SSR vs CSR in Modern Web Development. *Jonköping University Library (Jönköping University)*. 2025.
17. KUMAR, Ram L.; PRIYA, Prerna. SEO Optimization in Web Development: How Next.js Helps. *International Journal for Research in Applied Science and Engineering Technology*. 2025, roč. 13, č. 5, s. 7011–7018.
18. CHEN, K. Improving Front-end Performance through Modular Rendering and Adaptive Hydration (MRAH) in React Applications. *ArXiv.org*. 2025.
19. META PLATFORMS, INC. *React* [online]. 2026. [cit. 2026-03-08]. Dostupné z: <https://react.dev/>.
20. GOOGLE LLC. *Angular* [online]. 2026. [cit. 2026-03-08]. Dostupné z: <https://angular.dev/>.
21. YOU, Evan; VUE COMMUNITY. *Vue.js* [online]. 2026. [cit. 2026-03-08]. Dostupné z: <https://vuejs.org/>.
22. SVELTE CONTRIBUTORS. *Svelte* [online]. 2026. [cit. 2026-03-08]. Dostupné z: <https://svelte.dev/>.
23. BOSTOCK, Mike. *D3.js* [online]. 2026. [cit. 2026-03-09]. Dostupné z: <https://d3js.org/>.
24. CYTOSCAPE CONSORTIUM. *Cytoscape.js* [online]. 2026. [cit. 2026-03-09]. Dostupné z: <https://js.cytoscape.org/>.
25. ALMENDE, B.V.; VIS.JS CONTRIBUTORS. *Vis-network* [online]. 2026. [cit. 2026-03-09]. Dostupné z: <https://visjs.org/>.
26. GITHUB, INC. *GitHub Actions* [online]. 2026. [cit. 2026-03-10]. Dostupné z: <https://docs.github.com/en/actions>.
27. GITHUB, INC. *GitHub Pages* [online]. 2026. [cit. 2026-03-10]. Dostupné z: <https://docs.github.com/en/pages>.